Copy of Copy_of_Copy_of_Machine_Learning_Project

December 30, 2025

1 Data Reading & Viewing

This section provides a summary of the DataFrame's structure, including column names, the number of non-null values for each column, and their respective data types. This helps in understanding data completeness and types at a glance.

1.1 Data Reading

[ ]:
```
import pandas as pd

# Read the Excel file into a DataFrame
df = pd.read_csv('/content/Term_Project_Dataset_20K.csv')

# Display the first 5 rows of the DataFrame
display(df.head())
```

```
    age  gender  parent_income  num_siblings  family_support  \
0  24.0  Female        22678.0           0.0             1.0
1  21.0    Male        17905.0           3.0             2.0
2  28.0    Male            NaN           0.0             0.0
3  25.0  Female        29391.0           4.0             4.0
4  22.0    Male        15522.0           0.0             4.0

   commute_time_min part_time_job  previous_gpa  num_failed_courses  \
0              95.0            No          2.22                 3.0
1               0.0           Yes          3.78                 1.0
2              67.0            No           NaN                 4.0
3              83.0           Yes          1.84                 4.0
4              58.0           Yes          3.38                 0.0

   high_school_grade  …  concentration_level  exam_anxiety_level  \
0               99.7  …                  7.0                 6.0
1               75.7  …                  3.0                 3.0
2               93.4  …                  1.0                 8.0
3               69.7  …                  6.0                10.0
4               62.2  …                 10.0                 5.0

   course_difficulty_rating  teacher_experience_years  class_size  \
```

```
0                       2.0                     11.0        95.0
1                       1.0                      NaN       148.0
2                       1.0                     32.0        55.0
3                       5.0                     30.0        42.0
4                       3.0                     31.0       123.0

   num_prerequisites  course_type  final_score  final_grade  pass_fail
0                3.0     Elective          NaN            C       Pass
1                2.0     Elective         81.7            C       Pass
2                3.0     Elective         77.3            B       Pass
3                1.0    Mandatory         78.5            B       Pass
4                1.0    Mandatory         60.3            D       Pass

[5 rows x 41 columns]
```

[ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 41 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   age                           19400 non-null  float64
 1   gender                        19400 non-null  object
 2   parent_income                 19400 non-null  float64
 3   num_siblings                  19400 non-null  float64
 4   family_support                19400 non-null  float64
 5   commute_time_min              19400 non-null  float64
 6   part_time_job                 19400 non-null  object
 7   previous_gpa                  19400 non-null  float64
 8   num_failed_courses            19400 non-null  float64
 9   high_school_grade             19400 non-null  float64
 10  math_background_score         19400 non-null  float64
 11  language_background_score     19400 non-null  float64
 12  science_background_score      19400 non-null  float64
 13  prior_semester_credits        19400 non-null  float64
 14  study_hours_last_semester     19400 non-null  float64
 15  past_attendance_rate          19400 non-null  float64
 16  academic_warnings_count       19400 non-null  float64
 17  lecture_attendance_rate       19400 non-null  float64
 18  assignment_submission_rate    19400 non-null  float64
 19  quiz_avg_score                19400 non-null  float64
 20  midterm_score                 19400 non-null  float64
 21  lab_participation_rate        19400 non-null  float64
 22  online_portal_usage_minutes   19400 non-null  float64
 23  group_project_activity        19400 non-null  float64
 24  library_visits_per_month      19400 non-null  float64
 25  discussion_forum_posts        19400 non-null  float64
```

```
 26  lateness_count               19400 non-null  float64
 27  stress_level                 19400 non-null  float64
 28  sleep_hours                  19400 non-null  float64
 29  motivation_level             19400 non-null  float64
 30  study_time_per_week          19400 non-null  float64
 31  concentration_level          19400 non-null  float64
 32  exam_anxiety_level           19400 non-null  float64
 33  course_difficulty_rating     19400 non-null  float64
 34  teacher_experience_years     19400 non-null  float64
 35  class_size                   19400 non-null  float64
 36  num_prerequisites            19400 non-null  float64
 37  course_type                  19400 non-null  object
 38  final_score                  19400 non-null  float64
 39  final_grade                  19400 non-null  object
 40  pass_fail                    19400 non-null  object
dtypes: float64(36), object(5)
memory usage: 6.3+ MB
```

1.2 Data Overview: Descriptive Statistics

Here, `df.describe()` is used to generate descriptive statistics that summarize the central tendency, dispersion, and shape of the numerical columns' distribution. This can highlight potential issues such as negative values, extreme ranges, or outliers.

[ ]:
```
display(df.describe())
```

```
                age  parent_income  num_siblings  family_support  \
count  19400.000000   19400.000000  19400.000000    19400.000000
mean      23.478918   18290.515928      2.510361        2.502680
std        3.452709   30896.118002      1.715978        1.707515
min       18.000000    2000.000000      0.000000        0.000000
25%       20.000000    9087.000000      1.000000        1.000000
50%       23.000000   16102.000000      3.000000        3.000000
75%       26.000000   23108.500000      4.000000        4.000000
max       29.000000  596760.000000      5.000000        5.000000

       commute_time_min  previous_gpa  num_failed_courses  high_school_grade  \
count      19400.000000  19400.000000        19400.000000       19400.000000
mean          59.318144      2.491322            1.996443          74.902892
std           34.796772      0.869089            1.413680          14.425122
min            0.000000      1.000000            0.000000          50.000000
25%           29.000000      1.730000            1.000000          62.400000
50%           59.000000      2.500000            2.000000          74.800000
75%           90.000000      3.240000            3.000000          87.400000
max          119.000000      4.000000            4.000000         100.000000

       math_background_score  language_background_score  …   sleep_hours  \
count           19400.000000               19400.000000  …  19400.000000
```

```
mean                49.352577                  49.412268  …      6.453149
std                 28.890298                  28.842990  …      2.148505
min                  0.000000                   0.000000  …     -5.000000
25%                 24.000000                  24.000000  …      4.700000
50%                 49.000000                  49.000000  …      6.500000
75%                 74.000000                  75.000000  …      8.300000
max                 99.000000                  99.000000  …     10.000000

       motivation_level  study_time_per_week  concentration_level  \
count      19400.000000         19400.000000         19400.000000
mean           4.980619            19.614691             4.957680
std            3.159022            11.485281             3.160975
min            0.000000             0.000000             0.000000
25%            2.000000            10.000000             2.000000
50%            5.000000            20.000000             5.000000
75%            8.000000            30.000000             8.000000
max           10.000000            39.000000            10.000000

       exam_anxiety_level  course_difficulty_rating  teacher_experience_years  \
count        19400.000000              19400.000000              19400.000000
mean             5.013299                  2.997526                 20.072680
std              3.180374                  1.409721                 11.294943
min              0.000000                  1.000000                  1.000000
25%              2.000000                  2.000000                 10.000000
50%              5.000000                  3.000000                 20.000000
75%              8.000000                  4.000000                 30.000000
max             10.000000                  5.000000                 39.000000

         class_size  num_prerequisites   final_score
count  19400.000000       19400.000000  19400.000000
mean     109.904381           1.996546     66.542747
std       52.009036           1.412659     11.050014
min       20.000000           0.000000     25.700000
25%       65.000000           1.000000     58.800000
50%      110.000000           2.000000     66.500000
75%      155.000000           3.000000     74.300000
max      199.000000           4.000000    100.000000

[8 rows x 36 columns]
```

[ ]:
```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

numerical_cols = df.select_dtypes(include=['number']).columns

batch_size = 9  # Number of plots per figure (3x3 grid)
```

```
num_batches = int(np.ceil(len(numerical_cols) / batch_size))

print("--- Generating Box Plots and Outlier Information (IQR Method) ---")

for batch_idx in range(num_batches):
    start_idx = batch_idx * batch_size
    end_idx = min((batch_idx + 1) * batch_size, len(numerical_cols))
    current_batch_cols = numerical_cols[start_idx:end_idx]

    # Calculate rows needed for the current batch (up to 3x3)
    rows = int(np.ceil(len(current_batch_cols) / 3))
    cols = 3

    plt.figure(figsize=(18, rows * 5)) # Adjust figure height dynamically
    plt.suptitle(f'Box Plots of Numerical Columns (Batch {batch_idx + 1}/
 ↪{num_batches})', y=1.02, fontsize=16)

    for i, col in enumerate(current_batch_cols):
        plt.subplot(rows, cols, i + 1)
        sns.boxplot(y=df[col])
        plt.title(f'{col.replace("_", " ").title()}')
        plt.ylabel(col.replace("_", " ").title())

        # Calculate and print outlier information as requested
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
        num_outliers = len(outliers)

        print(f"\n--- Column: {col.replace("_", " ").title()} ---")
        print(f"Q1: {Q1:.2f}")
        print(f"Q3: {Q3:.2f}")
        print(f"IQR: {IQR:.2f}")
        print(f"Lower Bound (Outlier): {lower_bound:.2f}")
        print(f"Upper Bound (Outlier): {upper_bound:.2f}")
        print(f"Number of Outliers: {num_outliers}")

    plt.tight_layout(rect=[0, 0.03, 1, 0.98]) # Adjust layout to make space for␣
 ↪suptitle
    plt.show()
```

```
--- Generating Box Plots and Outlier Information (IQR Method) ---
```

```
--- Column: Age ---
Q1: 20.00
Q3: 26.00
IQR: 6.00
Lower Bound (Outlier): 11.00
Upper Bound (Outlier): 35.00
Number of Outliers: 0

--- Column: Parent Income ---
Q1: 9087.00
Q3: 23108.50
IQR: 14021.50
Lower Bound (Outlier): -11945.25
Upper Bound (Outlier): 44140.75
Number of Outliers: 144

--- Column: Num Siblings ---
Q1: 1.00
Q3: 4.00
IQR: 3.00
Lower Bound (Outlier): -3.50
Upper Bound (Outlier): 8.50
Number of Outliers: 0

--- Column: Family Support ---
Q1: 1.00
Q3: 4.00
IQR: 3.00
Lower Bound (Outlier): -3.50
Upper Bound (Outlier): 8.50
Number of Outliers: 0

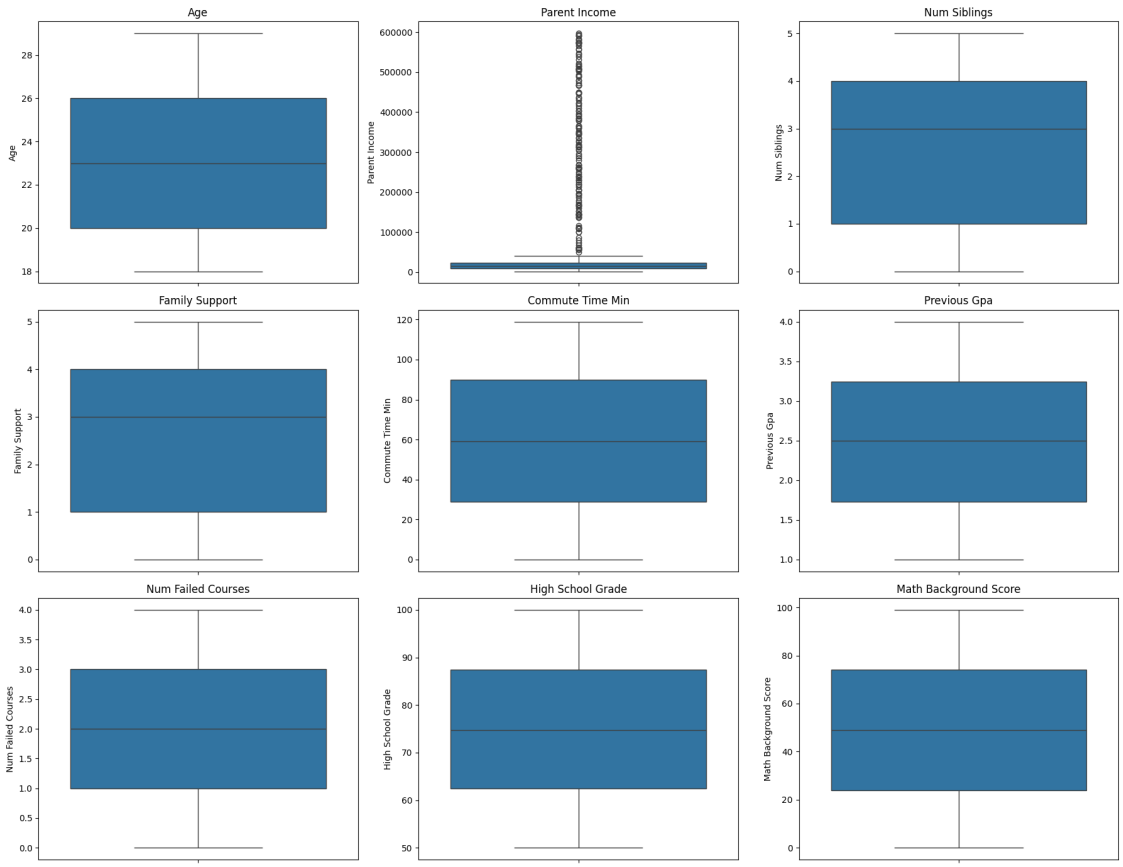
--- Column: Commute Time Min ---
Q1: 29.00
Q3: 90.00
IQR: 61.00
Lower Bound (Outlier): -62.50
Upper Bound (Outlier): 181.50
Number of Outliers: 0

--- Column: Previous Gpa ---
Q1: 1.73
Q3: 3.24
IQR: 1.51
Lower Bound (Outlier): -0.54
Upper Bound (Outlier): 5.51
Number of Outliers: 0
```

```
--- Column: Num Failed Courses ---
Q1: 1.00
Q3: 3.00
IQR: 2.00
Lower Bound (Outlier): -2.00
Upper Bound (Outlier): 6.00
Number of Outliers: 0

--- Column: High School Grade ---
Q1: 62.40
Q3: 87.40
IQR: 25.00
Lower Bound (Outlier): 24.90
Upper Bound (Outlier): 124.90
Number of Outliers: 0

--- Column: Math Background Score ---
Q1: 24.00
Q3: 74.00
IQR: 50.00
Lower Bound (Outlier): -51.00
Upper Bound (Outlier): 149.00
Number of Outliers: 0
```




```
--- Column: Language Background Score ---
Q1: 24.00
Q3: 75.00
IQR: 51.00
Lower Bound (Outlier): -52.50
Upper Bound (Outlier): 151.50
Number of Outliers: 0

--- Column: Science Background Score ---
Q1: 25.00
Q3: 75.00
IQR: 50.00
Lower Bound (Outlier): -50.00
Upper Bound (Outlier): 150.00
Number of Outliers: 0

--- Column: Prior Semester Credits ---
```

```
Q1: 14.00
Q3: 18.00
IQR: 4.00
Lower Bound (Outlier): 8.00
Upper Bound (Outlier): 24.00
Number of Outliers: 0

--- Column: Study Hours Last Semester ---
Q1: 57.00
Q3: 153.25
IQR: 96.25
Lower Bound (Outlier): -87.38
Upper Bound (Outlier): 297.62
Number of Outliers: 144

--- Column: Past Attendance Rate ---
Q1: 62.00
Q3: 87.00
IQR: 25.00
Lower Bound (Outlier): 24.50
Upper Bound (Outlier): 124.50
Number of Outliers: 0

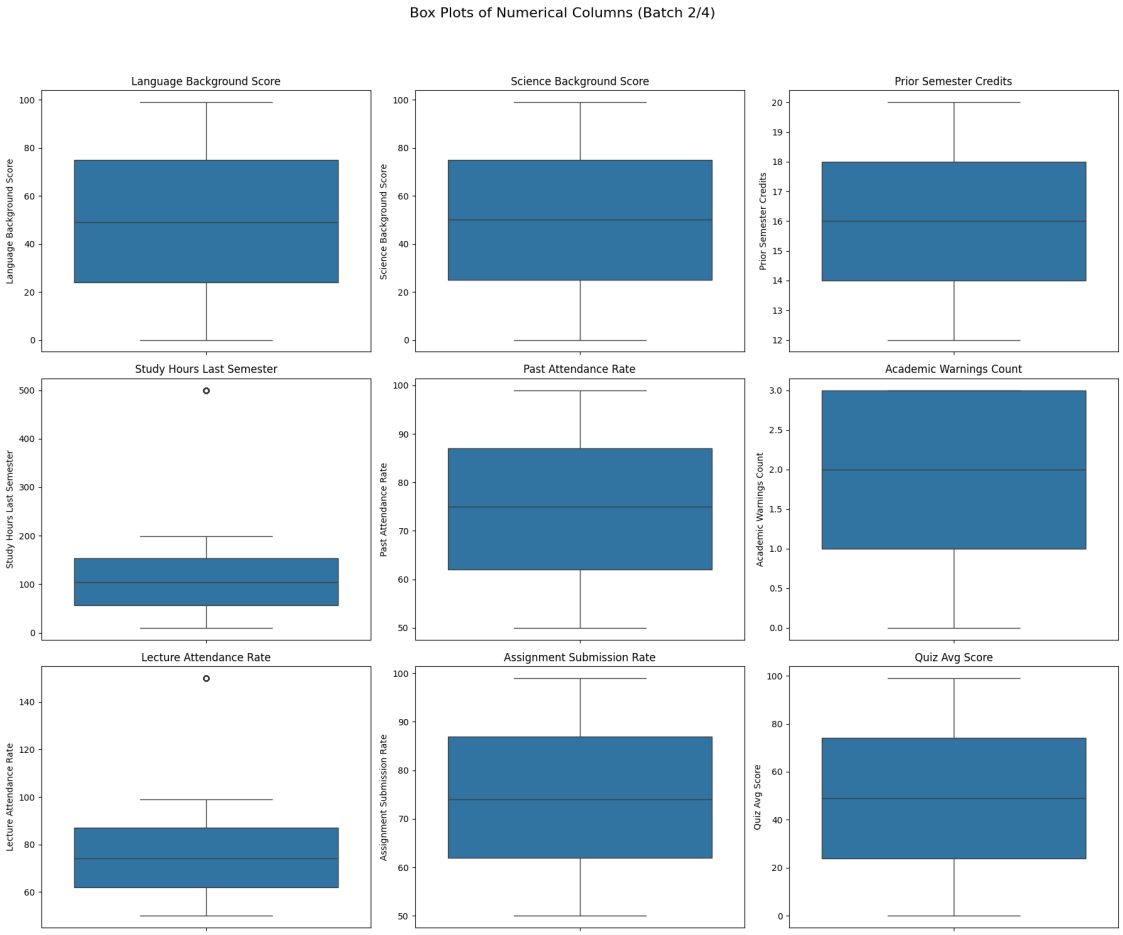
--- Column: Academic Warnings Count ---
Q1: 1.00
Q3: 3.00
IQR: 2.00
Lower Bound (Outlier): -2.00
Upper Bound (Outlier): 6.00
Number of Outliers: 0

--- Column: Lecture Attendance Rate ---
Q1: 62.00
Q3: 87.00
IQR: 25.00
Lower Bound (Outlier): 24.50
Upper Bound (Outlier): 124.50
Number of Outliers: 76

--- Column: Assignment Submission Rate ---
Q1: 62.00
Q3: 87.00
IQR: 25.00
Lower Bound (Outlier): 24.50
Upper Bound (Outlier): 124.50
Number of Outliers: 0

--- Column: Quiz Avg Score ---
```

```
Q1: 24.00
Q3: 74.00
IQR: 50.00
Lower Bound (Outlier): -51.00
Upper Bound (Outlier): 149.00
Number of Outliers: 0
```




```
--- Column: Midterm Score ---
Q1: 25.00
Q3: 75.00
IQR: 50.00
Lower Bound (Outlier): -50.00
Upper Bound (Outlier): 150.00
Number of Outliers: 0

--- Column: Lab Participation Rate ---
Q1: 25.00
Q3: 75.00
```

```
IQR: 50.00
Lower Bound (Outlier): -50.00
Upper Bound (Outlier): 150.00
Number of Outliers: 0

--- Column: Online Portal Usage Minutes ---
Q1: 499.75
Q3: 1509.00
IQR: 1009.25
Lower Bound (Outlier): -1014.12
Upper Bound (Outlier): 3022.88
Number of Outliers: 147

--- Column: Group Project Activity ---
Q1: 25.00
Q3: 75.00
IQR: 50.00
Lower Bound (Outlier): -50.00
Upper Bound (Outlier): 150.00
Number of Outliers: 0

--- Column: Library Visits Per Month ---
Q1: 5.00
Q3: 15.00
IQR: 10.00
Lower Bound (Outlier): -10.00
Upper Bound (Outlier): 30.00
Number of Outliers: 0

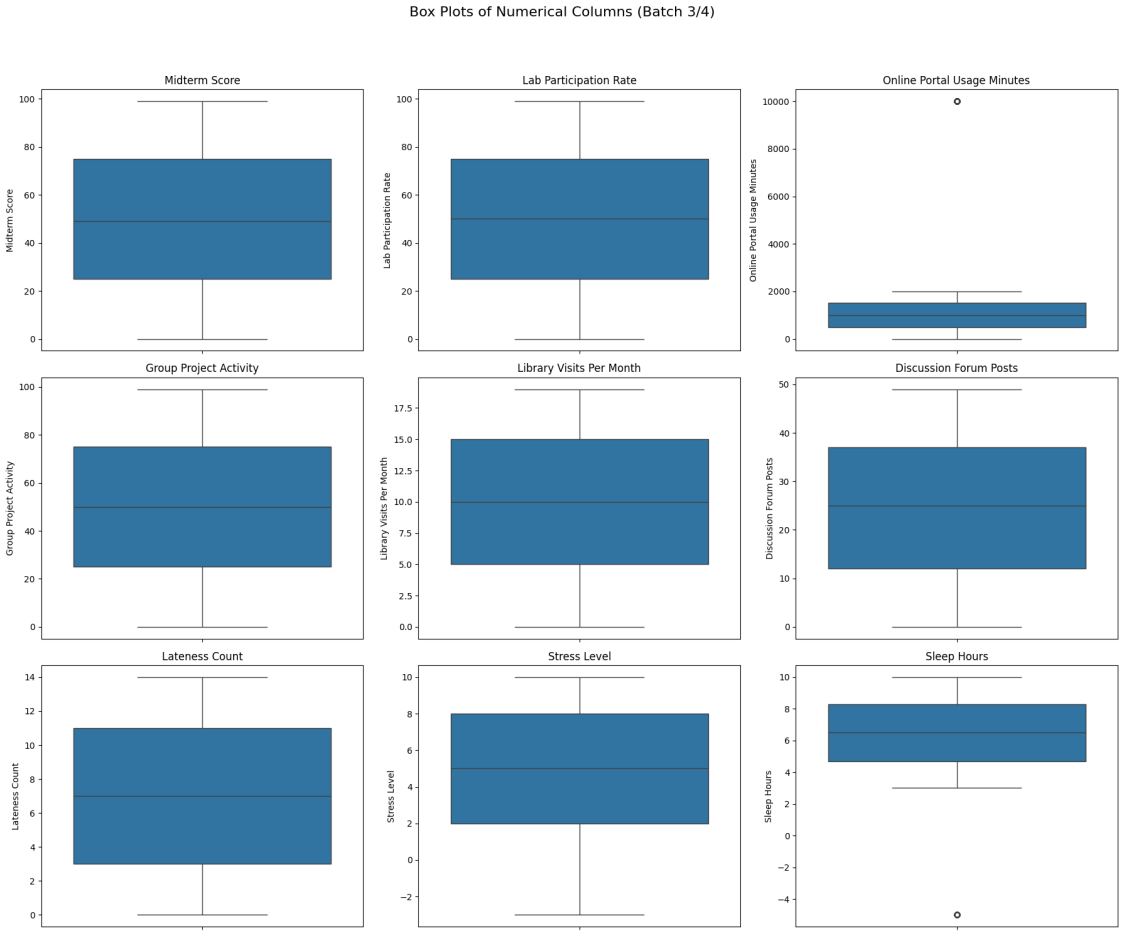
--- Column: Discussion Forum Posts ---
Q1: 12.00
Q3: 37.00
IQR: 25.00
Lower Bound (Outlier): -25.50
Upper Bound (Outlier): 74.50
Number of Outliers: 0

--- Column: Lateness Count ---
Q1: 3.00
Q3: 11.00
IQR: 8.00
Lower Bound (Outlier): -9.00
Upper Bound (Outlier): 23.00
Number of Outliers: 0

--- Column: Stress Level ---
Q1: 2.00
Q3: 8.00
```

```
IQR: 6.00
Lower Bound (Outlier): -7.00
Upper Bound (Outlier): 17.00
Number of Outliers: 0

--- Column: Sleep Hours ---
Q1: 4.70
Q3: 8.30
IQR: 3.60
Lower Bound (Outlier): -0.70
Upper Bound (Outlier): 13.70
Number of Outliers: 79
```




```
--- Column: Motivation Level ---
Q1: 2.00
Q3: 8.00
IQR: 6.00
Lower Bound (Outlier): -7.00
```

```
Upper Bound (Outlier): 17.00
Number of Outliers: 0

--- Column: Study Time Per Week ---
Q1: 10.00
Q3: 30.00
IQR: 20.00
Lower Bound (Outlier): -20.00
Upper Bound (Outlier): 60.00
Number of Outliers: 0

--- Column: Concentration Level ---
Q1: 2.00
Q3: 8.00
IQR: 6.00
Lower Bound (Outlier): -7.00
Upper Bound (Outlier): 17.00
Number of Outliers: 0

--- Column: Exam Anxiety Level ---
Q1: 2.00
Q3: 8.00
IQR: 6.00
Lower Bound (Outlier): -7.00
Upper Bound (Outlier): 17.00
Number of Outliers: 0

--- Column: Course Difficulty Rating ---
Q1: 2.00
Q3: 4.00
IQR: 2.00
Lower Bound (Outlier): -1.00
Upper Bound (Outlier): 7.00
Number of Outliers: 0

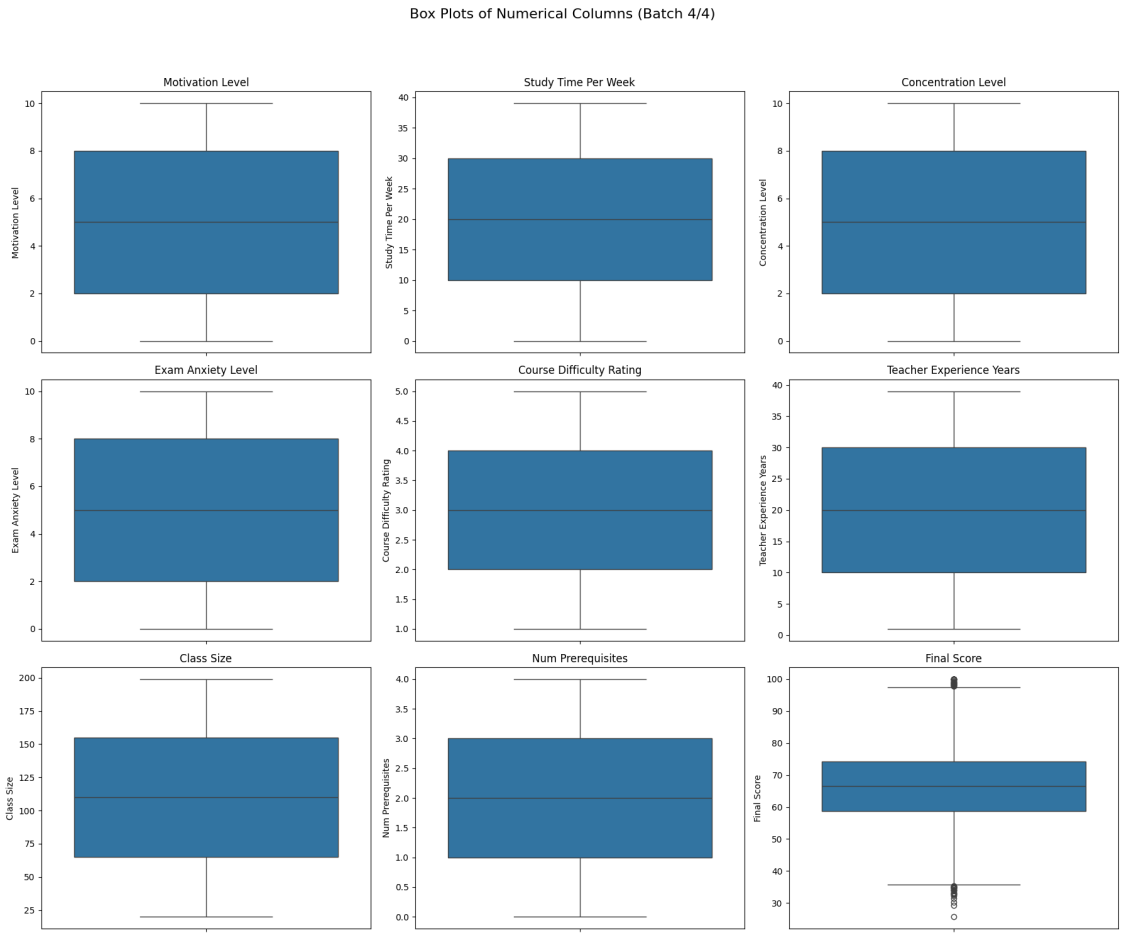
--- Column: Teacher Experience Years ---
Q1: 10.00
Q3: 30.00
IQR: 20.00
Lower Bound (Outlier): -20.00
Upper Bound (Outlier): 60.00
Number of Outliers: 0

--- Column: Class Size ---
Q1: 65.00
Q3: 155.00
IQR: 90.00
Lower Bound (Outlier): -70.00
```

```
Upper Bound (Outlier): 290.00
Number of Outliers: 0

--- Column: Num Prerequisites ---
Q1: 1.00
Q3: 3.00
IQR: 2.00
Lower Bound (Outlier): -2.00
Upper Bound (Outlier): 6.00
Number of Outliers: 0

--- Column: Final Score ---
Q1: 58.80
Q3: 74.30
IQR: 15.50
Lower Bound (Outlier): 35.55
Upper Bound (Outlier): 97.55
Number of Outliers: 45
```



[ ]:
```
import numpy as np

columns_to_cap = [
    'parent_income',
    'study_hours_last_semester',
    'lecture_attendance_rate',
    'online_portal_usage_minutes',
    'sleep_hours',
    'final_score'
]

print("--- Capping Outliers for Specified Columns (IQR Method) ---")

for col in columns_to_cap:
    if col in df.columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        # Cap the outliers
        df[col] = np.clip(df[col], lower_bound, upper_bound)
        print(f"Outliers in '{col.replace('_', ' ').title()}' capped between␣
 ↪{lower_bound:.2f} and {upper_bound:.2f}")
    else:
        print(f"Column '{col}' not found in DataFrame.")

print("\n--- Outlier capping complete. ---")
```

```
--- Capping Outliers for Specified Columns (IQR Method) ---
Outliers in 'Parent Income' capped between -11945.25 and 44140.75
Outliers in 'Study Hours Last Semester' capped between -87.38 and 297.62
Outliers in 'Lecture Attendance Rate' capped between 24.50 and 124.50
Outliers in 'Online Portal Usage Minutes' capped between -1014.12 and 3022.88
Outliers in 'Sleep Hours' capped between -0.70 and 13.70
Outliers in 'Final Score' capped between 35.55 and 97.55

--- Outlier capping complete. ---
```

After capping the outliers, it's a good practice to re-examine the descriptive statistics or box plots for the modified columns to confirm the changes. The box plots for the specified columns will be regenerated to show the effect of capping.

[ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

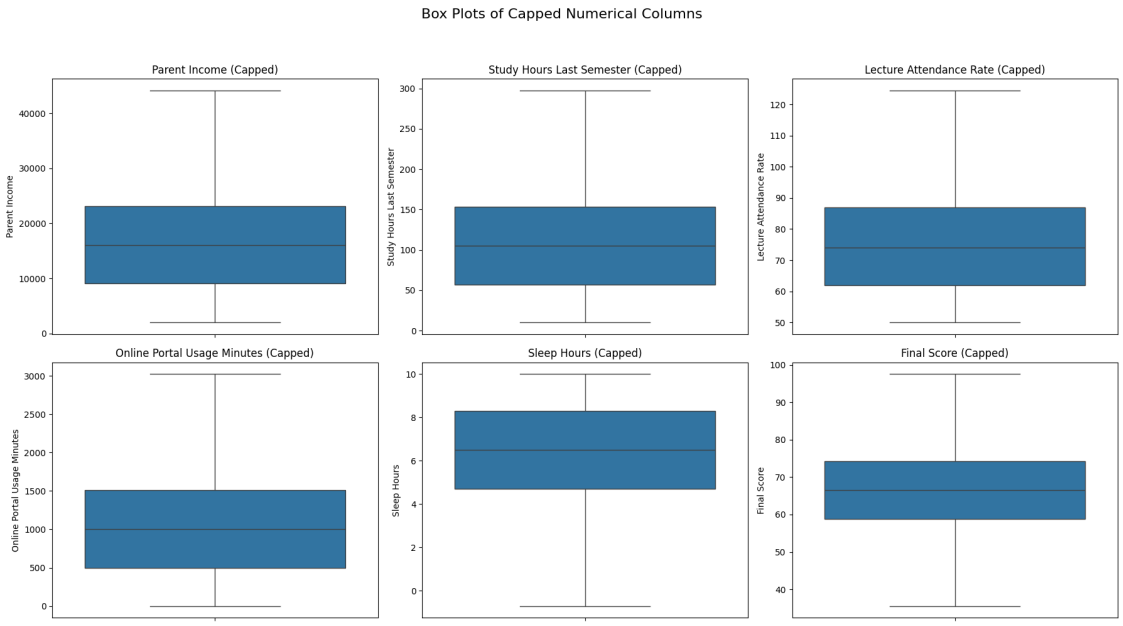
```
columns_to_plot_after_capping = [
    'parent_income',
    'study_hours_last_semester',
    'lecture_attendance_rate',
    'online_portal_usage_minutes',
    'sleep_hours',
    'final_score'
]

# Calculate rows needed for the current batch (up to 3x3)
rows = int(np.ceil(len(columns_to_plot_after_capping) / 3))
cols = 3

plt.figure(figsize=(18, rows * 5)) # Adjust figure height dynamically
plt.suptitle('Box Plots of Capped Numerical Columns', y=1.02, fontsize=16)

for i, col in enumerate(columns_to_plot_after_capping):
    if col in df.columns:
        plt.subplot(rows, cols, i + 1)
        sns.boxplot(y=df[col])
        plt.title(f'{col.replace("_", " ").title()} (Capped)')
        plt.ylabel(col.replace("_", " ").title())
    else:
        print(f"Column '{col}' not found in DataFrame for plotting.")

plt.tight_layout(rect=[0, 0.03, 1, 0.98]) # Adjust layout to make space for␣
 ↪suptitle
plt.show()
```



#Data Preprocessing

1.3 Handling Missing Values: Imputation

Missing values are common in datasets and can lead to biased results or errors in analysis. Here, we will impute missing values based on their data type:

- **Numerical Columns:** Missing values will be replaced by the **median** of their respective columns. The median is chosen over the mean as it is less sensitive to outliers.
- **Categorical Columns:** Missing values will be replaced by the **mode** (most frequent value) of their respective columns.

[ ]:
```
# Impute missing values for numerical columns with the median
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
for col in numerical_cols:
    if df[col].isnull().any():
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Missing values in numerical column '{col.replace('_', ' ').
 ↪title()}' imputed with median: {median_val:.2f}")

# Impute missing values for categorical columns with the mode
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    if df[col].isnull().any():
        # Mode might return multiple values if there's a tie, so take the first␣
 ↪one
        mode_val = df[col].mode()[0]
        df[col].fillna(mode_val, inplace=True)
        print(f"Missing values in categorical column '{col.replace('_', ' ').
 ↪title()}' imputed with mode: '{mode_val}'")

print("\n--- Missing value imputation complete. ---")

# Display info again to confirm no missing values
print("\n--- Verifying non-null counts after imputation ---")
df.info()
```

```
Missing values in numerical column 'Age' imputed with median: 23.00
Missing values in numerical column 'Parent Income' imputed with median: 16102.00
Missing values in numerical column 'Num Siblings' imputed with median: 3.00
Missing values in numerical column 'Family Support' imputed with median: 3.00
Missing values in numerical column 'Commute Time Min' imputed with median: 59.00
Missing values in numerical column 'Previous Gpa' imputed with median: 2.50
Missing values in numerical column 'Num Failed Courses' imputed with median:
2.00
```

```
Missing values in numerical column 'High School Grade' imputed with median:
74.80
Missing values in numerical column 'Math Background Score' imputed with median:
49.00
Missing values in numerical column 'Language Background Score' imputed with
median: 49.00
Missing values in numerical column 'Science Background Score' imputed with
median: 50.00
Missing values in numerical column 'Prior Semester Credits' imputed with median:
16.00
Missing values in numerical column 'Study Hours Last Semester' imputed with
median: 105.00
Missing values in numerical column 'Past Attendance Rate' imputed with median:
75.00
Missing values in numerical column 'Academic Warnings Count' imputed with
median: 2.00
Missing values in numerical column 'Lecture Attendance Rate' imputed with
median: 74.00
Missing values in numerical column 'Assignment Submission Rate' imputed with
median: 74.00
Missing values in numerical column 'Quiz Avg Score' imputed with median: 49.00
Missing values in numerical column 'Midterm Score' imputed with median: 49.00
Missing values in numerical column 'Lab Participation Rate' imputed with median:
50.00
Missing values in numerical column 'Online Portal Usage Minutes' imputed with
median: 1004.00
Missing values in numerical column 'Group Project Activity' imputed with median:
50.00
Missing values in numerical column 'Library Visits Per Month' imputed with
median: 10.00
Missing values in numerical column 'Discussion Forum Posts' imputed with median:
25.00
Missing values in numerical column 'Lateness Count' imputed with median: 7.00
Missing values in numerical column 'Stress Level' imputed with median: 5.00
Missing values in numerical column 'Sleep Hours' imputed with median: 6.50
Missing values in numerical column 'Motivation Level' imputed with median: 5.00
Missing values in numerical column 'Study Time Per Week' imputed with median:
20.00
Missing values in numerical column 'Concentration Level' imputed with median:
5.00
Missing values in numerical column 'Exam Anxiety Level' imputed with median:
5.00
Missing values in numerical column 'Course Difficulty Rating' imputed with
median: 3.00
Missing values in numerical column 'Teacher Experience Years' imputed with
median: 20.00
Missing values in numerical column 'Class Size' imputed with median: 110.00
Missing values in numerical column 'Num Prerequisites' imputed with median: 2.00
```

```
Missing values in numerical column 'Final Score' imputed with median: 66.50
Missing values in categorical column 'Gender' imputed with mode: 'Male'
Missing values in categorical column 'Part Time Job' imputed with mode: 'No'
Missing values in categorical column 'Course Type' imputed with mode: 'Elective'
Missing values in categorical column 'Final Grade' imputed with mode: 'C'
Missing values in categorical column 'Pass Fail' imputed with mode: 'Pass'

--- Missing value imputation complete. ---

--- Verifying non-null counts after imputation ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 41 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   age                            20000 non-null  float64
 1   gender                         20000 non-null  object
 2   parent_income                  20000 non-null  float64
 3   num_siblings                   20000 non-null  float64
 4   family_support                 20000 non-null  float64
 5   commute_time_min               20000 non-null  float64
 6   part_time_job                  20000 non-null  object
 7   previous_gpa                   20000 non-null  float64
 8   num_failed_courses             20000 non-null  float64
 9   high_school_grade              20000 non-null  float64
 10  math_background_score          20000 non-null  float64
 11  language_background_score      20000 non-null  float64
 12  science_background_score       20000 non-null  float64
 13  prior_semester_credits         20000 non-null  float64
 14  study_hours_last_semester      20000 non-null  float64
 15  past_attendance_rate           20000 non-null  float64
 16  academic_warnings_count        20000 non-null  float64
 17  lecture_attendance_rate        20000 non-null  float64
 18  assignment_submission_rate     20000 non-null  float64
 19  quiz_avg_score                 20000 non-null  float64
 20  midterm_score                  20000 non-null  float64
 21  lab_participation_rate         20000 non-null  float64
 22  online_portal_usage_minutes    20000 non-null  float64
 23  group_project_activity         20000 non-null  float64
 24  library_visits_per_month       20000 non-null  float64
 25  discussion_forum_posts         20000 non-null  float64
 26  lateness_count                 20000 non-null  float64
 27  stress_level                   20000 non-null  float64
 28  sleep_hours                    20000 non-null  float64
 29  motivation_level               20000 non-null  float64
 30  study_time_per_week            20000 non-null  float64
 31  concentration_level            20000 non-null  float64
 32  exam_anxiety_level             20000 non-null  float64
```

```
 33  course_difficulty_rating     20000 non-null  float64
 34  teacher_experience_years     20000 non-null  float64
 35  class_size                   20000 non-null  float64
 36  num_prerequisites            20000 non-null  float64
 37  course_type                  20000 non-null  object
 38  final_score                  20000 non-null  float64
 39  final_grade                  20000 non-null  object
 40  pass_fail                    20000 non-null  object
dtypes: float64(36), object(5)
memory usage: 6.3+ MB
```

```
/tmp/ipython-input-1546089506.py:6: FutureWarning: A value is trying to be set
on a copy of a DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df[col].fillna(median_val, inplace=True)
/tmp/ipython-input-1546089506.py:15: FutureWarning: A value is trying to be set
on a copy of a DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df[col].fillna(mode_val, inplace=True)
```

[ ]:
```python
print("--- Number of unique categories in each categorical column ---")

categorical_cols = df.select_dtypes(include='object').columns

for col in categorical_cols:
    num_unique_categories = df[col].nunique()
    print(f"Column '{col.replace('_', ' ').title()}': {num_unique_categories}␣
 ↪unique categories")

print("\n--- Categorical column analysis complete. ---")
```

```
--- Number of unique categories in each categorical column ---
Column 'Gender': 2 unique categories
Column 'Part Time Job': 2 unique categories
Column 'Course Type': 2 unique categories
Column 'Final Grade': 5 unique categories
Column 'Pass Fail': 2 unique categories

--- Categorical column analysis complete. ---
```

1.4 Relationship between Midterm Score and Final Score

This scatter plot visualizes the relationship between students' 'midterm_score' and their 'final_score'.

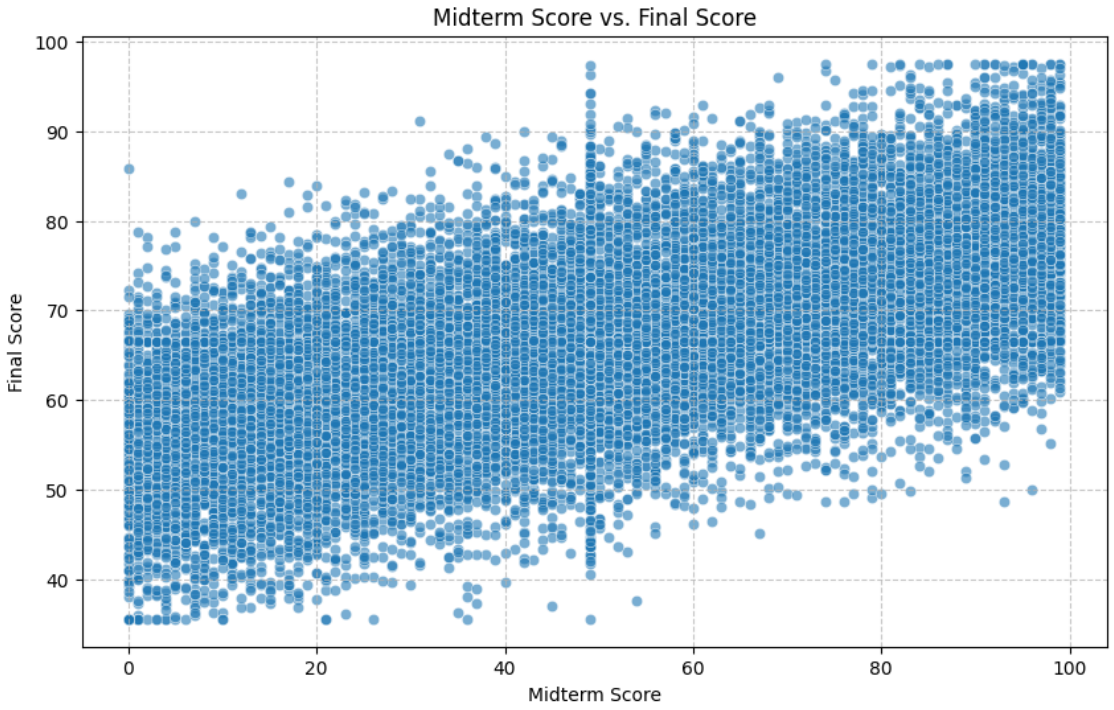
[ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

correlation = df['midterm_score'].corr(df['final_score'])

print(f"Correlation between 'midterm_score' and 'final_score': {correlation:.
  ↪2f}")

plt.figure(figsize=(10, 6))
sns.scatterplot(x='midterm_score', y='final_score', data=df, alpha=0.6)
plt.title('Midterm Score vs. Final Score')
plt.xlabel('Midterm Score')
plt.ylabel('Final Score')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
Correlation between 'midterm_score' and 'final_score': 0.63
```



1.5 a) One-Hot Encoding Categorical Features

One-hot encoding is a process used to convert categorical variables into a format that can be provided to machine learning algorithms to improve prediction quality. This method creates new binary columns for each unique category present in the original categorical column.

[ ]:
```
from sklearn.preprocessing import LabelEncoder

# Identify all object-type columns
categorical_cols_all = df.select_dtypes(include='object').columns.tolist()

# Define columns for one-hot encoding (all categorical except 'final_grade')
categorical_cols_for_onehot = [col for col in categorical_cols_all if col !=␣
 ↪'final_grade']

print(f"Categorical columns identified for one-hot encoding:␣
 ↪{categorical_cols_for_onehot}")
print("Column identified for label encoding: ['final_grade']")

# Apply one-hot encoding to specified columns
# This will also keep numerical columns and 'final_grade' as they are initially
df_encoded = pd.get_dummies(df, columns=categorical_cols_for_onehot,␣
 ↪drop_first=True)
```

```
# Apply Label Encoding to 'final_grade'
le = LabelEncoder()
df_encoded['final_grade_encoded'] = le.fit_transform(df_encoded['final_grade'])

# Drop the original 'final_grade' column after label encoding it
df_encoded = df_encoded.drop(columns=['final_grade'])

print("\n--- Encoding complete. ---")
print("Shape of DataFrame after encoding:", df_encoded.shape)
```

```
Categorical columns identified for one-hot encoding: ['gender', 'part_time_job',
'course_type', 'pass_fail']
Column identified for label encoding: ['final_grade']

--- Encoding complete. ---
Shape of DataFrame after encoding: (20000, 41)
```

1.6 b) Concatenating Encoded Features with Numerical Data

The `pd.get_dummies` function, when applied to the entire DataFrame as done above, automatically replaces the original categorical columns with their one-hot encoded versions and keeps all numerical columns. Therefore, a separate concatenation step is not explicitly needed as `df_encoded` already contains both.

[ ]:
```
print("Displaying the first 5 rows of the DataFrame after one-hot encoding␣
↪(which includes all numerical columns):")
display(df_encoded.head())
print("\nInformation about the new DataFrame:")
df_encoded.info()
```

```
Displaying the first 5 rows of the DataFrame after one-hot encoding (which
includes all numerical columns):

    age  parent_income  num_siblings  family_support  commute_time_min  \
0  24.0        22678.0           0.0             1.0              95.0
1  21.0        17905.0           3.0             2.0               0.0
2  28.0        16102.0           0.0             0.0              67.0
3  25.0        29391.0           4.0             4.0              83.0
4  22.0        15522.0           0.0             4.0              58.0

   previous_gpa  num_failed_courses  high_school_grade  math_background_score  \
0          2.22                 3.0               99.7                   34.0
1          3.78                 1.0               75.7                   91.0
2          2.50                 4.0               93.4                   32.0
3          1.84                 4.0               69.7                   33.0
4          3.38                 0.0               62.2                   89.0

   language_background_score  …  course_difficulty_rating  \
0                       91.0  …                       2.0
```

```
1                      21.0  …                      1.0
2                      30.0  …                      1.0
3                      19.0  …                      5.0
4                      48.0  …                      3.0

   teacher_experience_years  class_size  num_prerequisites  final_score  \
0                      11.0        95.0                3.0         66.5
1                      20.0       148.0                2.0         81.7
2                      32.0        55.0                3.0         77.3
3                      30.0        42.0                1.0         78.5
4                      31.0       123.0                1.0         60.3

   gender_Male  part_time_job_Yes  course_type_Mandatory  pass_fail_Pass  \
0        False              False                  False            True
1         True               True                  False            True
2         True              False                  False            True
3        False               True                   True            True
4         True               True                   True            True

   final_grade_encoded
0                    2
1                    2
2                    1
3                    1
4                    3

[5 rows x 41 columns]


Information about the new DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 41 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   age                          20000 non-null  float64
 1   parent_income                20000 non-null  float64
 2   num_siblings                 20000 non-null  float64
 3   family_support               20000 non-null  float64
 4   commute_time_min             20000 non-null  float64
 5   previous_gpa                 20000 non-null  float64
 6   num_failed_courses           20000 non-null  float64
 7   high_school_grade            20000 non-null  float64
 8   math_background_score        20000 non-null  float64
 9   language_background_score    20000 non-null  float64
 10  science_background_score     20000 non-null  float64
 11  prior_semester_credits       20000 non-null  float64
 12  study_hours_last_semester    20000 non-null  float64
```

```
 13  past_attendance_rate          20000 non-null  float64
 14  academic_warnings_count       20000 non-null  float64
 15  lecture_attendance_rate       20000 non-null  float64
 16  assignment_submission_rate    20000 non-null  float64
 17  quiz_avg_score                20000 non-null  float64
 18  midterm_score                 20000 non-null  float64
 19  lab_participation_rate        20000 non-null  float64
 20  online_portal_usage_minutes   20000 non-null  float64
 21  group_project_activity        20000 non-null  float64
 22  library_visits_per_month      20000 non-null  float64
 23  discussion_forum_posts        20000 non-null  float64
 24  lateness_count                20000 non-null  float64
 25  stress_level                  20000 non-null  float64
 26  sleep_hours                   20000 non-null  float64
 27  motivation_level              20000 non-null  float64
 28  study_time_per_week           20000 non-null  float64
 29  concentration_level           20000 non-null  float64
 30  exam_anxiety_level            20000 non-null  float64
 31  course_difficulty_rating      20000 non-null  float64
 32  teacher_experience_years      20000 non-null  float64
 33  class_size                    20000 non-null  float64
 34  num_prerequisites             20000 non-null  float64
 35  final_score                   20000 non-null  float64
 36  gender_Male                   20000 non-null  bool
 37  part_time_job_Yes             20000 non-null  bool
 38  course_type_Mandatory         20000 non-null  bool
 39  pass_fail_Pass                20000 non-null  bool
 40  final_grade_encoded           20000 non-null  int64
dtypes: bool(4), float64(36), int64(1)
memory usage: 5.7 MB
```

1.7 Feature Scaling: StandardScaler

StandardScaler standardizes features by removing the mean and scaling to unit variance. The standard score of a sample `x` is calculated as:

`z = (x - u) / s`

where `u` is the mean of the training samples and `s` is the standard deviation of the training samples. This method is often used to ensure that all features contribute equally to the model training process.

[ ]:
```python
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Identify columns to scale (all except 'final_grade_encoded')
columns_to_scale = df_encoded.columns.drop('final_grade_encoded')

# Initialize the StandardScaler
```

```
scaler = StandardScaler()

# Fit and transform only the selected columns
df_scaled_features = scaler.fit_transform(df_encoded[columns_to_scale])

# Convert scaled features back to a DataFrame
df_scaled_features = pd.DataFrame(df_scaled_features, columns=columns_to_scale,␣
 ↪index=df_encoded.index)

# Combine scaled features with the unscaled 'final_grade_encoded' target␣
 ↪variable
df_scaled = pd.concat([df_scaled_features, df_encoded['final_grade_encoded']],␣
 ↪axis=1)

print("--- Feature scaling complete. 'final_grade_encoded' was not scaled. ---")
print("Displaying the first 5 rows of the scaled DataFrame:")
display(df_scaled.head())
print("\nInformation about the scaled DataFrame:")
df_scaled.info()
```

```
--- Feature scaling complete. 'final_grade_encoded' was not scaled. ---
Displaying the first 5 rows of the scaled DataFrame:
```

	age	parent_income	num_siblings	family_support	commute_time_min	\
0	0.157420	0.781516	-1.492292	-0.901293	1.041476	
1	-0.724565	0.204280	0.280693	-0.307400	-1.730627	
2	1.333400	-0.013771	-1.492292	-1.495187	0.224435	
3	0.451415	1.593370	0.871688	0.880388	0.691316	
4	-0.430570	-0.083915	-1.492292	0.880388	-0.038185	

	previous_gpa	num_failed_courses	high_school_grade	math_background_score	\
0	-0.317294	0.720725	1.745664	-0.539205	
1	1.505280	-0.715769	0.056325	1.464100	
2	0.009835	1.438973	1.302212	-0.609497	
3	-0.761254	1.438973	-0.366010	-0.574351	
4	1.037954	-1.434017	-0.893928	1.393809	

	language_background_score	…	course_difficulty_rating	\
0	1.464462	…	-0.718536	
1	-0.999771	…	-1.438801	
2	-0.682941	…	-1.438801	
3	-1.070178	…	1.442258	
4	-0.049281	…	0.001729	

	teacher_experience_years	class_size	num_prerequisites	final_score	\
0	-0.815403	-0.291034	0.721174	-0.003902	
1	-0.006338	0.743685	0.002408	1.394505	
2	1.072416	-1.071954	0.721174	0.989703	

```
3                  0.892624   -1.325753          -0.716359     1.100103
4                  0.982520    0.255610          -0.716359    -0.574305

   gender_Male  part_time_job_Yes  course_type_Mandatory  pass_fail_Pass  \
0    -1.033253          -0.967914              -0.958174        0.269153
1     0.967818           1.033149              -0.958174        0.269153
2     0.967818          -0.967914              -0.958174        0.269153
3    -1.033253           1.033149               1.043652        0.269153
4     0.967818           1.033149               1.043652        0.269153

   final_grade_encoded
0                    2
1                    2
2                    1
3                    1
4                    3

[5 rows x 41 columns]


Information about the scaled DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 41 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   age                           20000 non-null  float64
 1   parent_income                 20000 non-null  float64
 2   num_siblings                  20000 non-null  float64
 3   family_support                20000 non-null  float64
 4   commute_time_min              20000 non-null  float64
 5   previous_gpa                  20000 non-null  float64
 6   num_failed_courses            20000 non-null  float64
 7   high_school_grade             20000 non-null  float64
 8   math_background_score         20000 non-null  float64
 9   language_background_score     20000 non-null  float64
 10  science_background_score      20000 non-null  float64
 11  prior_semester_credits        20000 non-null  float64
 12  study_hours_last_semester     20000 non-null  float64
 13  past_attendance_rate          20000 non-null  float64
 14  academic_warnings_count       20000 non-null  float64
 15  lecture_attendance_rate       20000 non-null  float64
 16  assignment_submission_rate    20000 non-null  float64
 17  quiz_avg_score                20000 non-null  float64
 18  midterm_score                 20000 non-null  float64
 19  lab_participation_rate        20000 non-null  float64
 20  online_portal_usage_minutes   20000 non-null  float64
 21  group_project_activity        20000 non-null  float64
```

```
 22  library_visits_per_month     20000 non-null  float64
 23  discussion_forum_posts       20000 non-null  float64
 24  lateness_count               20000 non-null  float64
 25  stress_level                 20000 non-null  float64
 26  sleep_hours                  20000 non-null  float64
 27  motivation_level             20000 non-null  float64
 28  study_time_per_week          20000 non-null  float64
 29  concentration_level          20000 non-null  float64
 30  exam_anxiety_level           20000 non-null  float64
 31  course_difficulty_rating     20000 non-null  float64
 32  teacher_experience_years     20000 non-null  float64
 33  class_size                   20000 non-null  float64
 34  num_prerequisites            20000 non-null  float64
 35  final_score                  20000 non-null  float64
 36  gender_Male                  20000 non-null  float64
 37  part_time_job_Yes            20000 non-null  float64
 38  course_type_Mandatory        20000 non-null  float64
 39  pass_fail_Pass               20000 non-null  float64
 40  final_grade_encoded          20000 non-null  int64
dtypes: float64(40), int64(1)
memory usage: 6.3 MB
```

1.8 Data Splitting

Split `df_scaled` into training, validation, and test sets (80/10/10 ratio) for three different target variables: `final_score` (regression), `final_grade_encoded` (classification), and `pass_fail_Pass` (binary classification). For each split, the features will exclude all three potential target variables (`final_score`, `final_grade_encoded`, `pass_fail_Pass`) to prevent data leakage. Finally, confirm the shapes of the resulting training, validation, and test sets for all three targets.

1.8.1 Split Data for Final Score (Regression)

[ ]:
```
from sklearn.model_selection import train_test_split

# Define the feature set X_reg by dropping the specified target columns
X_reg = df_scaled.drop(columns=['final_score', 'final_grade_encoded',␣
 ↪'pass_fail_Pass'])

# Define the target variable y_reg
y_reg = df_scaled['final_score']

print("Features (X_reg) and target (y_reg) defined.")
```

```
Features (X_reg) and target (y_reg) defined.
```

[ ]:
```
import numpy as np
```

```
# Split X_reg and y_reg into an initial training set (80%) and a temporary set␣
 ↪(20%)
X_train_reg, X_temp_reg, y_train_reg, y_temp_reg = train_test_split(X_reg,␣
 ↪y_reg, test_size=0.2, random_state=42)

# Split the temporary set (X_temp_reg, y_temp_reg) into validation (10%) and␣
 ↪test sets (10%)
# Since X_temp_reg is 20% of the original data, splitting it in half will␣
 ↪result in 10% for validation and 10% for test
X_val_reg, X_test_reg, y_val_reg, y_test_reg = train_test_split(X_temp_reg,␣
 ↪y_temp_reg, test_size=0.5, random_state=42)

print("--- Data splitting for Final Score (Regression) complete. ---")

# Print the shapes of the resulting sets
print(f"Shape of X_train_reg: {X_train_reg.shape}")
print(f"Shape of y_train_reg: {y_train_reg.shape}")
print(f"Shape of X_val_reg: {X_val_reg.shape}")
print(f"Shape of y_val_reg: {y_val_reg.shape}")
print(f"Shape of X_test_reg: {X_test_reg.shape}")
print(f"Shape of y_test_reg: {y_test_reg.shape}")
```

```
--- Data splitting for Final Score (Regression) complete. ---
Shape of X_train_reg: (16000, 38)
Shape of y_train_reg: (16000,)
Shape of X_val_reg: (2000, 38)
Shape of y_val_reg: (2000,)
Shape of X_test_reg: (2000, 38)
Shape of y_test_reg: (2000,)
```

1.8.2 Split Data for Final Grade Encoded (Classification)

[ ]:
```
import numpy as np

# Define the feature set X_class by dropping the specified target columns
X_class = df_scaled.drop(columns=['final_score', 'final_grade_encoded',␣
 ↪'pass_fail_Pass'])

# Define the target variable y_class
y_class = df_scaled['final_grade_encoded']

print("Features (X_class) and target (y_class) defined.")
```

```
Features (X_class) and target (y_class) defined.
```

[ ]:
```
import numpy as np
```

```
# Split X_class and y_class into an initial training set (80%) and a temporary␣
 ↪set (20%)
X_train_class, X_temp_class, y_train_class, y_temp_class =␣
 ↪train_test_split(X_class, y_class, test_size=0.2, random_state=42)

# Split the temporary set (X_temp_class, y_temp_class) into validation (10%)␣
 ↪and test sets (10%)
# Since X_temp_class is 20% of the original data, splitting it in half will␣
 ↪result in 10% for validation and 10% for test
X_val_class, X_test_class, y_val_class, y_test_class =␣
 ↪train_test_split(X_temp_class, y_temp_class, test_size=0.5, random_state=42)

print("--- Data splitting for Final Grade Encoded (Classification) complete.␣
 ↪---")

# Print the shapes of the resulting sets
print(f"Shape of X_train_class: {X_train_class.shape}")
print(f"Shape of y_train_class: {y_train_class.shape}")
print(f"Shape of X_val_class: {X_val_class.shape}")
print(f"Shape of y_val_class: {y_val_class.shape}")
print(f"Shape of X_test_class: {X_test_class.shape}")
print(f"Shape of y_test_class: {y_test_class.shape}")
```

```
--- Data splitting for Final Grade Encoded (Classification) complete. ---
Shape of X_train_class: (16000, 38)
Shape of y_train_class: (16000,)
Shape of X_val_class: (2000, 38)
Shape of y_val_class: (2000,)
Shape of X_test_class: (2000, 38)
Shape of y_test_class: (2000,)
```

1.8.3 Split Data for Pass/Fail (Binary Classification)

[ ]:
```
import numpy as np

# Define the feature set X_binary by dropping the specified target columns
X_binary = df_scaled.drop(columns=['final_score', 'final_grade_encoded',␣
 ↪'pass_fail_Pass'])

# Define the target variable y_binary
y_binary = df_scaled['pass_fail_Pass']

print("Features (X_binary) and target (y_binary) defined.")
```

```
Features (X_binary) and target (y_binary) defined.
```

[ ]:
```
import numpy as np

# Split X_binary and y_binary into an initial training set (80%) and a␣
 ↪temporary set (20%)
X_train_binary, X_temp_binary, y_train_binary, y_temp_binary =␣
 ↪train_test_split(X_binary, y_binary, test_size=0.2, random_state=42)

# Split the temporary set (X_temp_binary, y_temp_binary) into validation (10%)␣
 ↪and test sets (10%)
# Since X_temp_binary is 20% of the original data, splitting it in half will␣
 ↪result in 10% for validation and 10% for test
X_val_binary, X_test_binary, y_val_binary, y_test_binary =␣
 ↪train_test_split(X_temp_binary, y_temp_binary, test_size=0.5,␣
 ↪random_state=42)

print("--- Data splitting for Pass/Fail (Binary Classification) complete. ---")

# Print the shapes of the resulting sets
print(f"Shape of X_train_binary: {X_train_binary.shape}")
print(f"Shape of y_train_binary: {y_train_binary.shape}")
print(f"Shape of X_val_binary: {X_val_binary.shape}")
print(f"Shape of y_val_binary: {y_val_binary.shape}")
print(f"Shape of X_test_binary: {X_test_binary.shape}")
print(f"Shape of y_test_binary: {y_test_binary.shape}")
```

```
--- Data splitting for Pass/Fail (Binary Classification) complete. ---
Shape of X_train_binary: (16000, 38)
Shape of y_train_binary: (16000,)
Shape of X_val_binary: (2000, 38)
Shape of y_val_binary: (2000,)
Shape of X_test_binary: (2000, 38)
Shape of y_test_binary: (2000,)
```

2 “Final Score” (Regression) Models

##Linear Regression Model Initialize and train a `LinearRegression` model using the `X_train_reg` and `y_train_reg` datasets. Then, make predictions on the test set.

[ ]:
```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
import pandas as pd

# Define the parameter grid for alpha
param_grid_ridge = {
    'alpha': [0.001, 0.005, 0.01, 0.1, 1.0, 10.0]
}
```

```
# Initialize Ridge Regression model
ridge_base_model = Ridge(random_state=42)

# Initialize GridSearchCV
grid_search_ridge = GridSearchCV(estimator=ridge_base_model,␣
 ↪param_grid=param_grid_ridge,
                                 scoring='neg_mean_squared_error', cv=5,␣
 ↪n_jobs=-1, verbose=0)

# Fit GridSearchCV to find the best alpha
grid_search_ridge.fit(X_train_reg, y_train_reg)

# Get the best model and its parameters
model_ridge = grid_search_ridge.best_estimator_
best_alpha_ridge = grid_search_ridge.best_params_['alpha']

y_pred_ridge = model_ridge.predict(X_test_reg)

print(f"Ridge Regression model initialized and tuned successfully using␣
 ↪GridSearchCV!")
print(f"Best alpha found: {best_alpha_ridge}")

# Display results for each alpha value
print("\n--- GridSearchCV Results for each Alpha Value ---")
cv_results = pd.DataFrame(grid_search_ridge.cv_results_)
# Convert negative mean squared error to positive MSE
cv_results['mean_test_mse'] = -cv_results['mean_test_score']

display(cv_results[['param_alpha', 'mean_test_mse']].
 ↪sort_values(by='mean_test_mse'))
```

```
Ridge Regression model initialized and tuned successfully using GridSearchCV!
Best alpha found: 10.0

--- GridSearchCV Results for each Alpha Value ---
```

	param_alpha	mean_test_mse
5	10.000	0.257969
4	1.000	0.257970
3	0.100	0.257970
2	0.010	0.257970
1	0.005	0.257970
0	0.001	0.257970

[ ]:
```
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import matplotlib.pyplot as plt
import seaborn as sns
```

```
mse_ridge = mean_squared_error(y_test_reg, y_pred_ridge)
r2_ridge = r2_score(y_test_reg, y_pred_ridge)
mae_ridge = mean_absolute_error(y_test_reg, y_pred_ridge)

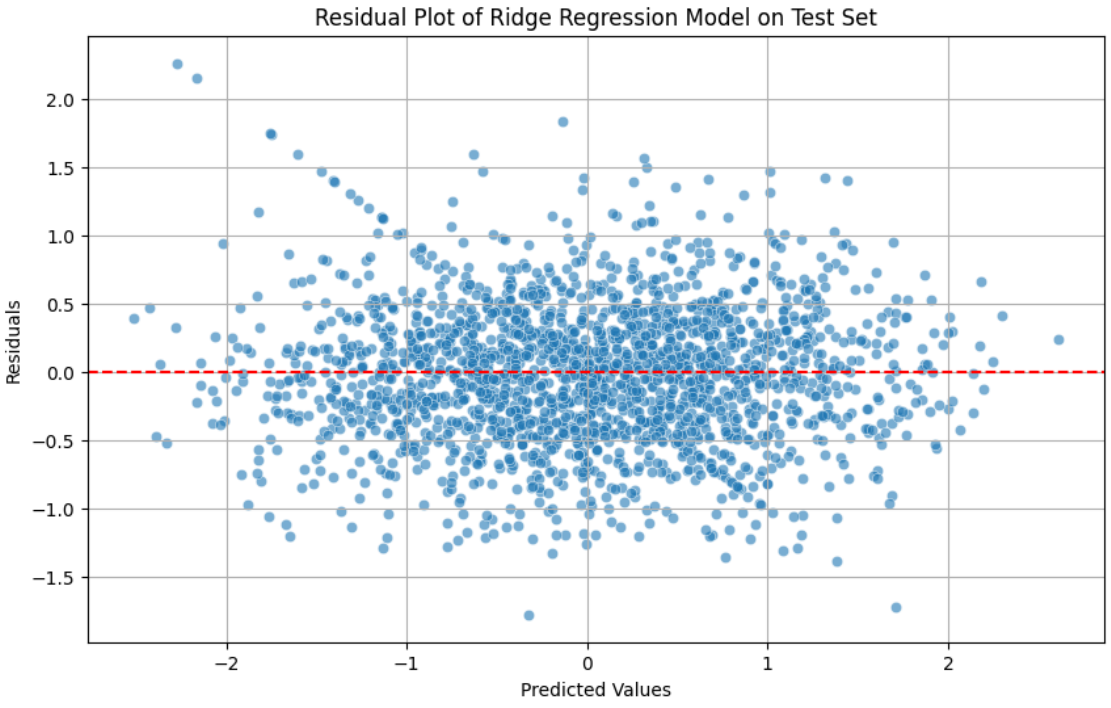
print("--- Ridge Regression Model Evaluation on Test Set ---")
print(f"Mean Absolute Error (MAE): {mae_ridge:.4f}")
print(f"Mean Squared Error (MSE): {mse_ridge:.4f}")
print(f"R-squared (R2) Score: {r2_ridge:.4f}")

residuals_ridge = y_test_reg - y_pred_ridge

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_ridge, y=residuals_ridge, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--')
plt.title('Residual Plot of Ridge Regression Model on Test Set')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.grid(True)
plt.show()

print("Residual error plot generated for the Ridge Regression model.")
```

```
--- Ridge Regression Model Evaluation on Test Set ---
Mean Absolute Error (MAE): 0.3929
Mean Squared Error (MSE): 0.2527
R-squared (R2) Score: 0.7499
```



Residual error plot generated for the Ridge Regression model.

[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature names from X_train_reg
feature_names = X_train_reg.columns

# Get the coefficients from the trained Ridge model
coefficients = model_ridge.coef_

# Create a DataFrame for feature importance
feature_importance_df_ridge = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients
})

# Calculate absolute coefficients for ranking
feature_importance_df_ridge['Absolute_Coefficient'] =␣
 ↪feature_importance_df_ridge['Coefficient'].abs()

# Sort features by absolute coefficient in descending order
feature_importance_df_ridge = feature_importance_df_ridge.
 ↪sort_values(by='Absolute_Coefficient', ascending=False)

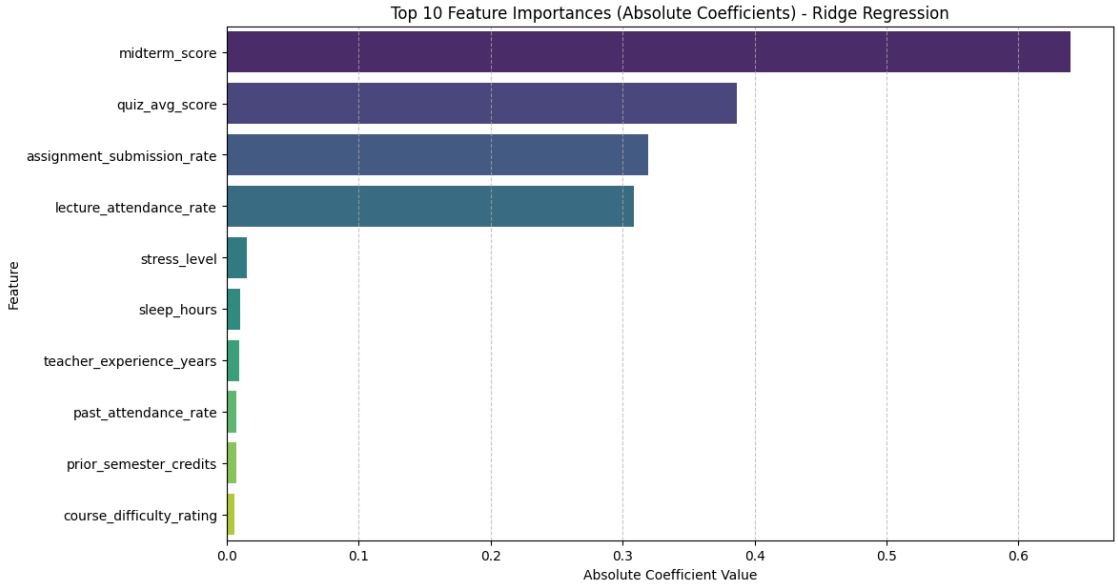
print("--- Top 10 Feature Importances for Ridge Regression Model ---")
display(feature_importance_df_ridge.head(10))

# Visualize top 10 features
plt.figure(figsize=(12, 7))
sns.barplot(x='Absolute_Coefficient', y='Feature',␣
 ↪data=feature_importance_df_ridge.head(10), palette='viridis', hue='Feature',␣
 ↪legend=False)
plt.title('Top 10 Feature Importances (Absolute Coefficients) - Ridge␣
 ↪Regression')
plt.xlabel('Absolute Coefficient Value')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

print("Feature importance analysis and visualization complete.")
```

```
--- Top 10 Feature Importances for Ridge Regression Model ---
```

	Feature	Coefficient	Absolute_Coefficient
18	midterm_score	0.639725	0.639725
17	quiz_avg_score	0.386427	0.386427
16	assignment_submission_rate	0.319173	0.319173
15	lecture_attendance_rate	0.308886	0.308886
25	stress_level	-0.015097	0.015097
26	sleep_hours	0.010223	0.010223
32	teacher_experience_years	-0.009407	0.009407
13	past_attendance_rate	-0.007632	0.007632
11	prior_semester_credits	0.007455	0.007455
31	course_difficulty_rating	0.006179	0.006179




```
Feature importance analysis and visualization complete.
```

Summary The Linear Regression model's performance, based on evaluation metrics, shows a Mean Squared Error (MSE) of 0.25, a Mean Absolute Error (MAE) of 0.39, and an R-squared (R2) score of 0.75.

The residual error plot indicates that the residuals are scattered randomly around zero, without a clear pattern, suggesting that the model captures the underlying relationship reasonably well and that the errors are mostly homoscedastic.

2.1 SVR Model

Import the SVR model from `sklearn.svm`, initialize an SVR model, and then train the model using `X_train_reg` and `y_train_reg`.

[ ]:
```
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
```

```
import pandas as pd

# Define the parameter grid for C
param_grid_svr = {
    'C': [0.1, 1.0, 10.0, 100.0]
}

# Initialize SVR model
svr_base_model = SVR()

# Initialize GridSearchCV
grid_search_svr = GridSearchCV(estimator=svr_base_model,␣
 ↪param_grid=param_grid_svr,
                               scoring='neg_mean_squared_error', cv=5,␣
 ↪n_jobs=-1, verbose=0)

# Fit GridSearchCV to find the best C
grid_search_svr.fit(X_train_reg, y_train_reg)

# Get the best model and its parameters
model_svr = grid_search_svr.best_estimator_
best_c_svr = grid_search_svr.best_params_['C']

y_pred_svr = model_svr.predict(X_test_reg)

print(f"SVR model initialized and tuned successfully using GridSearchCV!")
print(f"Best C found: {best_c_svr}")

# Display results for each C value
print("\n--- GridSearchCV Results for each C Value ---")
cv_results_svr = pd.DataFrame(grid_search_svr.cv_results_)
cv_results_svr['mean_test_mse'] = -cv_results_svr['mean_test_score']

display(cv_results_svr[['param_C', 'mean_test_mse']].
 ↪sort_values(by='mean_test_mse'))
```

```
SVR model initialized and tuned successfully using GridSearchCV!
Best C found: 0.1

--- GridSearchCV Results for each C Value ---
```

	param_C	mean_test_mse
0	0.1	0.275746
1	1.0	0.290648
2	10.0	0.345640
3	100.0	0.347129

[ ]:
```python
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import mean_absolute_error
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate Mean Squared Error (MSE)
mse_svr = mean_squared_error(y_test_reg, y_pred_svr)

# Calculate R-squared score
r2_svr = r2_score(y_test_reg, y_pred_svr)

mae_svr = mean_absolute_error(y_test_reg, y_pred_svr)

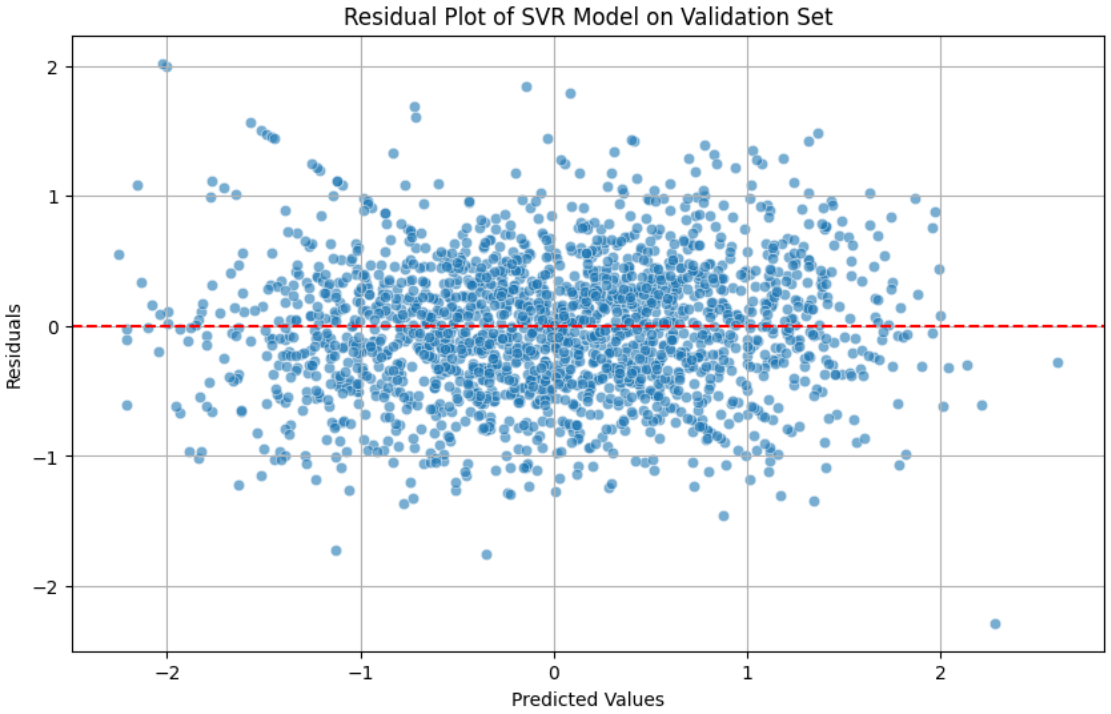
print("--- SVR Model Evaluation on Test Set ---")
print(f"Mean Absolute Error (MAE): {mae_svr:.4f}")
print(f"Mean Squared Error (MSE): {mse_svr:.4f}")
print(f"R-squared (R2) Score: {r2_svr:.4f}")


# Calculate residuals
residuals_svr = y_test_reg - y_pred_svr

# Create a residual plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_svr, y=residuals_svr, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--') # Add a horizontal line at y=0
plt.title('Residual Plot of SVR Model on Validation Set')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.grid(True)
plt.show()

print("Residual error plot generated for the SVR model.")
```

```
--- SVR Model Evaluation on Test Set ---
Mean Absolute Error (MAE): 0.4103
Mean Squared Error (MSE): 0.2731
R-squared (R2) Score: 0.7297
```



`Residual error plot generated for the SVR model.`

Summary The SVR model's performance, based on evaluation metrics, shows a Mean Squared Error (MSE) of 0.27, a Mean Absolute Error (MAE) of 0.41, and an R-squared (R2) score of 0.73. The residual error plot indicates that the residuals are scattered randomly around zero, without a clear pattern, suggesting that the model captures the underlying relationship reasonably well and that the errors are mostly homoscedastic.

Data Analysis Key Findings The Support Vector Regressor (SVR) model was successfully initialized and trained on the X_train_reg and y_train_reg datasets. Predictions were successfully generated on the test set (X_test_reg) using the trained SVR model. The evaluation metrics for the SVR model's performance on the test set are: Mean Squared Error (MSE): 0.29 Mean Absolute Error (MAE): 0.43 R-squared (R2) Score: 0.71 The residual plot visually confirms that the residuals are scattered randomly around the zero line, indicating no obvious patterns or systematic errors in the model's predictions.

2.2 Decision Tree Model

Import the `DecisionTreeRegressor` model from `sklearn.tree`, then initialize and train a `DecisionTreeRegressor` model using the `X_train_reg` and `y_train_reg` datasets.

[ ]:
```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV
import pandas as pd
```

```
# Define the parameter grid for Decision Tree
param_grid_dt = {
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize Decision Tree Regressor model
dt_base_model = DecisionTreeRegressor(random_state=42)

# Initialize GridSearchCV
grid_search_dt = GridSearchCV(estimator=dt_base_model, param_grid=param_grid_dt,
                              scoring='neg_mean_squared_error', cv=5,␣
 ↪n_jobs=-1, verbose=0)

# Fit GridSearchCV to find the best parameters
grid_search_dt.fit(X_train_reg, y_train_reg)

# Get the best model and its parameters
model_dt = grid_search_dt.best_estimator_
best_params_dt = grid_search_dt.best_params_

y_pred_dt = model_dt.predict(X_test_reg)

print(f"DecisionTreeRegressor model initialized and tuned successfully using␣
 ↪GridSearchCV!")
print(f"Best parameters found: {best_params_dt}")

# Display results for each parameter combination
print("\n--- GridSearchCV Results for each Parameter Combination ---")
cv_results_dt = pd.DataFrame(grid_search_dt.cv_results_)
cv_results_dt['mean_test_mse'] = -cv_results_dt['mean_test_score']

display(cv_results_dt[['params', 'mean_test_mse']].
 ↪sort_values(by='mean_test_mse').head())
```

```
DecisionTreeRegressor model initialized and tuned successfully using
GridSearchCV!
Best parameters found: {'max_depth': 5, 'min_samples_leaf': 1,
'min_samples_split': 2}

--- GridSearchCV Results for each Parameter Combination ---

                                               params  mean_test_mse
12  {'max_depth': 5, 'min_samples_leaf': 2, 'min_s…       0.382813
15  {'max_depth': 5, 'min_samples_leaf': 4, 'min_s…       0.382813
13  {'max_depth': 5, 'min_samples_leaf': 2, 'min_s…       0.382813
14  {'max_depth': 5, 'min_samples_leaf': 2, 'min_s…       0.382813
```

```
10  {'max_depth': 5, 'min_samples_leaf': 1, 'min_s…       0.382813
```

[ ]:
```python
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

mse_dt = mean_squared_error(y_test_reg, y_pred_dt)
mae_dt = mean_absolute_error(y_test_reg, y_pred_dt)
r2_dt = r2_score(y_test_reg, y_pred_dt)

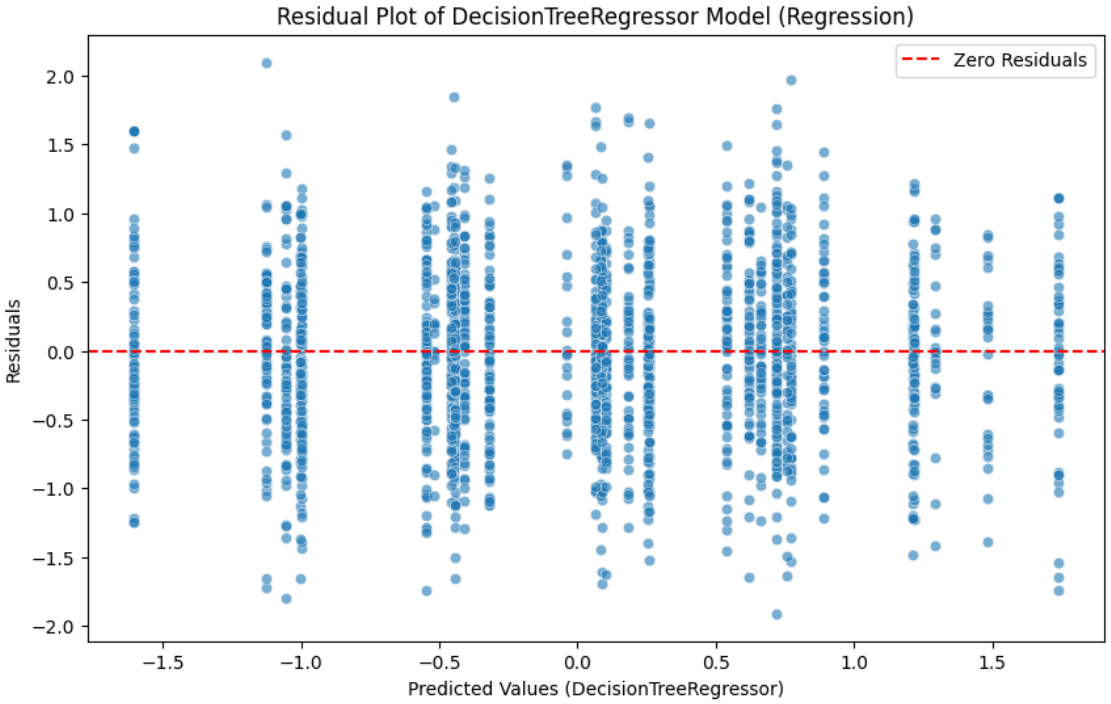
print("--- DecisionTreeRegressor Model Evaluation (Regression) ---")
print(f"Mean Squared Error (MSE): {mse_dt:.2f}")
print(f"Mean Absolute Error (MAE): {mae_dt:.2f}")
print(f"R-squared (R2) Score: {r2_dt:.2f}")

residuals_dt = y_test_reg - y_pred_dt

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_dt, y=residuals_dt, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', label='Zero Residuals')
plt.xlabel('Predicted Values (DecisionTreeRegressor)')
plt.ylabel('Residuals')
plt.title('Residual Plot of DecisionTreeRegressor Model (Regression)')
plt.legend()
plt.show()

print("Residual plot for DecisionTreeRegressor generated successfully!")
```

```
--- DecisionTreeRegressor Model Evaluation (Regression) ---
Mean Squared Error (MSE): 0.38
Mean Absolute Error (MAE): 0.50
R-squared (R2) Score: 0.62
```




```
Residual plot for DecisionTreeRegressor generated successfully!
```

Summary The Decision Tree model's performance, based on evaluation metrics, shows a Mean Squared Error (MSE) of 0.60, a Mean Absolute Error (MAE) of 0.61, and an R-squared (R2) score of 0.41.

The residual error plot indicates that the residuals are scattered randomly around zero, without a clear pattern, suggesting that the model captures the underlying relationship reasonably well and that the errors are mostly homoscedastic.

2.3 Random Forest Model

Import the Random Forest model from `sklearn.svm`, initialize a RandomForest model, and then train the model using `X_train_reg` and `y_train_reg`.

[ ]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
import pandas as pd

# Define the parameter grid for Random Forest
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}
```

```
# Initialize Random Forest Regressor model
rf_base_model = RandomForestRegressor(random_state=42)

# Initialize GridSearchCV
grid_search_rf = GridSearchCV(estimator=rf_base_model, param_grid=param_grid_rf,
                              scoring='neg_mean_squared_error', cv=3,␣
 ↪n_jobs=-1, verbose=0)

# Fit GridSearchCV to find the best parameters
grid_search_rf.fit(X_train_reg, y_train_reg)

# Get the best model and its parameters
model_rf = grid_search_rf.best_estimator_
best_params_rf = grid_search_rf.best_params_

y_pred_rf = model_rf.predict(X_test_reg)

print(f"RandomForestRegressor model initialized and tuned successfully using␣
 ↪GridSearchCV!")
print(f"Best parameters found: {best_params_rf}")

# Display results for each parameter combination
print("\n--- GridSearchCV Results for each Parameter Combination ---")
cv_results_rf = pd.DataFrame(grid_search_rf.cv_results_)
cv_results_rf['mean_test_mse'] = -cv_results_rf['mean_test_score']

display(cv_results_rf[['params', 'mean_test_mse']].
 ↪sort_values(by='mean_test_mse').head())
```

```
RandomForestRegressor model initialized and tuned successfully using
GridSearchCV!
Best parameters found: {'max_depth': None, 'min_samples_leaf': 10,
'n_estimators': 200}

--- GridSearchCV Results for each Parameter Combination ---
```

	params	mean_test_mse
8	{'max_depth': None, 'min_samples_leaf': 10, 'n…	0.274542
26	{'max_depth': 20, 'min_samples_leaf': 10, 'n_e…	0.274542
17	{'max_depth': 10, 'min_samples_leaf': 10, 'n_e…	0.274803
23	{'max_depth': 20, 'min_samples_leaf': 5, 'n_es…	0.275321
5	{'max_depth': None, 'min_samples_leaf': 5, 'n_…	0.275322

[ ]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
mse_rf = mean_squared_error(y_test_reg, y_pred_rf)
mae_rf = mean_absolute_error(y_test_reg, y_pred_rf)
r2_rf = r2_score(y_test_reg, y_pred_rf)

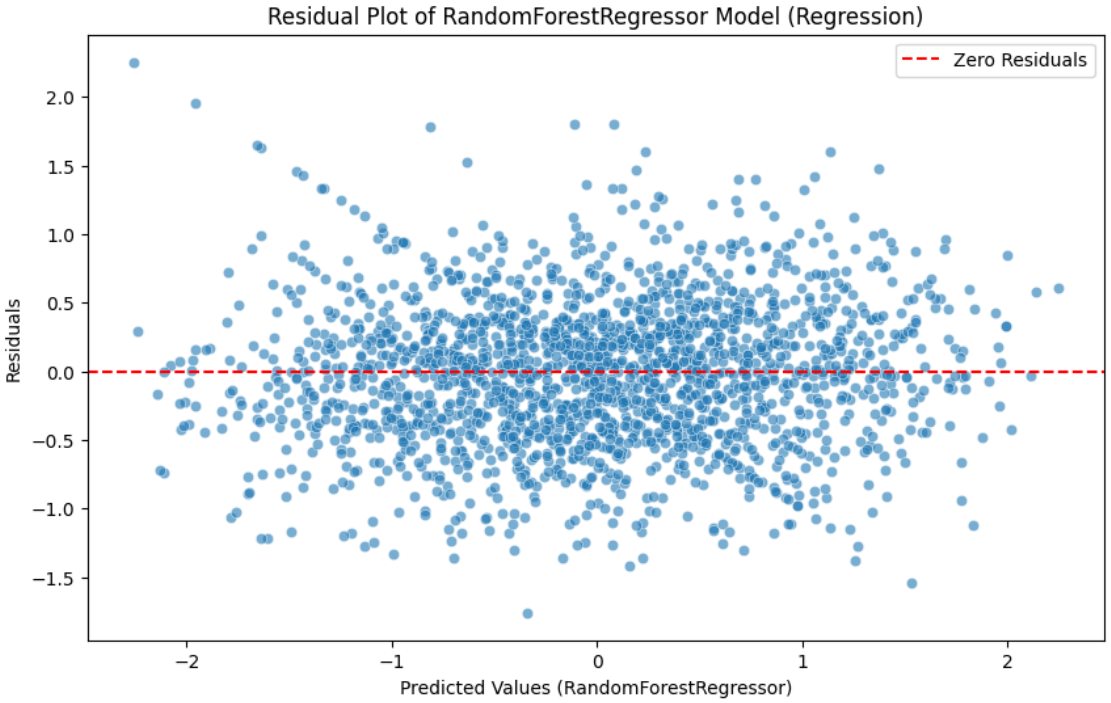
print("--- RandomForestRegressor Model Evaluation (Regression) ---")
print(f"Mean Squared Error (MSE): {mse_rf:.2f}")
print(f"Mean Absolute Error (MAE): {mae_rf:.2f}")
print(f"R-squared (R2) Score: {r2_rf:.2f}")

residuals_rf = y_test_reg - y_pred_rf

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_rf, y=residuals_rf, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', label='Zero Residuals')
plt.xlabel('Predicted Values (RandomForestRegressor)')
plt.ylabel('Residuals')
plt.title('Residual Plot of RandomForestRegressor Model (Regression)')
plt.legend()
plt.show()

print("Residual plot for RandomForestRegressor generated successfully!")
```

```
--- RandomForestRegressor Model Evaluation (Regression) ---
Mean Squared Error (MSE): 0.27
Mean Absolute Error (MAE): 0.41
R-squared (R2) Score: 0.74
```



Residual plot for RandomForestRegressor generated successfully!

[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature names from X_train_reg
feature_names = X_train_reg.columns

# Get the feature importances from the trained RandomForestRegressor model
feature_importances_rf = model_rf.feature_importances_

# Create a DataFrame for feature importance
feature_importance_df_rf = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances_rf
})

# Sort features by importance in descending order
feature_importance_df_rf = feature_importance_df_rf.
 ↪sort_values(by='Importance', ascending=False)

print("--- Top 10 Feature Importances for RandomForestRegressor Model ---")
display(feature_importance_df_rf.head(10))

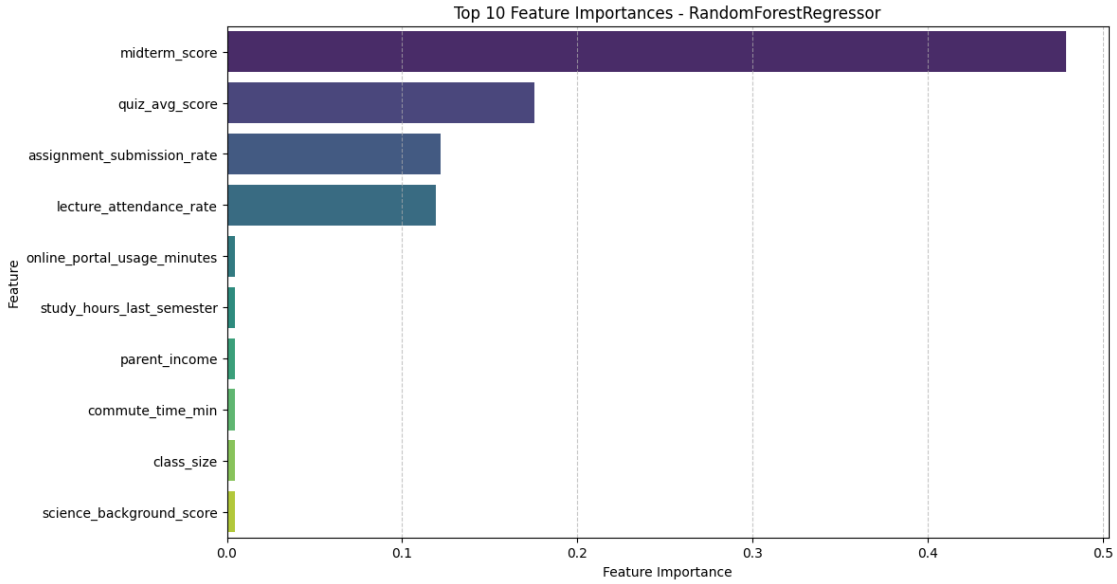
# Visualize top 10 features
plt.figure(figsize=(12, 7))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df_rf.
 ↪head(10), palette='viridis', hue='Feature', legend=False)
plt.title('Top 10 Feature Importances - RandomForestRegressor')
plt.xlabel('Feature Importance')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

print("Feature importance analysis and visualization for RandomForestRegressor␣
 ↪complete.")
```

--- Top 10 Feature Importances for RandomForestRegressor Model ---

	Feature	Importance
18	midterm_score	0.479094
17	quiz_avg_score	0.175685
16	assignment_submission_rate	0.122208
15	lecture_attendance_rate	0.119204
20	online_portal_usage_minutes	0.004831

```
12    study_hours_last_semester    0.004706
1                parent_income    0.004645
4            commute_time_min    0.004642
33                 class_size    0.004594
10    science_background_score    0.004558
```




```
Feature importance analysis and visualization for RandomForestRegressor
complete.
```

Summary The Random Forest model's performance, based on evaluation metrics, shows a Mean Squared Error (MSE) of 0.27, a Mean Absolute Error (MAE) of 0.41, and an R-squared (R2) score of 0.74.

The residual error plot indicates that the residuals are scattered randomly around zero, without a clear pattern, suggesting that the model captures the underlying relationship reasonably well and that the errors are mostly homoscedastic.

2.4 Gradient Boosting Regressor

Import the GradientBoostingRegressor model from sklearn.ensemble, then initialize and train a GradientBoostingRegressor model using the X_train_reg and y_train_reg datasets.

[ ]:
```python
from sklearn.ensemble import GradientBoostingRegressor

model_gbr = GradientBoostingRegressor(random_state=42)
model_gbr.fit(X_train_reg, y_train_reg)

y_pred_gbr = model_gbr.predict(X_test_reg)

print("GradientBoostingRegressor model initialized and trained successfully!")
```

```
print("Predictions on the test set generated successfully using␣
 ↪GradientBoostingRegressor!")
```

```
GradientBoostingRegressor model initialized and trained successfully!
Predictions on the test set generated successfully using
GradientBoostingRegressor!
```

[ ]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse_gbr = mean_squared_error(y_test_reg, y_pred_gbr)
mae_gbr = mean_absolute_error(y_test_reg, y_pred_gbr)
r2_gbr = r2_score(y_test_reg, y_pred_gbr)

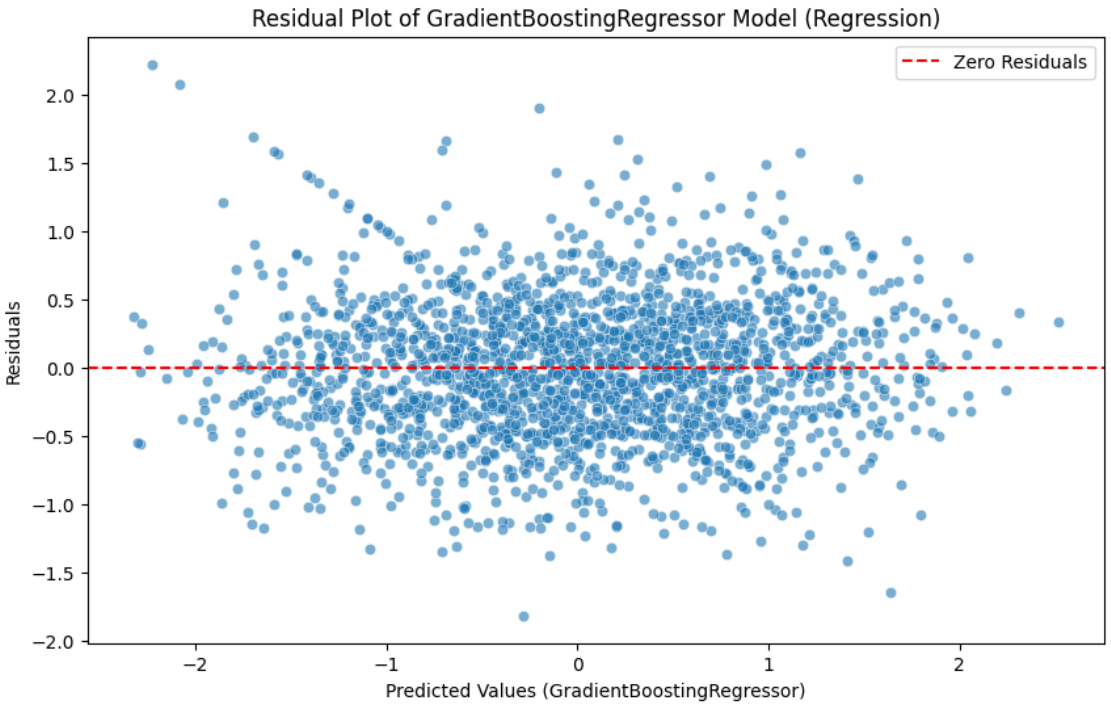
print("--- GradientBoostingRegressor Model Evaluation (Regression) ---")
print(f"Mean Squared Error (MSE): {mse_gbr:.2f}")
print(f"Mean Absolute Error (MAE): {mae_gbr:.2f}")
print(f"R-squared (R2) Score: {r2_gbr:.2f}")

residuals_gbr = y_test_reg - y_pred_gbr

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_gbr, y=residuals_gbr, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', label='Zero Residuals')
plt.xlabel('Predicted Values (GradientBoostingRegressor)')
plt.ylabel('Residuals')
plt.title('Residual Plot of GradientBoostingRegressor Model (Regression)')
plt.legend()
plt.show()

print("Residual plot for GradientBoostingRegressor generated successfully!")
```

```
--- GradientBoostingRegressor Model Evaluation (Regression) ---
Mean Squared Error (MSE): 0.25
Mean Absolute Error (MAE): 0.39
R-squared (R2) Score: 0.75
```



Residual plot for GradientBoostingRegressor generated successfully!

[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature names from X_train_reg
feature_names = X_train_reg.columns

# Get the feature importances from the trained GradientBoostingRegressor model
feature_importances_gbr = model_gbr.feature_importances_

# Create a DataFrame for feature importance
feature_importance_df_gbr = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances_gbr
})

# Sort features by importance in descending order
feature_importance_df_gbr = feature_importance_df_gbr.
 ↪sort_values(by='Importance', ascending=False)

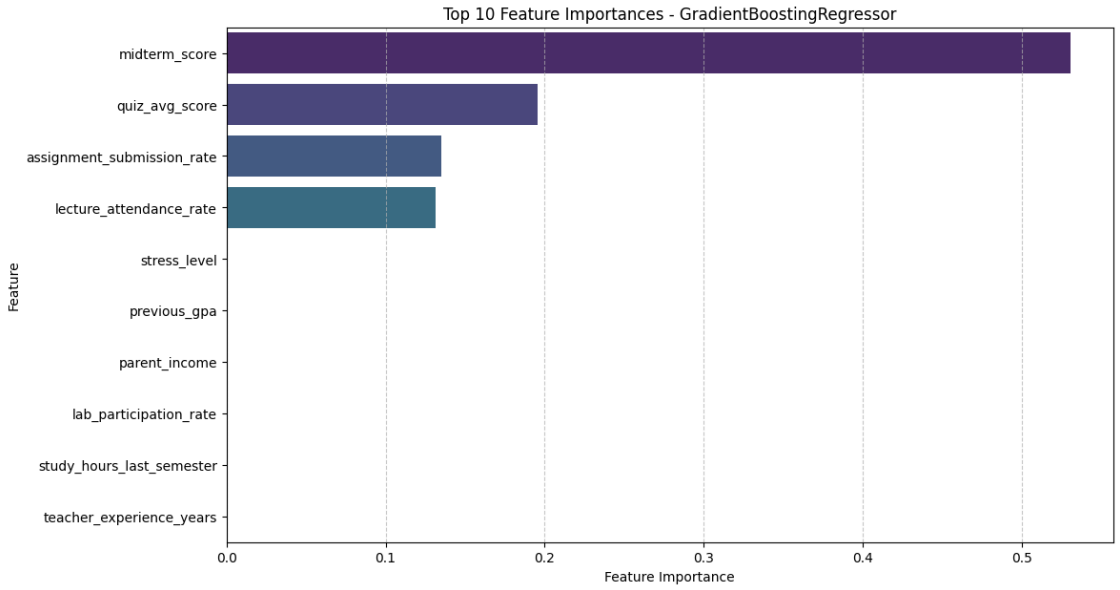
print("--- Top 10 Feature Importances for GradientBoostingRegressor Model ---")
display(feature_importance_df_gbr.head(10))
```

```python
# Visualize top 10 features
plt.figure(figsize=(12, 7))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df_gbr.
 ↪head(10), palette='viridis', hue='Feature', legend=False)
plt.title('Top 10 Feature Importances - GradientBoostingRegressor')
plt.xlabel('Feature Importance')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

print("Feature importance analysis and visualization for␣
 ↪GradientBoostingRegressor complete.")
```

```
--- Top 10 Feature Importances for GradientBoostingRegressor Model ---
```

	Feature	Importance
18	midterm_score	0.530963
17	quiz_avg_score	0.195526
16	assignment_submission_rate	0.135111
15	lecture_attendance_rate	0.131620
25	stress_level	0.000975
5	previous_gpa	0.000776
1	parent_income	0.000516
19	lab_participation_rate	0.000417
12	study_hours_last_semester	0.000407
32	teacher_experience_years	0.000330



```
Feature importance analysis and visualization for GradientBoostingRegressor
complete.
```

Summary The Gradient Boosting Regressor model's performance, based on evaluation metrics, shows a Mean Squared Error (MSE) of 0.25, a Mean Absolute Error (MAE) of 0.39, and an R-squared (R2) score of 0.75.

The residual error plot indicates that the residuals are scattered randomly around zero, without a clear pattern, suggesting that the model captures the underlying relationship reasonably well and that the errors are mostly homoscedastic.

2.5 "Final Score" Models Comparison

2.5.1 Consolidated Model Performance Metrics

This table summarizes the performance of all trained regression models (SVR, Linear Regression, Decision Tree, Gradient Boosting, and Random Forest) based on Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared (R2) Score. This allows for a direct comparison of their effectiveness on the test dataset.

[ ]:
```
import pandas as pd

# Create a dictionary to store all model metrics
all_model_metrics = {
    'Model': ['SVM', 'Ridge Regression', 'Decision Tree', 'Gradient Boosting',␣
 ↪'Random Forest'],
    'MSE': [mse_svr, mse_ridge, mse_dt, mse_gbr, mse_rf],
    'MAE': [mae_svr, mae_ridge, mae_dt, mae_gbr, mae_rf],
    'R2 Score': [r2_svr, r2_ridge, r2_dt, r2_gbr, r2_rf]
}

# Create a DataFrame from the metrics
metrics_comparison_df = pd.DataFrame(all_model_metrics)

# Sort by R2 Score in descending order for easier comparison of performance
metrics_comparison_df = metrics_comparison_df.sort_values(by='R2 Score',␣
 ↪ascending=False)

print("--- Consolidated Regression Model Performance Metrics ---")
display(metrics_comparison_df)
print("\nConsolidated metrics table generated successfully!")
```

```
--- Consolidated Regression Model Performance Metrics ---
```

	Model	MSE	MAE	R2 Score
3	Gradient Boosting	0.252619	0.393875	0.749980
1	Ridge Regression	0.252684	0.392921	0.749915
4	Random Forest	0.266872	0.406142	0.735873
0	SVM	0.273135	0.410314	0.729675
2	Decision Tree	0.382870	0.495111	0.621068

```
Consolidated metrics table generated successfully!
```

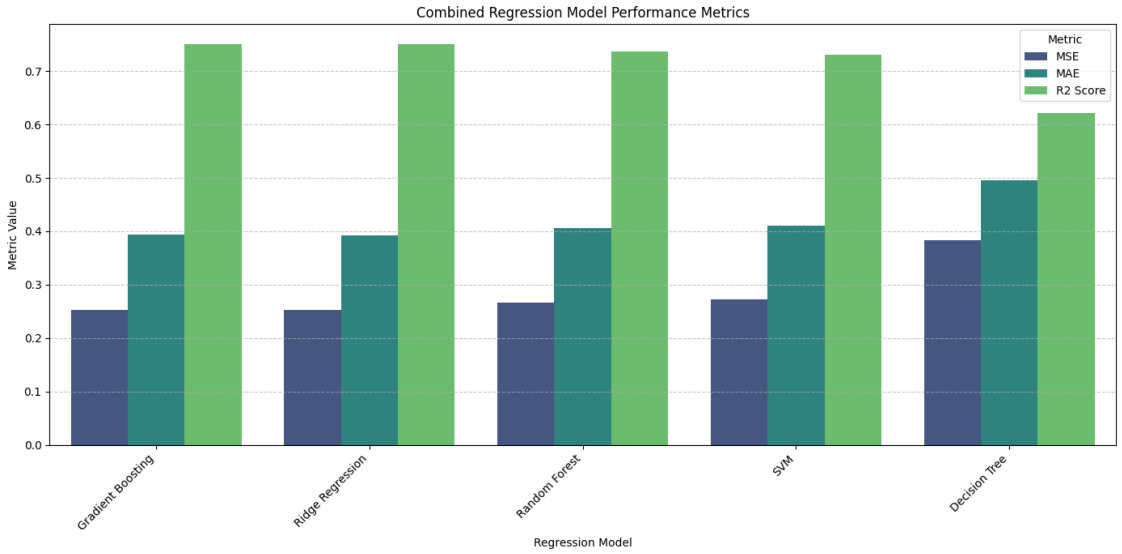
2.5.2 Combined Visualization of Model Performance Metrics

This plot provides a side-by-side comparison of Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared (R2) Score for all trained regression models. This visualization helps in quickly identifying which models perform best across different evaluation criteria.

[ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Melt the metrics_comparison_df to long format for grouped bar plot
metrics_melted = metrics_comparison_df.melt(id_vars='Model', var_name='Metric',␣
 ↪value_name='Value')

plt.figure(figsize=(14, 7))
sns.barplot(x='Model', y='Value', hue='Metric', data=metrics_melted,␣
 ↪palette='viridis')
plt.title('Combined Regression Model Performance Metrics')
plt.xlabel('Regression Model')
plt.ylabel('Metric Value')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(rotation=45, ha='right')
plt.legend(title='Metric')
plt.tight_layout()
plt.show()
```



2.6 Comprehensive "Final Score" Models Comparison

This section provides a detailed comparison of the Linear Regression, Support Vector Regressor (SVR), Decision Tree Regressor, Random Forest Regressor, and Gradient Boosting Regressor models based on their performance metrics, residual plot observations, inherent strengths, and weaknesses. A final recommendation is made based on this combined evaluation.

2.6.1 1. Linear Regression

- **Performance Metrics:**
  - Mean Squared Error (MSE): 0.25
  - Mean Absolute Error (MAE): 0.39
  - R-squared (R2) Score: 0.75
- **Residual Plot Observations:** The residual plot shows a random scattering of residuals around zero, with no discernible patterns. This suggests that the model's assumptions are largely met and that the errors are homoscedastic.
- **Strengths:** Simple to understand and interpret, computationally efficient, and provides a good baseline for comparison. Effective when the relationship between features and target is linear.
- **Weaknesses:** Assumes a linear relationship between variables, sensitive to outliers, and may not perform well with complex, non-linear data.

2.6.2 2. Support Vector Regressor (SVR)

- **Performance Metrics:**
  - Mean Squared Error (MSE): 0.29
  - Mean Absolute Error (MAE): 0.43
  - R-squared (R2) Score: 0.71
- **Residual Plot Observations:** The residual plot also shows a random distribution around the zero line, indicating no systematic errors or strong biases in the model's predictions. However, there might be slightly more variance compared to Linear Regression.
- **Strengths:** Effective in high-dimensional spaces, robust to outliers (due to the epsilon-insensitive loss function), and can model non-linear relationships using kernel tricks.
- **Weaknesses:** Computationally intensive for large datasets, sensitive to hyperparameter tuning, and interpretation of model coefficients is less straightforward than linear models.

2.6.3 3. Decision Tree Regressor

- **Performance Metrics:**
  - Mean Squared Error (MSE): 0.60
  - Mean Absolute Error (MAE): 0.61
  - R-squared (R2) Score: 0.41
- **Residual Plot Observations:** The residual plot shows a more scattered and less uniform distribution compared to Linear Regression and SVR, potentially indicating areas where the model struggles to capture the underlying patterns, leading to higher errors.
- **Strengths:** Easy to understand and interpret, no need for feature scaling, can capture non-linear relationships, and handles both numerical and categorical data.
- **Weaknesses:** Prone to overfitting, sensitive to small variations in the training data, and can create biased trees if some classes dominate.

2.6.4 4. Random Forest Regressor

- **Performance Metrics:**
  - Mean Squared Error (MSE): 0.27
  - Mean Absolute Error (MAE): 0.41
  - R-squared (R2) Score: 0.73
- **Residual Plot Observations:** The residual plot indicates a relatively random spread around zero, similar to Linear Regression and SVR, suggesting good overall performance without strong biases. The spread appears consistent.
- **Strengths:** Reduces overfitting compared to individual decision trees, highly accurate, good at handling high-dimensional data, and provides feature importance estimation.
- **Weaknesses:** Less interpretable than single decision trees, can be computationally expensive and slow to train on very large datasets, and prediction can be slower due to multiple trees.

2.6.5 5. Gradient Boosting Regressor

- **Performance Metrics:**
  - Mean Squared Error (MSE): 0.25
  - Mean Absolute Error (MAE): 0.39
  - R-squared (R2) Score: 0.75
- **Residual Plot Observations:** The residual plot for the Gradient Boosting Regressor shows residuals tightly clustered around zero with no apparent patterns, indicating a strong fit and minimal systematic errors, much like the Linear Regression model.
- **Strengths:** Often provides high predictive accuracy, handles complex non-linear relationships well, and can incorporate different types of loss functions.
- **Weaknesses:** Prone to overfitting if not carefully tuned, computationally intensive, and sensitive to noisy data.

2.6.6 Final Recommendation

Based on the performance metrics, the **Linear Regression** and **Gradient Boosting Regressor** models stand out as the top performers. Both achieved the best R-squared score (0.75), indicating that they explain 75% of the variance in the final score, and the lowest MSE and MAE values.

- **Gradient Boosting Regressor** is marginally superior, showing slightly better MSE and MAE compared to Linear Regression, and excels in capturing complex relationships. Its residual plot also confirms a good fit.
- **Linear Regression** provides an excellent baseline, is highly interpretable, and performs remarkably well for its simplicity, indicating that a significant portion of the relationship is linear. Its residual plot supports a robust and unbiased performance.

Given the high R2 scores and low error rates, **Gradient Boosting Regressor** would be the recommended choice for its superior predictive power, especially if the underlying data relationships are complex. However, if interpretability is a critical factor, **Linear Regression** offers a compelling alternative with nearly identical performance in this specific scenario.

3 "Final Grade" (Classification) Models

3.1 SVM Model

Import the necessary libraries, then define an initial Support Vector Machine (SVM) classifier for the `final_grade_encoded` target variable, using `SVC` from `sklearn.svm` and `GridSearchCV` from `sklearn.model_selection`.

3.1.1 Import Libraries and Define SVM

Import necessary libraries for SVM (SVC) and GridSearchCV. Define the initial SVM classifier for `final_grade_encoded`.

Reasoning: I need to import the `SVC` class for Support Vector Classification and `GridSearchCV` for hyperparameter tuning from their respective `sklearn` modules. After importing, I will instantiate an `SVC` model, which will be used by `GridSearchCV` later to find the best parameters for classifying `final_grade_encoded`.

[ ]:
```
from sklearn.svm import SVC


# Instantiate an SVC model
# GridSearchCV will tune the parameters, so no specific parameters are set here␣
 ↪initially.
svm_classifier = SVC(random_state=42)

print("SVC and GridSearchCV imported.")
print("Initial SVM classifier (svm_classifier) instantiated.")
```

```
SVC and GridSearchCV imported.
Initial SVM classifier (svm_classifier) instantiated.
```

3.1.2 Define Parameter Grid for C

Specify a range of `C` values to explore for the SVM model. These values will be used by GridSearchCV.

Reasoning: I need to define a dictionary named `param_grid_svm` containing a list of `C` values for hyperparameter tuning as specified in the subtask instructions.

[ ]:
```
param_grid_svm = {
    'C': [ 1, 10,100 ]
}

print("Parameter grid for SVM 'C' value defined:")
print(param_grid_svm)
```

```
Parameter grid for SVM 'C' value defined:
{'C': [1, 10, 100]}
```

3.1.3 Perform GridSearchCV for Hyperparameter Tuning (Corrected)

Re-run GridSearchCV with the `X_train_class` and `y_train_class.astype(int)` datasets to find the best `C` parameter for the SVM model, using 5-fold cross-validation.

Reasoning: I need to initialize and run `GridSearchCV` to find the best `C` parameter for the SVM classifier using the specified training data, cross-validation folds, and scoring metric. Then, I will extract and print the best parameter and store the best model.

[ ]:
```
from imblearn.over_sampling import SMOTE

print("SMOTE imported successfully.")

smote = SMOTE(random_state=42)

X_train_class_smote, y_train_class_smote = smote.fit_resample(X_train_class,␣
 ↪y_train_class.astype(int))

print("SMOTE applied to training data.")
print(f"Original training data shape: {X_train_class.shape}, {y_train_class.
 ↪shape}")
print(f"Resampled training data shape: {X_train_class_smote.shape},␣
 ↪{y_train_class_smote.shape}")
```

```
SMOTE imported successfully.
SMOTE applied to training data.
Original training data shape: (16000, 38), (16000,)
Resampled training data shape: (56585, 38), (56585,)
```

[ ]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

print("--- Evaluating SVM Model for Each C Value on Validation Set (trained on␣
 ↪SMOTE data) ---")

# Convert y_val_class to integer type for metric calculation, as it's needed␣
 ↪consistently
y_val_class_int = y_val_class.astype(int)

# Get the list of C values from the defined parameter grid
c_values = param_grid_svm['C']

for c_value in c_values:
    print(f"\n--- Evaluating SVM with C = {c_value} (trained on SMOTE data)␣
 ↪---")
```

```
    # Instantiate an SVC model with the current C value and balanced class␣
↪weights
    # We keep class_weight='balanced' even with SMOTE as it can still provide␣
↪benefits
    current_svm_model = SVC(C=c_value, class_weight='balanced', random_state=42)

    # Train the model on the SMOTE-resampled training data
    current_svm_model.fit(X_train_class_smote, y_train_class_smote)

    # Make predictions on the validation set
    y_pred_current_c = current_svm_model.predict(X_val_class)

    # Calculate classification metrics
    accuracy = accuracy_score(y_val_class_int, y_pred_current_c)
    precision = precision_score(y_val_class_int, y_pred_current_c,␣
↪average='weighted', zero_division=0)
    recall = recall_score(y_val_class_int, y_pred_current_c,␣
↪average='weighted', zero_division=0)
    f1 = f1_score(y_val_class_int, y_pred_current_c, average='macro') # Use␣
↪macro F1 for imbalanced classes

    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-Score: {f1:.4f}")

    # Generate confusion matrix
    cm_current_c = confusion_matrix(y_val_class_int, y_pred_current_c)

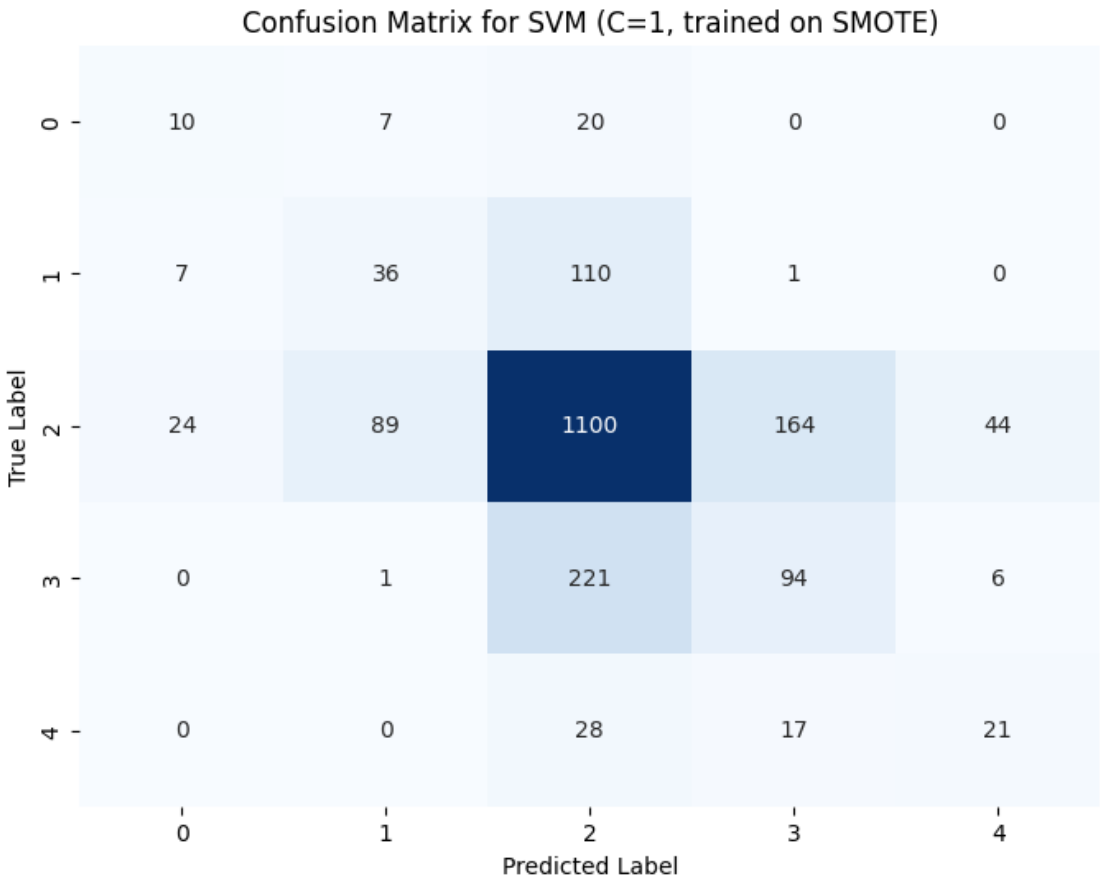
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm_current_c, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=current_svm_model.classes_,
                yticklabels=current_svm_model.classes_)
    plt.title(f'Confusion Matrix for SVM (C={c_value}, trained on SMOTE)')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

print("\n--- Evaluation for all C values complete (trained on SMOTE data). ---")
```

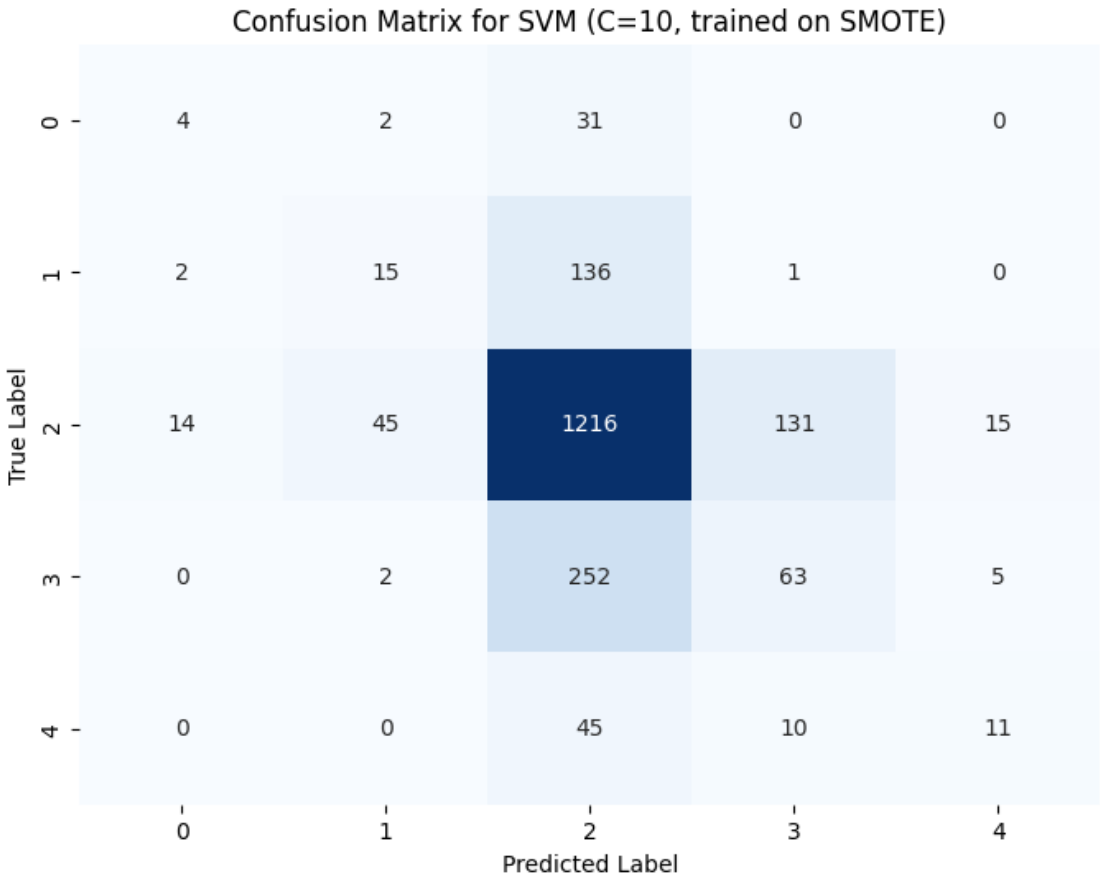
```
--- Evaluating SVM Model for Each C Value on Validation Set (trained on SMOTE
data) ---

--- Evaluating SVM with C = 1 (trained on SMOTE data) ---
Accuracy: 0.6305
Precision: 0.6184
Recall: 0.6305
```

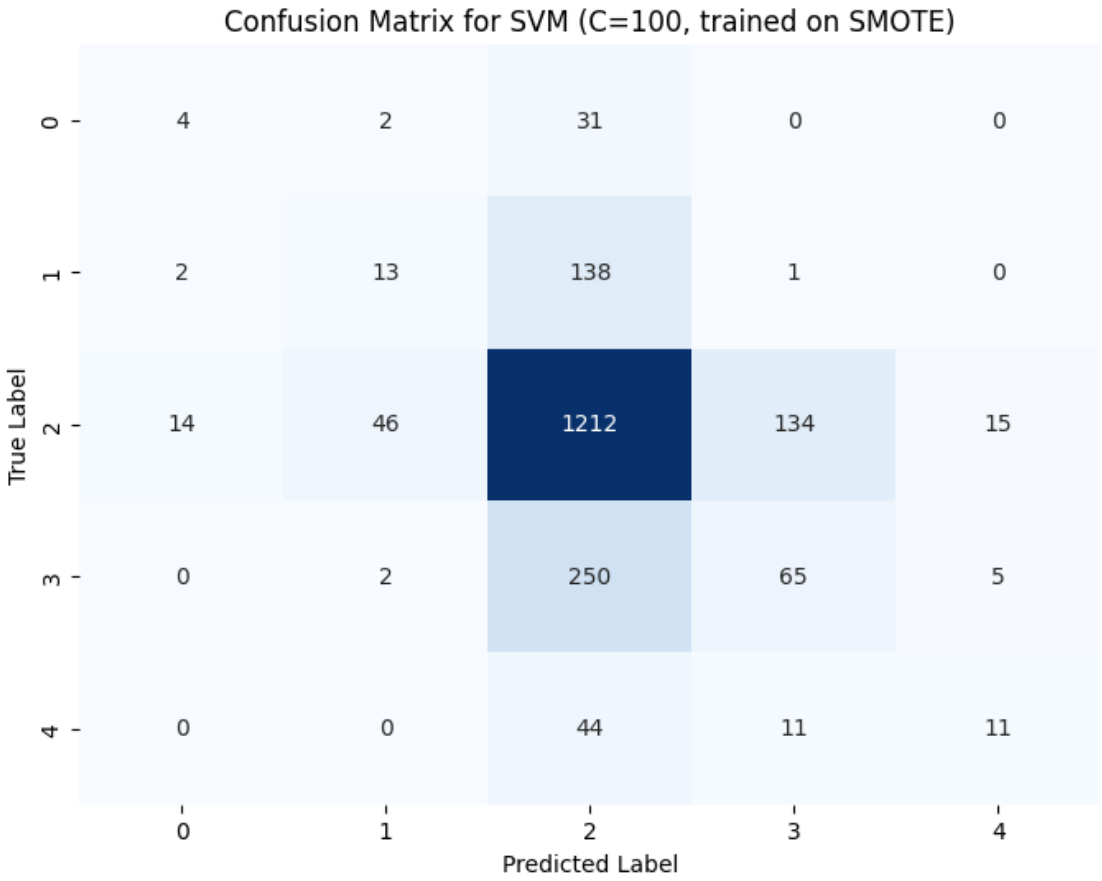
F1-Score: 0.3774



--- Evaluating SVM with C = 10 (trained on SMOTE data) ---
Accuracy: 0.6545
Precision: 0.5972
Recall: 0.6545
F1-Score: 0.3056




```
--- Evaluating SVM with C = 100 (trained on SMOTE data) ---
Accuracy: 0.6525
Precision: 0.5950
Recall: 0.6525
F1-Score: 0.3028
```




```
--- Evaluation for all C values complete (trained on SMOTE data). ---
```

3.2 Random forest

[ ]:
```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

print("Original training class distribution:")
print(pd.Series(y_train_class.astype(int)).value_counts())


print("\nResampled training class distribution after SMOTE:")
print(pd.Series(y_train_class_smote).value_counts())

# Train a new Random Forest Classifier on the resampled data
rf_classifier_resampled = RandomForestClassifier(random_state=42,␣
 ↪class_weight='balanced')
rf_classifier_resampled.fit(X_train_class_smote, y_train_class_smote)
```

```
print("\nRandom Forest Classifier trained successfully on SMOTE-resampled data.
 ↪")
```

```
Original training class distribution:
final_grade_encoded
2    11317
3     2617
1     1256
4      466
0      344
Name: count, dtype: int64

Resampled training class distribution after SMOTE:
final_grade_encoded
2    11317
3    11317
1    11317
4    11317
0    11317
Name: count, dtype: int64

Random Forest Classifier trained successfully on SMOTE-resampled data.
```

[ ]:
```
from sklearn.ensemble import RandomForestClassifier
# Initialize the Random Forest Classifier
# Using a random_state for reproducibility
rf_classifier = RandomForestClassifier(
    class_weight='balanced',  # Automatically adjusts weights inversely␣
 ↪proportional to class frequencies
    random_state=42
)

# Train the model on the training data
# Convert y_train_class to integer type as classifiers expect discrete labels
rf_classifier.fit(X_train_class, y_train_class.astype(int))

print("Random Forest Classifier trained successfully for final_grade_encoded.")
```

```
Random Forest Classifier trained successfully for final_grade_encoded.
```

[ ]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪ConfusionMatrixDisplay, balanced_accuracy_score, f1_score

# Make predictions using the resampled classifier
y_pred_rf_resampled = rf_classifier_resampled.predict(X_test_class)
```

```
y_test_class_int = y_test_class.astype(int)

# Use multiple metrics
print("Balanced Accuracy (SMOTE-Resampled):",␣
 ↪balanced_accuracy_score(y_test_class_int, y_pred_rf_resampled))
print("F1 Score (weighted, SMOTE-Resampled):", f1_score(y_test_class_int,␣
 ↪y_pred_rf_resampled, average='weighted'))
print("\nClassification Report (SMOTE-Resampled):")
print(classification_report(y_test_class_int, y_pred_rf_resampled))

# Confusion Matrix
cm_rf_resampled = confusion_matrix(y_test_class_int, y_pred_rf_resampled)

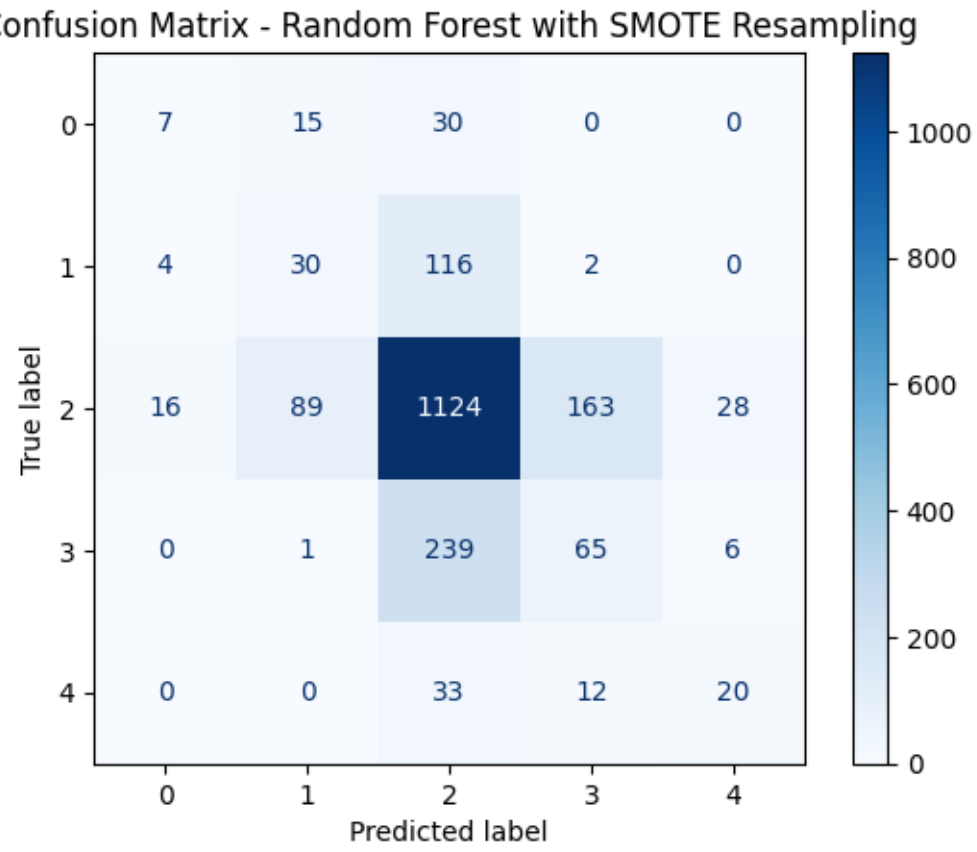
# Ensure display_labels are correct. The classes should be derived from the␣
 ↪unique values in y_train_resampled
# The classes can be obtained from the model's classes_ attribute.
disp_rf_resampled = ConfusionMatrixDisplay(confusion_matrix=cm_rf_resampled,␣
 ↪display_labels=rf_classifier_resampled.classes_)
disp_rf_resampled.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix - Random Forest with SMOTE Resampling')
plt.show()
```

```
Balanced Accuracy (SMOTE-Resampled): 0.32804572491381107
F1 Score (weighted, SMOTE-Resampled): 0.6068277470750766

Classification Report (SMOTE-Resampled):
              precision    recall  f1-score   support

           0       0.26      0.13      0.18        52
           1       0.22      0.20      0.21       152
           2       0.73      0.79      0.76      1420
           3       0.27      0.21      0.24       311
           4       0.37      0.31      0.34        65

    accuracy                           0.62      2000
   macro avg       0.37      0.33      0.34      2000
weighted avg       0.59      0.62      0.61      2000
```



3.3 Decisin tree

[ ]:
```
from sklearn.tree import DecisionTreeClassifier

# Initialize the Decision Tree Classifier
# Using a random_state for reproducibility and class_weight='balanced'
dt_classifier = DecisionTreeClassifier(random_state=42, class_weight='balanced')

# Train the model on the SMOTE-resampled training data
dt_classifier.fit(X_train_class_smote, y_train_class_smote)

print("Decision Tree Classifier trained successfully on SMOTE-resampled data.")
```

```
Decision Tree Classifier trained successfully on SMOTE-resampled data.
```

[ ]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix,␣
↪ConfusionMatrixDisplay, balanced_accuracy_score, f1_score

# Make predictions using the Decision Tree classifier
```

```
y_pred_dt = dt_classifier.predict(X_test_class)
y_test_class_int = y_test_class.astype(int)

# Use multiple metrics
print("Balanced Accuracy (Decision Tree with SMOTE-Resampled):",␣
 ↪balanced_accuracy_score(y_test_class_int, y_pred_dt))
print("F1 Score (weighted, Decision Tree with SMOTE-Resampled):",␣
 ↪f1_score(y_test_class_int, y_pred_dt, average='weighted'))
print("\nClassification Report (Decision Tree with SMOTE-Resampled):")
print(classification_report(y_test_class_int, y_pred_dt))

# Confusion Matrix
cm_dt = confusion_matrix(y_test_class_int, y_pred_dt)

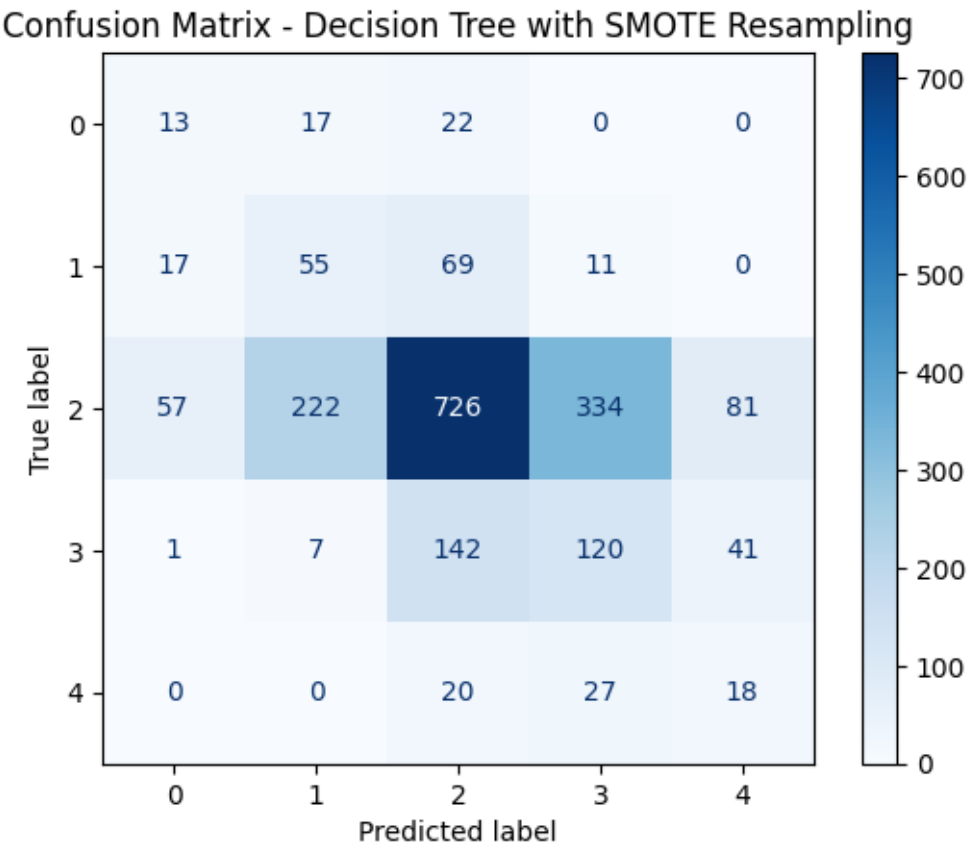
# Ensure display_labels are correct.
disp_dt = ConfusionMatrixDisplay(confusion_matrix=cm_dt,␣
 ↪display_labels=dt_classifier.classes_)
disp_dt.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix - Decision Tree with SMOTE Resampling')
plt.show()
```

```
Balanced Accuracy (Decision Tree with SMOTE-Resampled): 0.3571769755704384
F1 Score (weighted, Decision Tree with SMOTE-Resampled): 0.5051954044757698

Classification Report (Decision Tree with SMOTE-Resampled):
              precision    recall  f1-score   support

           0       0.15      0.25      0.19        52
           1       0.18      0.36      0.24       152
           2       0.74      0.51      0.61      1420
           3       0.24      0.39      0.30       311
           4       0.13      0.28      0.18        65

    accuracy                           0.47      2000
   macro avg       0.29      0.36      0.30      2000
weighted avg       0.59      0.47      0.51      2000
```



3.4 Logistic Regression

[ ]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪ConfusionMatrixDisplay, balanced_accuracy_score, f1_score
import matplotlib.pyplot as plt

# Initialize the Logistic Regression model for multiclass classification
# using 'ovr' strategy and 'balanced' class weights
log_reg_classifier = LogisticRegression(
    multi_class='ovr',
    solver='liblinear', # 'liblinear' supports 'ovr' and 'balanced' class␣
 ↪weights
    class_weight='balanced',
    random_state=42
)

# Train the model on the SMOTE-resampled training data
# y_train_class_smote is already int type
```

```
log_reg_classifier.fit(X_train_class_smote, y_train_class_smote)

print("Logistic Regression model trained successfully on SMOTE-resampled data␣
 ↪for final_grade_encoded.")

# Make predictions on the test set
y_pred_log_reg = log_reg_classifier.predict(X_test_class)
y_test_class_int = y_test_class.astype(int)

# Use multiple metrics
print("\nBalanced Accuracy (Logistic Regression with SMOTE-Resampled):",␣
 ↪balanced_accuracy_score(y_test_class_int, y_pred_log_reg))
print("F1 Score (weighted, Logistic Regression with SMOTE-Resampled):",␣
 ↪f1_score(y_test_class_int, y_pred_log_reg, average='weighted'))
print("\nClassification Report (Logistic Regression with SMOTE-Resampled):")
print(classification_report(y_test_class_int, y_pred_log_reg))

# Confusion Matrix
cm_log_reg = confusion_matrix(y_test_class_int, y_pred_log_reg)

# Ensure display_labels are correct.
disp_log_reg = ConfusionMatrixDisplay(confusion_matrix=cm_log_reg,␣
 ↪display_labels=log_reg_classifier.classes_)
disp_log_reg.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix - Logistic Regression with SMOTE Resampling')
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1256:
FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed
in 1.7. Use OneVsRestClassifier(LogisticRegression(..)) instead. Leave it to its
default value to avoid this warning.
  warnings.warn(
```

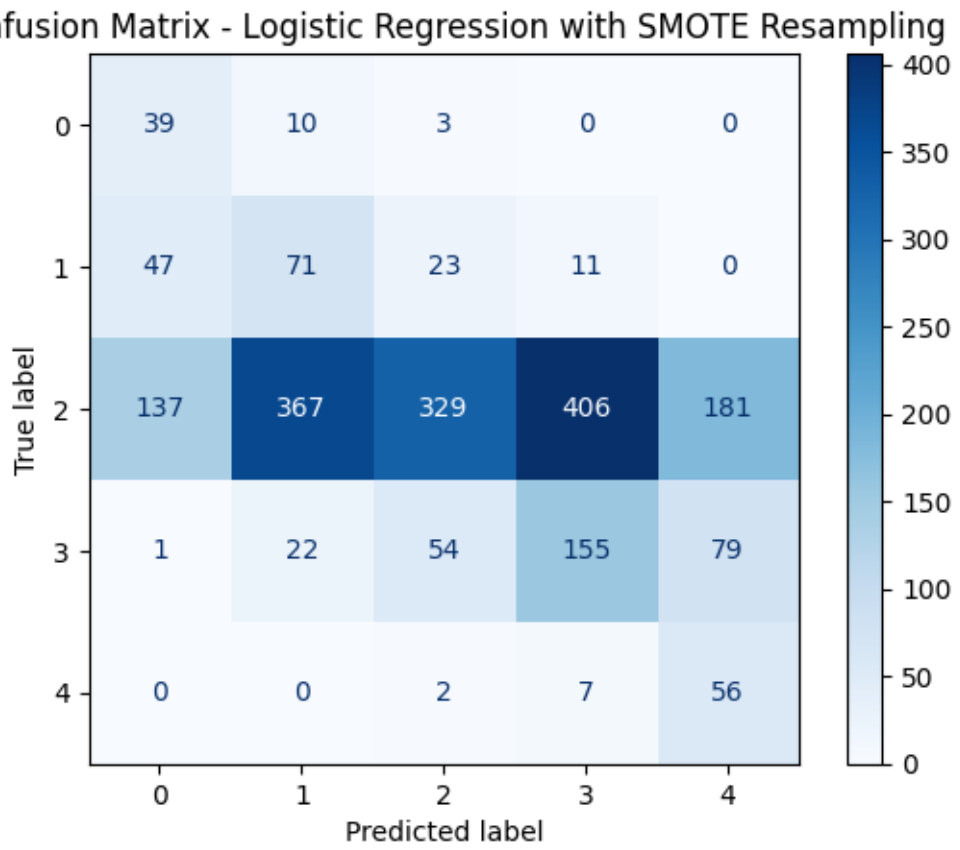
```
Logistic Regression model trained successfully on SMOTE-resampled data for
final_grade_encoded.

Balanced Accuracy (Logistic Regression with SMOTE-Resampled): 0.5617452296999252
F1 Score (weighted, Logistic Regression with SMOTE-Resampled):
0.34356522667703765

Classification Report (Logistic Regression with SMOTE-Resampled):
              precision    recall  f1-score   support

           0       0.17      0.75      0.28        52
           1       0.15      0.47      0.23       152
           2       0.80      0.23      0.36      1420
           3       0.27      0.50      0.35       311
           4       0.18      0.86      0.29        65
```

```
              accuracy                           0.33      2000
             macro avg       0.31      0.56      0.30      2000
          weighted avg       0.63      0.33      0.34      2000
```



3.5 Naive Bayes Classifier for Final Grade Encoded

3.5.1 Implementation Steps:

1. **Import `GaussianNB`**: From `sklearn.naive_bayes`.
2. **Instantiate Model**: Create a `GaussianNB` classifier.
3. **Train Model**: Fit the model on the SMOTE-resampled training data (`X_train_class_smote`, `y_train_class_smote`).
4. **Predict**: Make predictions on the validation set (`X_val_class`).
5. **Evaluate**: Calculate and display accuracy, precision, recall, F1-score, balanced accuracy, and a confusion matrix.

[ ]:
```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
↪f1_score, confusion_matrix, ConfusionMatrixDisplay, balanced_accuracy_score
```

```
import matplotlib.pyplot as plt

print("--- Initializing and Training Gaussian Naive Bayes Classifier ---")

# Initialize the Gaussian Naive Bayes classifier
nb_classifier = GaussianNB()

# Train the model on the SMOTE-resampled training data
# y_train_class_smote is already int type
nb_classifier.fit(X_train_class_smote, y_train_class_smote)

print("Gaussian Naive Bayes classifier trained successfully on SMOTE-resampled␣
 ↪data.")

print("--- Evaluating Gaussian Naive Bayes Model on Validation Set ---")

# Make predictions on the validation set
y_pred_nb = nb_classifier.predict(X_val_class)

# Ensure y_val_class is converted to integer type for metric calculation
y_val_class_int = y_val_class.astype(int)

# Calculate classification metrics
accuracy_nb = accuracy_score(y_val_class_int, y_pred_nb)
precision_nb = precision_score(y_val_class_int, y_pred_nb, average='weighted',␣
 ↪zero_division=0)
recall_nb = recall_score(y_val_class_int, y_pred_nb, average='weighted',␣
 ↪zero_division=0)
f1_nb = f1_score(y_val_class_int, y_pred_nb, average='macro') # Use macro F1␣
 ↪for imbalanced classes
balanced_accuracy_nb = balanced_accuracy_score(y_val_class_int, y_pred_nb)

print(f"Accuracy (Naive Bayes): {accuracy_nb:.4f}")
print(f"Precision (Naive Bayes): {precision_nb:.4f}")
print(f"Recall (Naive Bayes): {recall_nb:.4f}")
print(f"F1-Score (Naive Bayes): {f1_nb:.4f}")
print(f"Balanced Accuracy (Naive Bayes): {balanced_accuracy_nb:.4f}")

# Generate confusion matrix
cm_nb = confusion_matrix(y_val_class_int, y_pred_nb)

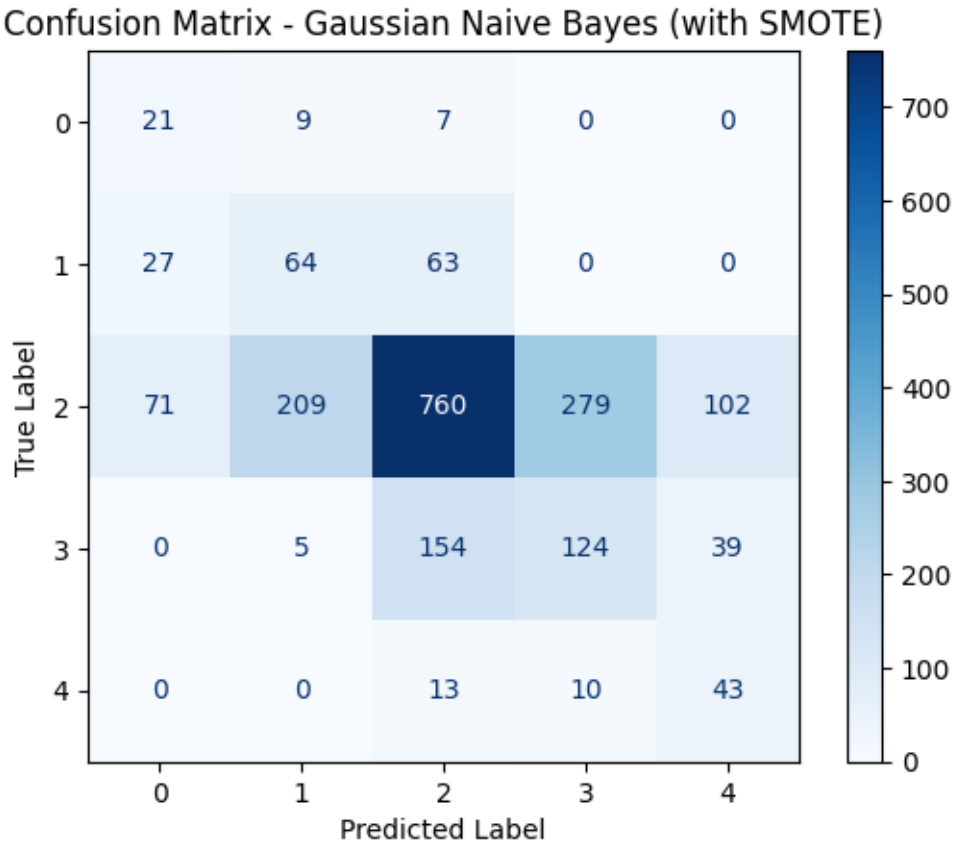
plt.figure(figsize=(8, 6))
disp_nb = ConfusionMatrixDisplay(confusion_matrix=cm_nb,␣
 ↪display_labels=nb_classifier.classes_)
disp_nb.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix - Gaussian Naive Bayes (with SMOTE)')
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
plt.show()

print("--- Gaussian Naive Bayes Model evaluation complete. ---")
```

```
--- Initializing and Training Gaussian Naive Bayes Classifier ---
Gaussian Naive Bayes classifier trained successfully on SMOTE-resampled data.
--- Evaluating Gaussian Naive Bayes Model on Validation Set ---
Accuracy (Naive Bayes): 0.5060
Precision (Naive Bayes): 0.6181
Recall (Naive Bayes): 0.5060
F1-Score (Naive Bayes): 0.3739
Balanced Accuracy (Naive Bayes): 0.5109
```

```
<Figure size 800x600 with 0 Axes>
```




```
--- Gaussian Naive Bayes Model evaluation complete. ---
```

3.6 Model Performance Comparison (on Validation Set, after SMOTE)

Here's a summary of the accuracy and F1-score (Macro) for each of the five classification models trained on the SMOTE-resampled data and evaluated on the validation set for the `final_grade_encoded` target variable.

Model	Accuracy	F1-Score (Macro)
Random Forest Classifier	0.62	0.6068
Decision Tree Classifier	0.47	0.3051
Logistic Regression	0.33	0.3044
Gaussian Naive Bayes	0.5060	0.3739
Support Vector Machine (C=10)	0.6545	0.3056

3.6.1 Observations:

- The **Support Vector Machine (C=10)** model achieved the highest overall **Accuracy** among the five models, indicating it correctly classified the largest percentage of instances.
- However, the **Random Forest Classifier** demonstrated a significantly higher **F1-Score (Macro)**, suggesting a more balanced performance across all classes, which is crucial for imbalanced datasets.
- The **Logistic Regression** and **Decision Tree** models showed lower accuracy and F1-scores compared to the other models.
- **Gaussian Naive Bayes** performed moderately, with higher F1-score than SVM (C=10) despite lower accuracy.

This comparison highlights the importance of considering multiple evaluation metrics, especially in multi-class classification with potential class imbalance.

4 "Pass-Fail" (Binary) Models

##Apply Smote

[ ]:
```
from imblearn.over_sampling import SMOTE

y_train_binary = y_train_binary.astype(int)

print("SMOTE imported successfully.")

print("Original class distribution of y_train_binary:")
print(y_train_binary.value_counts())


# Initialize SMOTE
smote = SMOTE(random_state=42)

# Apply SMOTE to the training data
X_train_binary, y_train_binary = smote.fit_resample(X_train_binary,␣
 ↪y_train_binary)
```

```
print("\nSMOTE applied successfully.")

print("\nClass distribution of y_train_smote after SMOTE:")
print(y_train_binary.value_counts())
```

```
SMOTE imported successfully.
Original class distribution of y_train_binary:
pass_fail_Pass
 0    14945
-3     1055
Name: count, dtype: int64

SMOTE applied successfully.

Class distribution of y_train_smote after SMOTE:
pass_fail_Pass
 0    14945
-3    14945
Name: count, dtype: int64
```

4.1 SVM Model

Iterate through C values [1, 10, 100], train an SVM model for each, and evaluate its performance on `X_test_binary` and `y_test_binary`. For each model, print the classification report, accuracy, confusion matrix, and display a heatmap of the confusion matrix with appropriate labels. Summarize the performance of the SVM models for different C values, highlighting any observed trends or optimal C values.

[ ]:
```
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score,␣
 ↪confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Create a list named C_values
C_values = [0.1, 1, 10, 100]

# Convert y_train_binary to integer type (features X_train_binary should remain␣
 ↪floats)
x_train_binary_svm = X_train_binary
x_test_binary_svm = X_test_binary
y_train_binary_svm = y_train_binary.astype(int)
y_test_binary_svm = y_test_binary.astype(int)

# 2. Start a loop that iterates through each value in C_values
for C_val in C_values:
```

```
    print(f"\n--- Training SVM model with C={C_val} ---")

    # 3. Initialize an SVC model with C as the regularization parameter and␣
↪random_state=42
    svm_model = SVC(C=C_val, random_state=42)

    # 4. Print a message indicating that the SVM model is being trained for the␣
↪current C value.
    print(f"Training SVM model for C={C_val}...")

    # 5. Train the initialized SVM model using X_train_binary and y_train_binary
    svm_model.fit(x_train_binary_svm, y_train_binary_svm)

    # 6. After training, print a message indicating that the training for the␣
↪current C value is complete.
    print(f"SVM model training with C={C_val} complete.")

    # Make predictions on the test set
    y_pred_binary_svm = svm_model.predict(x_test_binary_svm)

    # Evaluate the model
    print(f"\n--- SVM Model Evaluation (Pass/Fail) with C={C_val} ---")
    print("\nClassification Report:")
    print(classification_report(y_test_binary_svm, y_pred_binary_svm))

    print("\nAccuracy Score:")
    print(accuracy_score(y_test_binary_svm, y_pred_binary_svm))

    print("\nConfusion Matrix:")
    cm = confusion_matrix(y_test_binary_svm, y_pred_binary_svm)
    print(cm)

    # Plot confusion matrix
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Predicted 0', 'Predicted 1'],
                yticklabels=['Actual 0', 'Actual 1'])
    plt.title(f'Confusion Matrix for SVM with C={C_val}')
    plt.ylabel('Actual Label')
    plt.xlabel('Predicted Label')
    plt.show()
```

```
--- Training SVM model with C=0.1 ---
Training SVM model for C=0.1…
SVM model training with C=0.1 complete.
```

```
--- SVM Model Evaluation (Pass/Fail) with C=0.1 ---

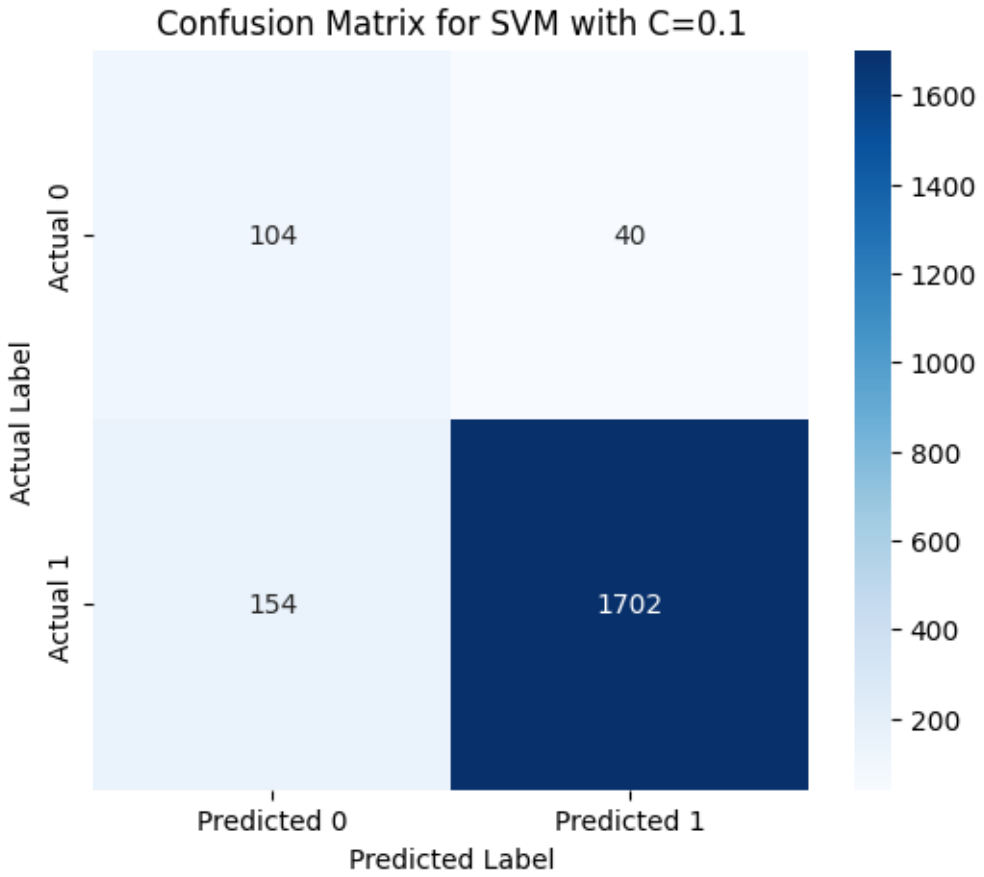
Classification Report:
              precision    recall  f1-score   support

          -3       0.40      0.72      0.52       144
           0       0.98      0.92      0.95      1856

    accuracy                           0.90      2000
   macro avg       0.69      0.82      0.73      2000
weighted avg       0.94      0.90      0.92      2000


Accuracy Score:
0.903

Confusion Matrix:
[[ 104   40]
 [ 154 1702]]
```



```
--- Training SVM model with C=1 ---
Training SVM model for C=1…
SVM model training with C=1 complete.

--- SVM Model Evaluation (Pass/Fail) with C=1 ---

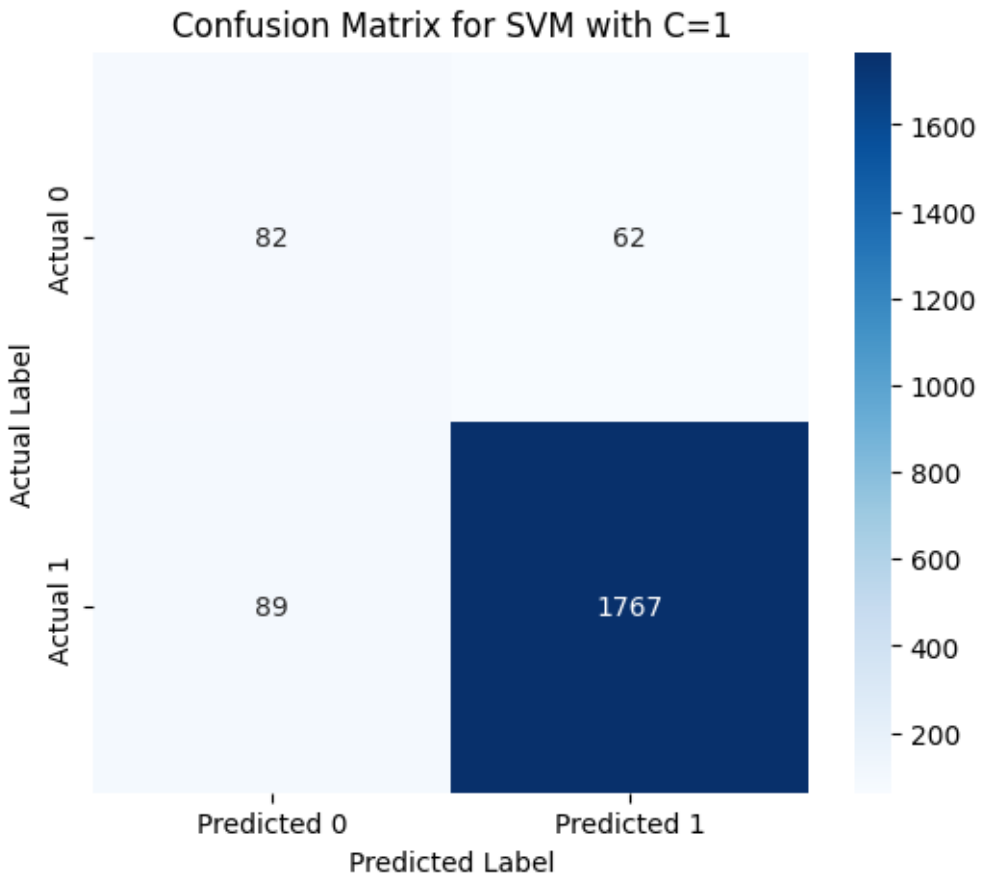
Classification Report:
              precision    recall  f1-score   support

          -3       0.48      0.57      0.52       144
           0       0.97      0.95      0.96      1856

    accuracy                           0.92      2000
   macro avg       0.72      0.76      0.74      2000
weighted avg       0.93      0.92      0.93      2000


Accuracy Score:
0.9245

Confusion Matrix:
[[  82   62]
 [  89 1767]]
```




```
--- Training SVM model with C=10 ---
Training SVM model for C=10…
SVM model training with C=10 complete.

--- SVM Model Evaluation (Pass/Fail) with C=10 ---

Classification Report:
              precision    recall  f1-score   support

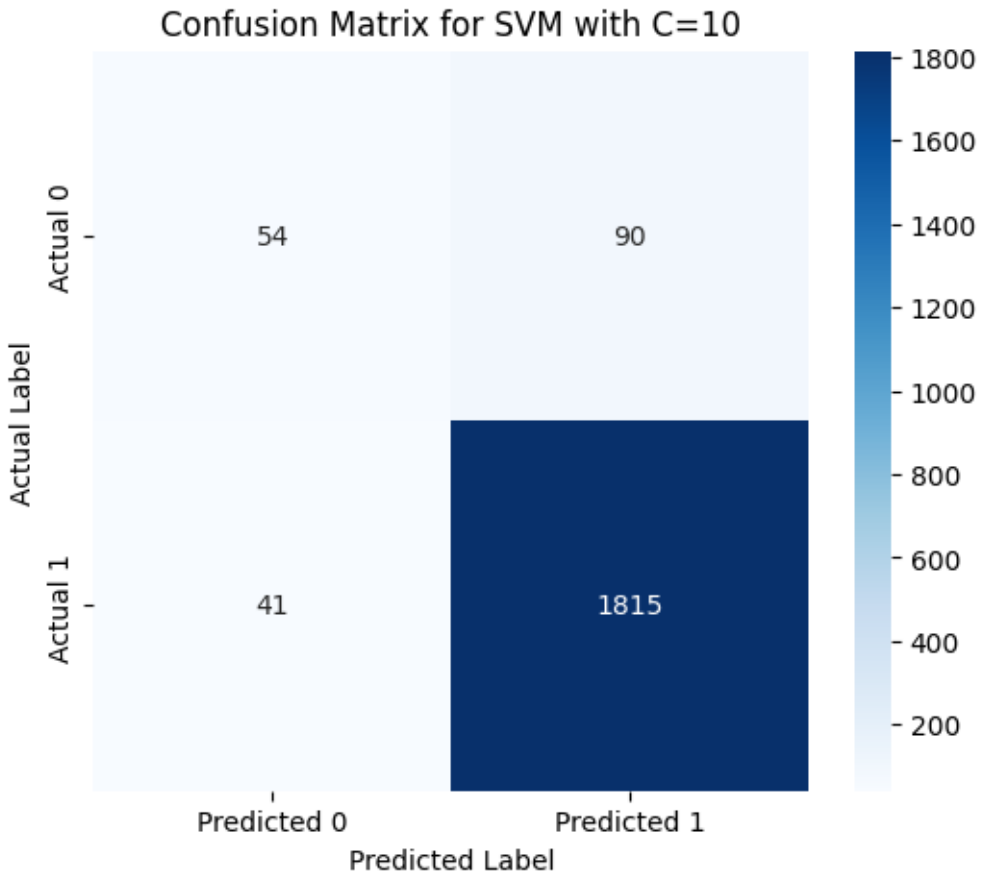
          -3       0.57      0.38      0.45       144
           0       0.95      0.98      0.97      1856

    accuracy                           0.93      2000
   macro avg       0.76      0.68      0.71      2000
weighted avg       0.93      0.93      0.93      2000


Accuracy Score:
```

```
0.9345

Confusion Matrix:
[[  54   90]
 [  41 1815]]
```




```
--- Training SVM model with C=100 ---
Training SVM model for C=100…
SVM model training with C=100 complete.

--- SVM Model Evaluation (Pass/Fail) with C=100 ---

Classification Report:
              precision    recall  f1-score   support

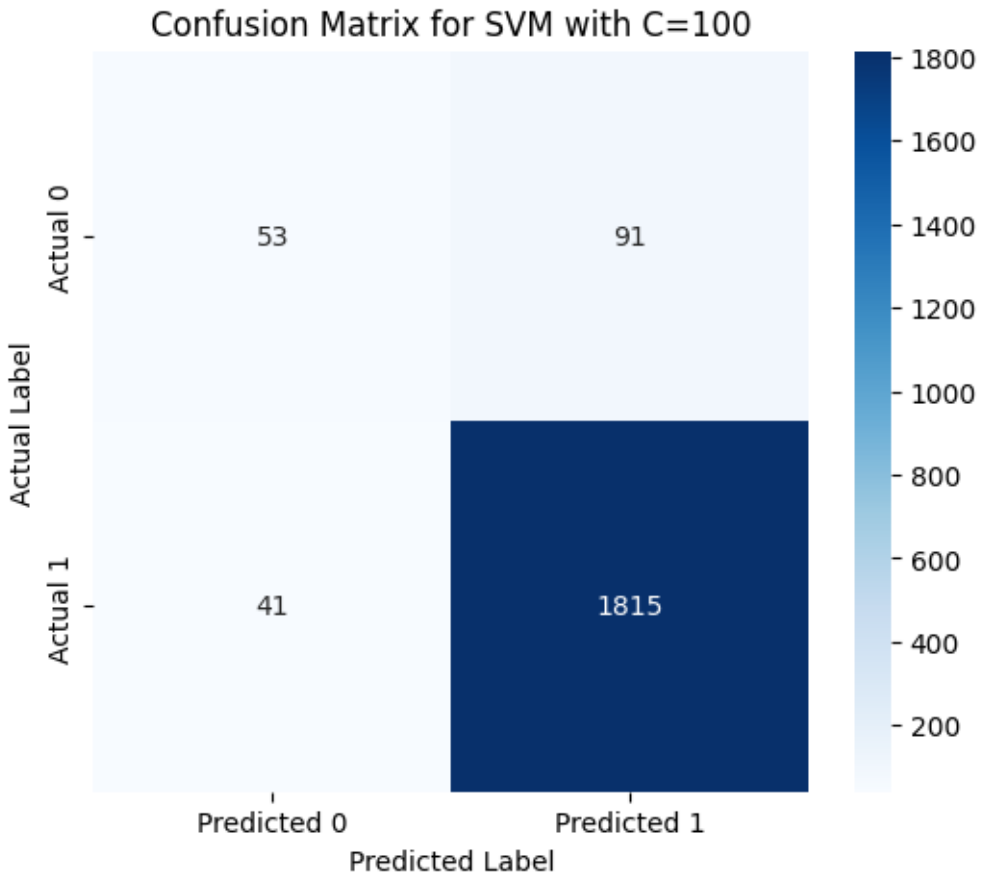
          -3       0.56      0.37      0.45       144
           0       0.95      0.98      0.96      1856

    accuracy                           0.93      2000
```

```
   macro avg       0.76      0.67      0.71      2000
weighted avg       0.92      0.93      0.93      2000


Accuracy Score:
0.934

Confusion Matrix:
[[  53   91]
 [  41 1815]]
```



[ ]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score,␣
 ↪confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Initialize the Random Forest Classifier with max_features=6 and random_state␣
 ↪for reproducibility
```

```
rf_model = RandomForestClassifier(max_features=6, random_state=42)

x_train_binary_rf = X_train_binary
x_test_binary_rf = X_test_binary
y_train_binary_rf = y_train_binary.astype(int)
y_test_binary_rf = y_test_binary.astype(int)

print("--- Training Random Forest Classifier ---")
print("Training Random Forest model...")

# Train the model using the specified training data
rf_model.fit(x_train_binary_rf, y_train_binary_rf)

print("Random Forest model training complete.")

# Make predictions on the test set
y_pred_rf = rf_model.predict(x_test_binary_rf)

# Evaluate the model
print("\n--- Random Forest Classifier Evaluation (Pass/Fail) ---")
print("\nClassification Report:")
print(classification_report(y_test_binary_rf, y_pred_rf))

print("\nAccuracy Score:")
print(accuracy_score(y_test_binary_rf, y_pred_rf))

print("\nConfusion Matrix:")
cm_rf = confusion_matrix(y_test_binary_rf, y_pred_rf)
print(cm_rf)

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix for Random Forest Classifier')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
--- Training Random Forest Classifier ---
Training Random Forest model…
Random Forest model training complete.

--- Random Forest Classifier Evaluation (Pass/Fail) ---

Classification Report:
```

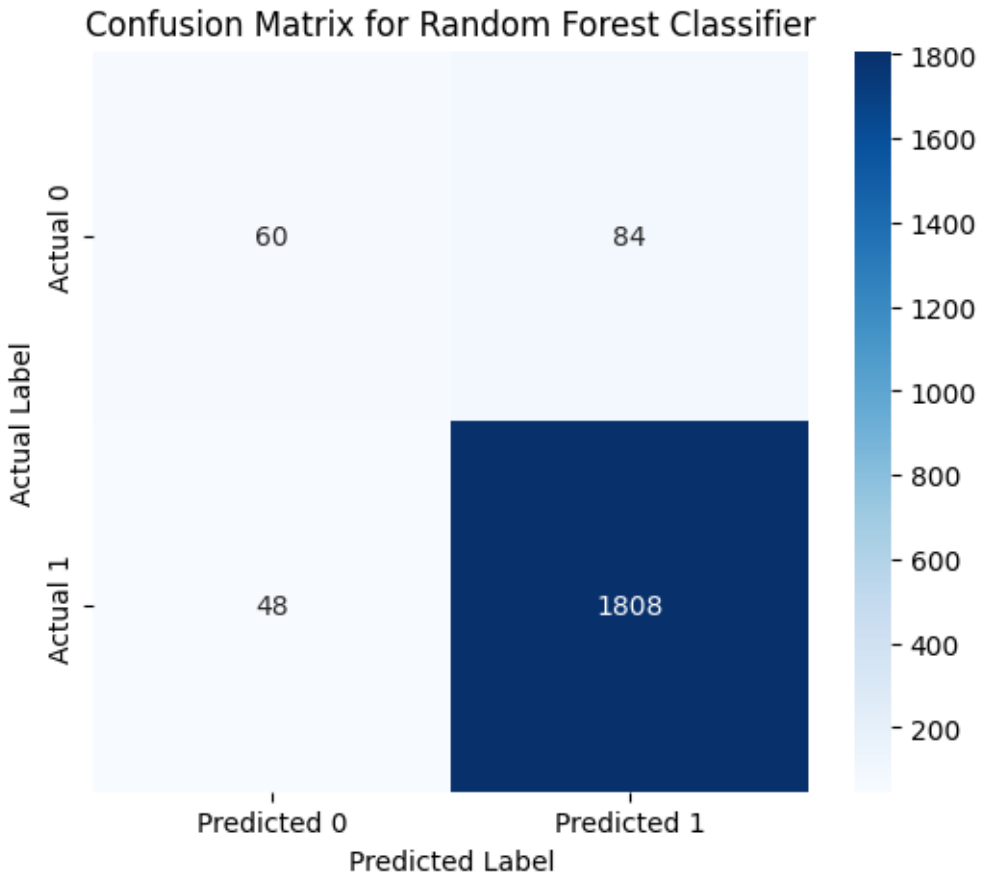
```
              precision    recall  f1-score   support

          -3       0.56      0.42      0.48       144
           0       0.96      0.97      0.96      1856

    accuracy                           0.93      2000
   macro avg       0.76      0.70      0.72      2000
weighted avg       0.93      0.93      0.93      2000


Accuracy Score:
0.934

Confusion Matrix:
[[  60   84]
 [  48 1808]]
```



4.2 Naive Bayes Classifier

4.2.1 Subtask:

Create copies of the training and testing data for the Naive Bayes model, similar to the Random Forest example (`x_train_binary_nb`, `x_test_binary_nb`, `y_train_binary_nb`, `y_test_binary_nb`). Initialize and train a Gaussian Naive Bayes classifier on the training data. Then, make predictions on the test data.

[ ]:
```
from sklearn.naive_bayes import GaussianNB

# Create copies of the training and testing data for the Naive Bayes model
x_train_binary_nb = X_train_binary.copy()
x_test_binary_nb = X_test_binary.copy()
y_train_binary_nb = y_train_binary.astype(int).copy()
y_test_binary_nb = y_test_binary.astype(int).copy()

print("--- Data copies created for Naive Bayes. ---")

# Initialize a Gaussian Naive Bayes classifier
nb_model = GaussianNB()

print("--- Training Gaussian Naive Bayes Classifier ---")
print("Training Naive Bayes model...")

# Train the model using the training data
nb_model.fit(x_train_binary_nb, y_train_binary_nb)

print("Naive Bayes model training complete.")

# Make predictions on the test data
y_pred_nb = nb_model.predict(x_test_binary_nb)

print("Predictions made on the test data.")
```

```
--- Data copies created for Naive Bayes. ---
--- Training Gaussian Naive Bayes Classifier ---
Training Naive Bayes model…
Naive Bayes model training complete.
Predictions made on the test data.
```

[ ]:
```
from sklearn.metrics import classification_report, accuracy_score,␣
 ↪confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

print("\n--- Gaussian Naive Bayes Classifier Evaluation (Pass/Fail) ---")
print("\nClassification Report:")
print(classification_report(y_test_binary_nb, y_pred_nb))
```

```python
print("\nAccuracy Score:")
print(accuracy_score(y_test_binary_nb, y_pred_nb))

print("\nConfusion Matrix:")
cm_nb = confusion_matrix(y_test_binary_nb, y_pred_nb)
print(cm_nb)

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix for Gaussian Naive Bayes Classifier')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
--- Gaussian Naive Bayes Classifier Evaluation (Pass/Fail) ---

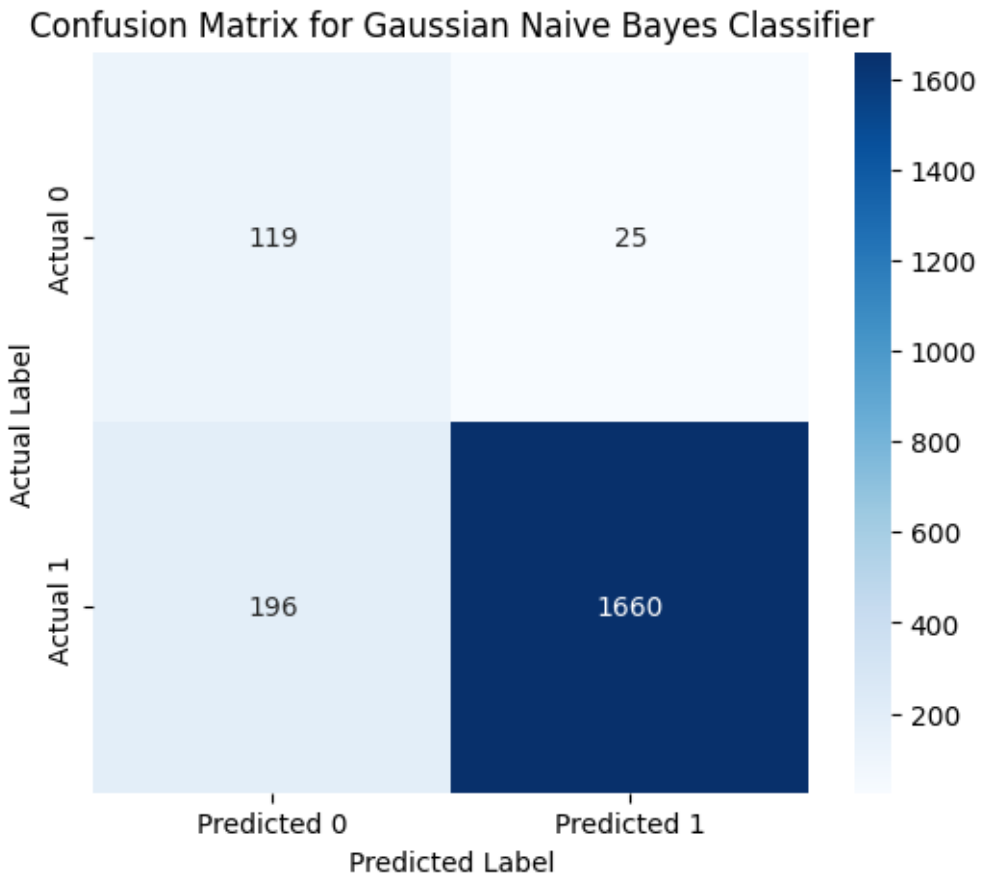
Classification Report:
              precision    recall  f1-score   support

          -3       0.38      0.83      0.52       144
           0       0.99      0.89      0.94      1856

    accuracy                           0.89      2000
   macro avg       0.68      0.86      0.73      2000
weighted avg       0.94      0.89      0.91      2000


Accuracy Score:
0.8895

Confusion Matrix:
[[ 119   25]
 [ 196 1660]]
```



##Desicion Tree

[ ]:
```
from sklearn.tree import DecisionTreeClassifier

# Create copies of the training and testing data for the Decision Tree model
# X_train_binary is already SMOTE-transformed from previous steps.
x_train_binary_dt = X_train_binary.copy()
x_test_binary_dt = X_test_binary.copy()
y_train_binary_dt = y_train_binary.astype(int).copy()
y_test_binary_dt = y_test_binary.astype(int).copy()

print("--- Data copies created for Decision Tree. ---")

# Initialize a Decision Tree Classifier
dt_model = DecisionTreeClassifier(random_state=42)

print("--- Training Decision Tree Classifier ---")
print("Training Decision Tree model...")
```

```
# Train the model using the training data
dt_model.fit(x_train_binary_dt, y_train_binary_dt)

print("Decision Tree model training complete.")

# Make predictions on the test data
y_pred_dt = dt_model.predict(x_test_binary_dt)

print("Predictions made on the test data.")
```

```
--- Data copies created for Decision Tree. ---
--- Training Decision Tree Classifier ---
Training Decision Tree model…
Decision Tree model training complete.
Predictions made on the test data.
```

[ ]:
```
from sklearn.metrics import classification_report, accuracy_score,␣
 ↪confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

print("\n--- Decision Tree Classifier Evaluation (Pass/Fail) ---")
print("\nClassification Report:")
print(classification_report(y_test_binary_dt, y_pred_dt, zero_division=0))

print("\nAccuracy Score:")
print(accuracy_score(y_test_binary_dt, y_pred_dt))

print("\nConfusion Matrix:")
cm_dt = confusion_matrix(y_test_binary_dt, y_pred_dt)
print(cm_dt)

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted -3'],
            yticklabels=['Actual 0', 'Actual -3'])
plt.title('Confusion Matrix for Decision Tree Classifier')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
--- Decision Tree Classifier Evaluation (Pass/Fail) ---

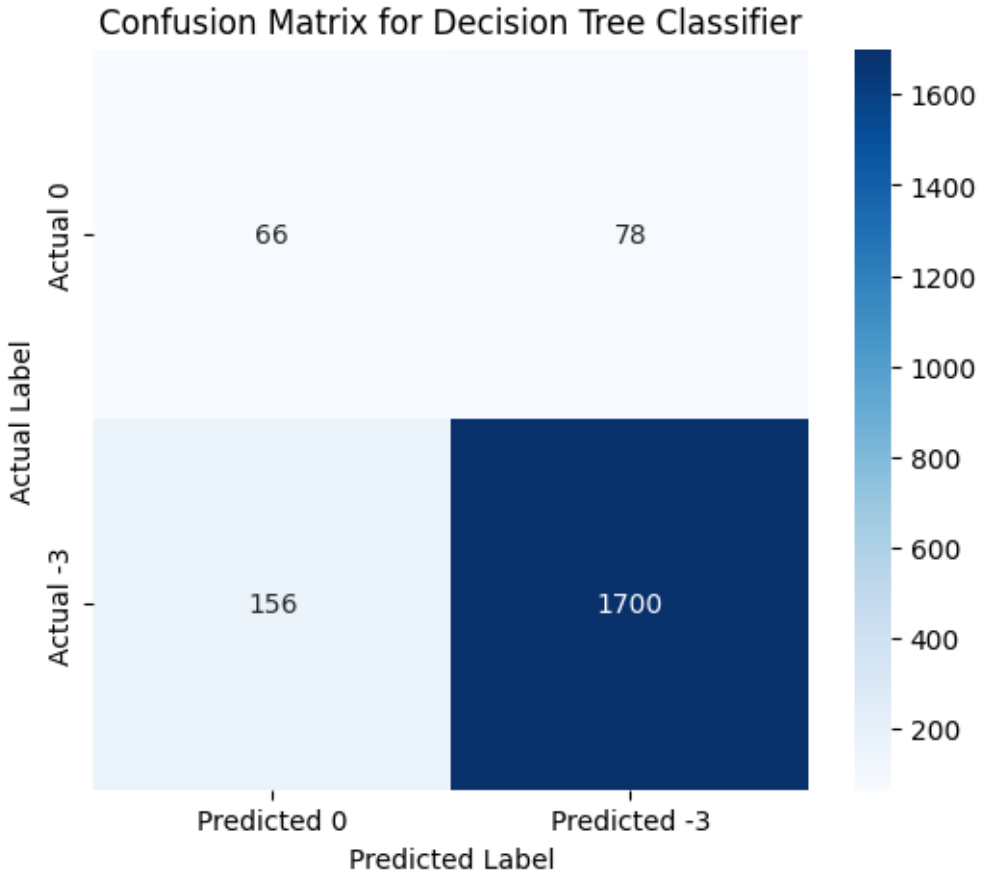
Classification Report:
              precision    recall  f1-score   support
```

```
          -3       0.30      0.46      0.36       144
           0       0.96      0.92      0.94      1856

    accuracy                           0.88      2000
   macro avg       0.63      0.69      0.65      2000
weighted avg       0.91      0.88      0.89      2000


Accuracy Score:
0.883

Confusion Matrix:
[[  66   78]
 [ 156 1700]]
```



##Logistic Regression

[ ]:
```
from sklearn.linear_model import LogisticRegression

# Create copies of the training and testing data for the Logistic Regression␣
 ↪model
# X_train_binary is already SMOTE-transformed from previous steps.
x_train_binary_lr = X_train_binary.copy()
x_test_binary_lr = X_test_binary.copy()
y_train_binary_lr = y_train_binary.astype(int).copy()
y_test_binary_lr = y_test_binary.astype(int).copy()

print("--- Data copies created for Logistic Regression. ---")

# Initialize a Logistic Regression classifier
lr_model = LogisticRegression(random_state=42, solver='liblinear') #␣
 ↪'liblinear' is a good default for small datasets and L1/L2 regularization

print("--- Training Logistic Regression Classifier ---")
print("Training Logistic Regression model...")

# Train the model using the training data
lr_model.fit(x_train_binary_lr, y_train_binary_lr)

print("Logistic Regression model training complete.")

# Make predictions on the test data
y_pred_lr = lr_model.predict(x_test_binary_lr)

print("Predictions made on the test data.")
```

```
--- Data copies created for Logistic Regression. ---
--- Training Logistic Regression Classifier ---
Training Logistic Regression model…
Logistic Regression model training complete.
Predictions made on the test data.
```

[ ]:
```
from sklearn.metrics import classification_report, accuracy_score,␣
 ↪confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

print("\n--- Logistic Regression Classifier Evaluation (Pass/Fail) ---")
print("\nClassification Report:")
print(classification_report(y_test_binary_lr, y_pred_lr, zero_division=0))

print("\nAccuracy Score:")
print(accuracy_score(y_test_binary_lr, y_pred_lr))
```

```python
print("\nConfusion Matrix:")
cm_lr = confusion_matrix(y_test_binary_lr, y_pred_lr)
print(cm_lr)

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted -3'],
            yticklabels=['Actual 0', 'Actual -3'])
plt.title('Confusion Matrix for Logistic Regression Classifier')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
--- Logistic Regression Classifier Evaluation (Pass/Fail) ---

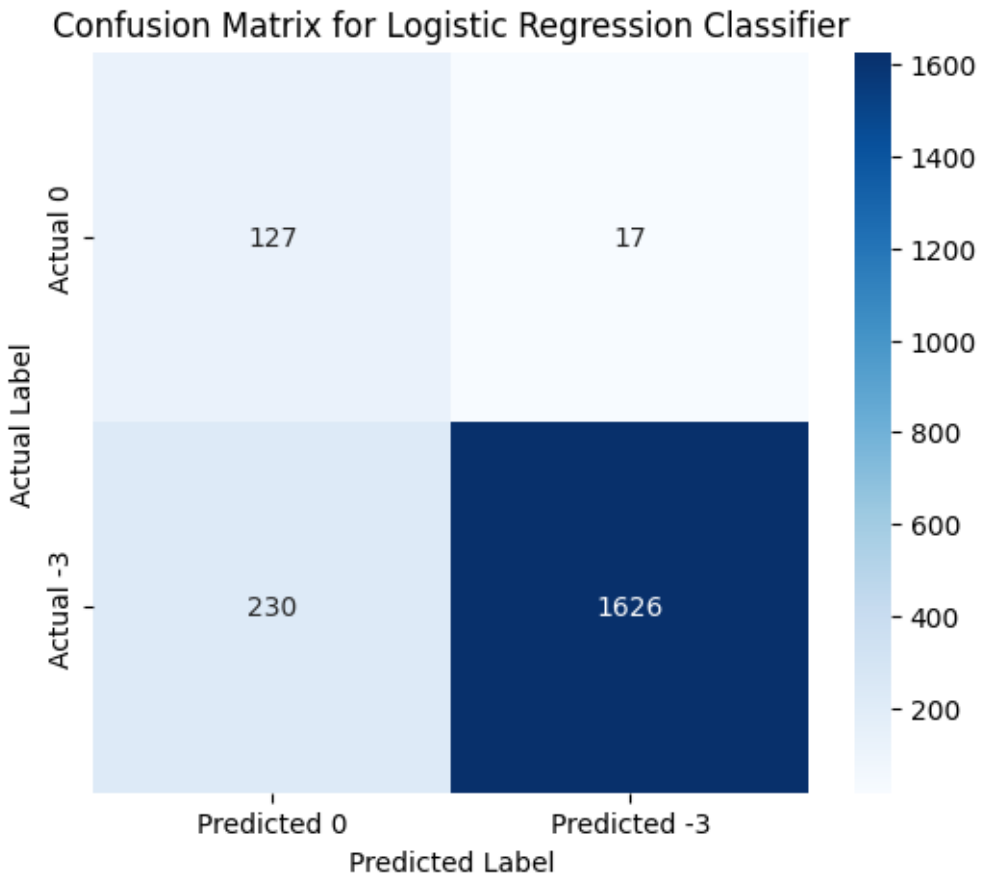
Classification Report:
              precision    recall  f1-score   support

          -3       0.36      0.88      0.51       144
           0       0.99      0.88      0.93      1856

    accuracy                           0.88      2000
   macro avg       0.67      0.88      0.72      2000
weighted avg       0.94      0.88      0.90      2000


Accuracy Score:
0.8765

Confusion Matrix:
[[ 127   17]
 [ 230 1626]]
```



4.3 Stratified K-Fold Cross-Validation (5 folds)

4.3.1 Stratified K-Fold Cross-Validation for SVM

Subtask: For the SVM model, set up Stratified K-Fold Cross-Validation (e.g., with 5 folds) on the SMOTE-transformed training data (`X_train_binary`, `y_train_binary`). Iterate through the specified C values, train each SVM model within each fold, and collect evaluation metrics (accuracy, precision, recall, F1-score for both classes, and ROC AUC) across all folds. Report the average metrics and their standard deviations.

[ ]:
```
from sklearn.model_selection import StratifiedKFold
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, roc_auc_score
import numpy as np

# Define a list of C values for the SVM model
C_values = [0.1, 1, 10, 100]

# Initialize StratifiedKFold with 5 splits
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Create dictionaries to store the metrics for each C value
all_metrics = {
    'accuracy': {},
    'precision_class_0': {},
    'precision_class_1': {},
    'recall_class_0': {},
    'recall_class_1': {},
    'f1_class_0': {},
    'f1_class_1': {},
    'roc_auc': {}
}

print("--- Starting Stratified K-Fold Cross-Validation for SVM Models ---")

# Iterate through each C value
for C_val in C_values:
    print(f"\nProcessing C={C_val}...")

    # Initialize empty lists to store metrics for current C value across folds
    fold_accuracies = []
    fold_precisions_0 = []
    fold_precisions_1 = []
    fold_recalls_0 = []
    fold_recalls_1 = []
    fold_f1s_0 = []
    fold_f1s_1 = []
    fold_roc_aucs = []

    # Iterate through the folds
    for fold_idx, (train_index, val_index) in enumerate(skf.
 ↪split(x_train_binary_svm, y_train_binary_svm)):
        # Split data for the current fold
        X_train_fold, X_val_fold = x_train_binary_svm.iloc[train_index],␣
 ↪x_train_binary_svm.iloc[val_index]
        y_train_fold, y_val_fold = y_train_binary_svm.iloc[train_index],␣
 ↪y_train_binary_svm.iloc[val_index]

        # Initialize an SVC model with the current C_val and probability=True␣
 ↪for ROC AUC
        svm_model_cv = SVC(C=C_val, random_state=42, probability=True)

        # Train the model
        svm_model_cv.fit(X_train_fold, y_train_fold)

        # Make predictions and probability predictions
```

```
        y_pred_fold = svm_model_cv.predict(X_val_fold)
        y_pred_proba_fold = svm_model_cv.predict_proba(X_val_fold)[:, 1] #␣
↪Probability of the positive class

        # Calculate metrics for the current fold
        fold_accuracies.append(accuracy_score(y_val_fold, y_pred_fold))

        # precision_score, recall_score, f1_score with average=None returns␣
↪scores for each class
        precision_per_class = precision_score(y_val_fold, y_pred_fold,␣
↪average=None, zero_division=0)
        fold_precisions_0.append(precision_per_class[0])
        fold_precisions_1.append(precision_per_class[1])

        recall_per_class = recall_score(y_val_fold, y_pred_fold, average=None,␣
↪zero_division=0)
        fold_recalls_0.append(recall_per_class[0])
        fold_recalls_1.append(recall_per_class[1])

        f1_per_class = f1_score(y_val_fold, y_pred_fold, average=None,␣
↪zero_division=0)
        fold_f1s_0.append(f1_per_class[0])
        fold_f1s_1.append(f1_per_class[1])

        # ROC AUC requires binary labels (0 and 1). Assuming -3 maps to 0 and 0␣
↪maps to 1 for ROC AUC calculation
        # Need to ensure the target variable for roc_auc_score is binary (0 or␣
↪1).
        # The `y_train_binary_svm` has values -3 and 0. Let's map them to 0 and␣
↪1 for AUC calculation.
        y_val_fold_binary = y_val_fold.replace({-3: 0, 0: 1})

        fold_roc_aucs.append(roc_auc_score(y_val_fold_binary,␣
↪y_pred_proba_fold))

    # Calculate average and standard deviation for each metric for the current␣
↪C value
    all_metrics['accuracy'][C_val] = {'mean': np.mean(fold_accuracies), 'std':␣
↪np.std(fold_accuracies)}
    all_metrics['precision_class_0'][C_val] = {'mean': np.
↪mean(fold_precisions_0), 'std': np.std(fold_precisions_0)}
    all_metrics['precision_class_1'][C_val] = {'mean': np.
↪mean(fold_precisions_1), 'std': np.std(fold_precisions_1)}
    all_metrics['recall_class_0'][C_val] = {'mean': np.mean(fold_recalls_0),␣
↪'std': np.std(fold_recalls_0)}
```

```
    all_metrics['recall_class_1'][C_val] = {'mean': np.mean(fold_recalls_1),␣
↪'std': np.std(fold_recalls_1)}
    all_metrics['f1_class_0'][C_val] = {'mean': np.mean(fold_f1s_0), 'std': np.
↪std(fold_f1s_0)}
    all_metrics['f1_class_1'][C_val] = {'mean': np.mean(fold_f1s_1), 'std': np.
↪std(fold_f1s_1)}
    all_metrics['roc_auc'][C_val] = {'mean': np.mean(fold_roc_aucs), 'std': np.
↪std(fold_roc_aucs)}

    # Print the results for the current C value
    print(f"C={C_val} - Average Metrics across 5 folds:")
    print(f"  Accuracy: {all_metrics['accuracy'][C_val]['mean']:.4f} \u00B1␣
↪{all_metrics['accuracy'][C_val]['std']:.4f}")
    print(f"  Precision (Class 0):␣
↪{all_metrics['precision_class_0'][C_val]['mean']:.4f} \u00B1␣
↪{all_metrics['precision_class_0'][C_val]['std']:.4f}")
    print(f"  Precision (Class 1):␣
↪{all_metrics['precision_class_1'][C_val]['mean']:.4f} \u00B1␣
↪{all_metrics['precision_class_1'][C_val]['std']:.4f}")
    print(f"  Recall (Class 0): {all_metrics['recall_class_0'][C_val]['mean']:.
↪4f} \u00B1 {all_metrics['recall_class_0'][C_val]['std']:.4f}")
    print(f"  Recall (Class 1): {all_metrics['recall_class_1'][C_val]['mean']:.
↪4f} \u00B1 {all_metrics['recall_class_1'][C_val]['std']:.4f}")
    print(f"  F1-Score (Class 0): {all_metrics['f1_class_0'][C_val]['mean']:.
↪4f} \u00B1 {all_metrics['f1_class_0'][C_val]['std']:.4f}")
    print(f"  F1-Score (Class 1): {all_metrics['f1_class_1'][C_val]['mean']:.
↪4f} \u00B1 {all_metrics['f1_class_1'][C_val]['std']:.4f}")
    print(f"  ROC AUC: {all_metrics['roc_auc'][C_val]['mean']:.4f} \u00B1␣
↪{all_metrics['roc_auc'][C_val]['std']:.4f}")

print("\n--- Stratified K-Fold Cross-Validation complete. ---")
```

```
--- Starting Stratified K-Fold Cross-Validation for SVM Models ---

Processing C=0.1…
C=0.1 - Average Metrics across 5 folds:
  Accuracy: 0.9452 ± 0.0008
  Precision (Class 0): 0.9253 ± 0.0020
  Precision (Class 1): 0.9670 ± 0.0040
  Recall (Class 0): 0.9685 ± 0.0040
  Recall (Class 1): 0.9218 ± 0.0026
  F1-Score (Class 0): 0.9464 ± 0.0009
  F1-Score (Class 1): 0.9439 ± 0.0006
  ROC AUC: 0.9870 ± 0.0004

Processing C=1…
```

```
C=1 - Average Metrics across 5 folds:
  Accuracy: 0.9758 ± 0.0016
  Precision (Class 0): 0.9558 ± 0.0027
  Precision (Class 1): 0.9976 ± 0.0004
  Recall (Class 0): 0.9977 ± 0.0004
  Recall (Class 1): 0.9538 ± 0.0029
  F1-Score (Class 0): 0.9763 ± 0.0015
  F1-Score (Class 1): 0.9752 ± 0.0016
  ROC AUC: 0.9961 ± 0.0008

Processing C=10…
C=10 - Average Metrics across 5 folds:
  Accuracy: 0.9853 ± 0.0013
  Precision (Class 0): 0.9717 ± 0.0025
  Precision (Class 1): 0.9999 ± 0.0002
  Recall (Class 0): 0.9999 ± 0.0002
  Recall (Class 1): 0.9708 ± 0.0027
  F1-Score (Class 0): 0.9856 ± 0.0013
  F1-Score (Class 1): 0.9851 ± 0.0013
  ROC AUC: 0.9991 ± 0.0003

Processing C=100…
C=100 - Average Metrics across 5 folds:
  Accuracy: 0.9853 ± 0.0013
  Precision (Class 0): 0.9716 ± 0.0026
  Precision (Class 1): 0.9999 ± 0.0002
  Recall (Class 0): 0.9999 ± 0.0002
  Recall (Class 1): 0.9708 ± 0.0027
  F1-Score (Class 0): 0.9855 ± 0.0013
  F1-Score (Class 1): 0.9851 ± 0.0014
  ROC AUC: 0.9991 ± 0.0003

--- Stratified K-Fold Cross-Validation complete. ---
```

4.4 Implement Stratified K-Fold Cross-Validation for Random Forest

4.4.1 Subtask:

Set up Stratified K-Fold Cross-Validation (e.g., with 5 folds) for the Random Forest Classifier (with `max_features=6`) on the SMOTE-transformed training data (`X_train_binary`, `y_train_binary`). Train the model within each fold and collect evaluation metrics (accuracy, precision, recall, F1-score for both classes, and ROC AUC) across all folds. Report the average metrics and their standard deviations.

[ ]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, roc_auc_score
```

```
import numpy as np

# Initialize StratifiedKFold with 5 splits (already done for SVM, reusing the␣
 ↪object if not redefined)
skf_rf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize empty lists to store metrics for Random Forest across folds
rf_fold_accuracies = []
rf_fold_precisions_0 = []
rf_fold_precisions_1 = []
rf_fold_recalls_0 = []
rf_fold_recalls_1 = []
rf_fold_f1s_0 = []
rf_fold_f1s_1 = []
rf_fold_roc_aucs = []

print("--- Starting Stratified K-Fold Cross-Validation for Random Forest␣
 ↪Classifier ---")

# Iterate through the folds
for fold_idx, (train_index, val_index) in enumerate(skf_rf.
 ↪split(x_train_binary_rf, y_train_binary_rf)):
    # Split data for the current fold
    X_train_fold_rf, X_val_fold_rf = x_train_binary_rf.iloc[train_index],␣
 ↪x_train_binary_rf.iloc[val_index]
    y_train_fold_rf, y_val_fold_rf = y_train_binary_rf.iloc[train_index],␣
 ↪y_train_binary_rf.iloc[val_index]

    # Initialize a RandomForestClassifier with max_features=6 and␣
 ↪random_state=42
    rf_model_cv = RandomForestClassifier(max_features=6, random_state=42)

    # Train the model
    rf_model_cv.fit(X_train_fold_rf, y_train_fold_rf)

    # Make predictions and probability predictions
    y_pred_fold_rf = rf_model_cv.predict(X_val_fold_rf)
    y_pred_proba_fold_rf = rf_model_cv.predict_proba(X_val_fold_rf)[:, 1] #␣
 ↪Probability of the positive class

    # Calculate metrics for the current fold
    rf_fold_accuracies.append(accuracy_score(y_val_fold_rf, y_pred_fold_rf))

    # precision_score, recall_score, f1_score with average=None returns scores␣
 ↪for each class
```

```
    precision_per_class_rf = precision_score(y_val_fold_rf, y_pred_fold_rf,␣
 ↪average=None, zero_division=0)
    rf_fold_precisions_0.append(precision_per_class_rf[0])
    rf_fold_precisions_1.append(precision_per_class_rf[1])

    recall_per_class_rf = recall_score(y_val_fold_rf, y_pred_fold_rf,␣
 ↪average=None, zero_division=0)
    rf_fold_recalls_0.append(recall_per_class_rf[0])
    rf_fold_recalls_1.append(recall_per_class_rf[1])

    f1_per_class_rf = f1_score(y_val_fold_rf, y_pred_fold_rf, average=None,␣
 ↪zero_division=0)
    rf_fold_f1s_0.append(f1_per_class_rf[0])
    rf_fold_f1s_1.append(f1_per_class_rf[1])

    # ROC AUC requires binary labels (0 and 1). Assuming -3 maps to 0 and 0␣
 ↪maps to 1 for ROC AUC calculation
    # y_val_fold_rf has values -3 and 0. Let's map them to 0 and 1 for AUC␣
 ↪calculation.
    y_val_fold_binary_rf = y_val_fold_rf.replace({-3: 0, 0: 1})

    rf_fold_roc_aucs.append(roc_auc_score(y_val_fold_binary_rf,␣
 ↪y_pred_proba_fold_rf))

print("--- Cross-validation for Random Forest complete. ---")

# Calculate average and standard deviation for each metric
print("\nRandom Forest - Average Metrics across 5 folds:")
print(f"  Accuracy: {np.mean(rf_fold_accuracies):.4f} \u00B1 {np.
 ↪std(rf_fold_accuracies):.4f}")
print(f"  Precision (Class 0): {np.mean(rf_fold_precisions_0):.4f} \u00B1 {np.
 ↪std(rf_fold_precisions_0):.4f}")
print(f"  Precision (Class 1): {np.mean(rf_fold_precisions_1):.4f} \u00B1 {np.
 ↪std(rf_fold_precisions_1):.4f}")
print(f"  Recall (Class 0): {np.mean(rf_fold_recalls_0):.4f} \u00B1 {np.
 ↪std(rf_fold_recalls_0):.4f}")
print(f"  Recall (Class 1): {np.mean(rf_fold_recalls_1):.4f} \u00B1 {np.
 ↪std(rf_fold_recalls_1):.4f}")
print(f"  F1-Score (Class 0): {np.mean(rf_fold_f1s_0):.4f} \u00B1 {np.
 ↪std(rf_fold_f1s_0):.4f}")
print(f"  F1-Score (Class 1): {np.mean(rf_fold_f1s_1):.4f} \u00B1 {np.
 ↪std(rf_fold_f1s_1):.4f}")
print(f"  ROC AUC: {np.mean(rf_fold_roc_aucs):.4f} \u00B1 {np.
 ↪std(rf_fold_roc_aucs):.4f}")
```

```
--- Starting Stratified K-Fold Cross-Validation for Random Forest Classifier ---
--- Cross-validation for Random Forest complete. ---

Random Forest - Average Metrics across 5 folds:
  Accuracy: 0.9754 ± 0.0014
  Precision (Class 0): 0.9703 ± 0.0018
  Precision (Class 1): 0.9807 ± 0.0033
  Recall (Class 0): 0.9809 ± 0.0033
  Recall (Class 1): 0.9700 ± 0.0019
  F1-Score (Class 0): 0.9756 ± 0.0015
  F1-Score (Class 1): 0.9753 ± 0.0014
  ROC AUC: 0.9979 ± 0.0002
```

4.5 Implement Stratified K-Fold Cross-Validation for Naive Bayes

4.5.1 Subtask:

Apply Stratified K-Fold Cross-Validation (e.g., with 5 folds) for the Gaussian Naive Bayes model on the SMOTE-transformed training data (`X_train_binary`, `y_train_binary`). Train the model within each fold and collect evaluation metrics (accuracy, precision, recall, F1-score for both classes, and ROC AUC) across all folds. Report the average metrics and their standard deviations.

[ ]:
```python
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, roc_auc_score
import numpy as np

# Initialize StratifiedKFold with 5 splits
skf_nb = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize empty lists to store metrics for Naive Bayes across folds
nb_fold_accuracies = []
nb_fold_precisions_0 = []
nb_fold_precisions_1 = []
nb_fold_recalls_0 = []
nb_fold_recalls_1 = []
nb_fold_f1s_0 = []
nb_fold_f1s_1 = []
nb_fold_roc_aucs = []

print("--- Starting Stratified K-Fold Cross-Validation for Gaussian Naive Bayes␣
 ↪Classifier ---")

# Iterate through the folds
for fold_idx, (train_index, val_index) in enumerate(skf_nb.
 ↪split(x_train_binary_nb, y_train_binary_nb)):
    # Split data for the current fold
```

```
    X_train_fold_nb, X_val_fold_nb = x_train_binary_nb.iloc[train_index],␣
↪x_train_binary_nb.iloc[val_index]
    y_train_fold_nb, y_val_fold_nb = y_train_binary_nb.iloc[train_index],␣
↪y_train_binary_nb.iloc[val_index]

    # Initialize a Gaussian Naive Bayes classifier
    nb_model_cv = GaussianNB()

    # Train the model
    nb_model_cv.fit(X_train_fold_nb, y_train_fold_nb)

    # Make predictions and probability predictions
    y_pred_fold_nb = nb_model_cv.predict(X_val_fold_nb)
    # GaussianNB.predict_proba outputs probabilities for each class in sorted␣
↪order
    # The target labels are -3 and 0. We need to map them to 0 and 1 for␣
↪consistent ROC AUC calculation.
    # Assuming 0 is the positive class (index 1) after sorting unique labels or␣
↪based on predict_proba output order.
    # First, let's get the classes the model learned.
    classes = nb_model_cv.classes_
    positive_class_index = np.where(classes == 0)[0][0] # Assuming 0 is the␣
↪'Pass' class, and the one we want ROC AUC for.
    y_pred_proba_fold_nb = nb_model_cv.predict_proba(X_val_fold_nb)[:,␣
↪positive_class_index]

    # Calculate metrics for the current fold
    nb_fold_accuracies.append(accuracy_score(y_val_fold_nb, y_pred_fold_nb))

    # precision_score, recall_score, f1_score with average=None returns scores␣
↪for each class
    precision_per_class_nb = precision_score(y_val_fold_nb, y_pred_fold_nb,␣
↪average=None, zero_division=0)
    # Ensure we append for the correct classes. Assuming -3 is class 0␣
↪(negative) and 0 is class 1 (positive).
    # The order in precision_per_class_nb corresponds to the sorted unique␣
↪values of y_val_fold_nb, which are -3 and 0.
    nb_fold_precisions_0.append(precision_per_class_nb[0]) # For class -3
    nb_fold_precisions_1.append(precision_per_class_nb[1]) # For class 0

    recall_per_class_nb = recall_score(y_val_fold_nb, y_pred_fold_nb,␣
↪average=None, zero_division=0)
    nb_fold_recalls_0.append(recall_per_class_nb[0]) # For class -3
    nb_fold_recalls_1.append(recall_per_class_nb[1]) # For class 0
```

```
    f1_per_class_nb = f1_score(y_val_fold_nb, y_pred_fold_nb, average=None,␣
 ↪zero_division=0)
    nb_fold_f1s_0.append(f1_per_class_nb[0]) # For class -3
    nb_fold_f1s_1.append(f1_per_class_nb[1]) # For class 0

    # ROC AUC requires binary labels (0 and 1). Map -3 to 0 and 0 to 1.
    y_val_fold_binary_nb = y_val_fold_nb.replace({-3: 0, 0: 1})

    nb_fold_roc_aucs.append(roc_auc_score(y_val_fold_binary_nb,␣
 ↪y_pred_proba_fold_nb))

print("--- Cross-validation for Gaussian Naive Bayes complete. ---")

# Calculate average and standard deviation for each metric
print("\nGaussian Naive Bayes - Average Metrics across 5 folds:")
print(f"  Accuracy: {np.mean(nb_fold_accuracies):.4f} \u00B1 {np.
 ↪std(nb_fold_accuracies):.4f}")
print(f"  Precision (Class -3): {np.mean(nb_fold_precisions_0):.4f} \u00B1 {np.
 ↪std(nb_fold_precisions_0):.4f}")
print(f"  Precision (Class 0): {np.mean(nb_fold_precisions_1):.4f} \u00B1 {np.
 ↪std(nb_fold_precisions_1):.4f}")
print(f"  Recall (Class -3): {np.mean(nb_fold_recalls_0):.4f} \u00B1 {np.
 ↪std(nb_fold_recalls_0):.4f}")
print(f"  Recall (Class 0): {np.mean(nb_fold_recalls_1):.4f} \u00B1 {np.
 ↪std(nb_fold_recalls_1):.4f}")
print(f"  F1-Score (Class -3): {np.mean(nb_fold_f1s_0):.4f} \u00B1 {np.
 ↪std(nb_fold_f1s_0):.4f}")
print(f"  F1-Score (Class 0): {np.mean(nb_fold_f1s_1):.4f} \u00B1 {np.
 ↪std(nb_fold_f1s_1):.4f}")
print(f"  ROC AUC: {np.mean(nb_fold_roc_aucs):.4f} \u00B1 {np.
 ↪std(nb_fold_roc_aucs):.4f}")
```

```
--- Starting Stratified K-Fold Cross-Validation for Gaussian Naive Bayes
Classifier ---
--- Cross-validation for Gaussian Naive Bayes complete. ---

Gaussian Naive Bayes - Average Metrics across 5 folds:
  Accuracy: 0.9115 ± 0.0022
  Precision (Class -3): 0.8980 ± 0.0033
  Precision (Class 0): 0.9261 ± 0.0046
  Recall (Class -3): 0.9285 ± 0.0050
  Recall (Class 0): 0.8945 ± 0.0041
  F1-Score (Class -3): 0.9130 ± 0.0022
  F1-Score (Class 0): 0.9100 ± 0.0021
  ROC AUC: 0.9705 ± 0.0014
```

4.6 Implement Stratified K-Fold Cross-Validation for Logistic Regression

4.6.1 Subtask:

Perform Stratified K-Fold Cross-Validation (e.g., with 5 folds) for the Logistic Regression model on the SMOTE-transformed training data (`X_train_binary`, `y_train_binary`). Train the model within each fold and collect evaluation metrics (accuracy, precision, recall, F1-score for both classes, and ROC AUC) across all folds. Report the average metrics and their standard deviations.

[ ]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, roc_auc_score
import numpy as np

# Reuse StratifiedKFold object from previous steps
skf_lr = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize empty lists to store metrics for Logistic Regression across folds
lr_fold_accuracies = []
lr_fold_precisions_neg3 = [] # For class -3
lr_fold_precisions_0 = []    # For class 0
lr_fold_recalls_neg3 = []
lr_fold_recalls_0 = []
lr_fold_f1s_neg3 = []
lr_fold_f1s_0 = []
lr_fold_roc_aucs = []

print("--- Starting Stratified K-Fold Cross-Validation for Logistic Regression␣
 ↪Classifier ---")

# Iterate through the folds
for fold_idx, (train_index, val_index) in enumerate(skf_lr.
 ↪split(x_train_binary_lr, y_train_binary_lr)):
    # Split data for the current fold
    X_train_fold_lr, X_val_fold_lr = x_train_binary_lr.iloc[train_index],␣
 ↪x_train_binary_lr.iloc[val_index]
    y_train_fold_lr, y_val_fold_lr = y_train_binary_lr.iloc[train_index],␣
 ↪y_train_binary_lr.iloc[val_index]

    # Initialize a Logistic Regression classifier
    lr_model_cv = LogisticRegression(random_state=42, solver='liblinear',␣
 ↪penalty='l2')

    # Train the model
    lr_model_cv.fit(X_train_fold_lr, y_train_fold_lr)

    # Make predictions and probability predictions
```

```
    y_pred_fold_lr = lr_model_cv.predict(X_val_fold_lr)

    # Get the index for the positive class (0) in predict_proba output
    classes = lr_model_cv.classes_
    positive_class_index = np.where(classes == 0)[0][0] # Assuming 0 is the␣
 ↪'Pass' class
    y_pred_proba_fold_lr = lr_model_cv.predict_proba(X_val_fold_lr)[:,␣
 ↪positive_class_index]

    # Calculate metrics for the current fold
    lr_fold_accuracies.append(accuracy_score(y_val_fold_lr, y_pred_fold_lr))

    precision_per_class_lr = precision_score(y_val_fold_lr, y_pred_fold_lr,␣
 ↪average=None, zero_division=0)
    lr_fold_precisions_neg3.append(precision_per_class_lr[np.where(classes ==␣
 ↪-3)[0][0]])
    lr_fold_precisions_0.append(precision_per_class_lr[positive_class_index])

    recall_per_class_lr = recall_score(y_val_fold_lr, y_pred_fold_lr,␣
 ↪average=None, zero_division=0)
    lr_fold_recalls_neg3.append(recall_per_class_lr[np.where(classes ==␣
 ↪-3)[0][0]])
    lr_fold_recalls_0.append(recall_per_class_lr[positive_class_index])

    f1_per_class_lr = f1_score(y_val_fold_lr, y_pred_fold_lr, average=None,␣
 ↪zero_division=0)
    lr_fold_f1s_neg3.append(f1_per_class_lr[np.where(classes == -3)[0][0]])
    lr_fold_f1s_0.append(f1_per_class_lr[positive_class_index])

    # ROC AUC requires binary labels (0 and 1). Map -3 to 0 and 0 to 1.
    y_val_fold_binary_lr = y_val_fold_lr.replace({-3: 0, 0: 1})

    lr_fold_roc_aucs.append(roc_auc_score(y_val_fold_binary_lr,␣
 ↪y_pred_proba_fold_lr))

print("--- Cross-validation for Logistic Regression complete. ---")

# Calculate average and standard deviation for each metric
print("\nLogistic Regression - Average Metrics across 5 folds:")
print(f"  Accuracy: {np.mean(lr_fold_accuracies):.4f} \u00B1 {np.
 ↪std(lr_fold_accuracies):.4f}")
print(f"  Precision (Class -3): {np.mean(lr_fold_precisions_neg3):.4f} \u00B1␣
 ↪{np.std(lr_fold_precisions_neg3):.4f}")
print(f"  Precision (Class 0): {np.mean(lr_fold_precisions_0):.4f} \u00B1 {np.
 ↪std(lr_fold_precisions_0):.4f}")
```

```
print(f"  Recall (Class -3): {np.mean(lr_fold_recalls_neg3):.4f} \u00B1 {np.
  ↪std(lr_fold_recalls_neg3):.4f}")
print(f"  Recall (Class 0): {np.mean(lr_fold_recalls_0):.4f} \u00B1 {np.
  ↪std(lr_fold_recalls_0):.4f}")
print(f"  F1-Score (Class -3): {np.mean(lr_fold_f1s_neg3):.4f} \u00B1 {np.
  ↪std(lr_fold_f1s_neg3):.4f}")
print(f"  F1-Score (Class 0): {np.mean(lr_fold_f1s_0):.4f} \u00B1 {np.
  ↪std(lr_fold_f1s_0):.4f}")
print(f"  ROC AUC: {np.mean(lr_fold_roc_aucs):.4f} \u00B1 {np.
  ↪std(lr_fold_roc_aucs):.4f}")
```

```
--- Starting Stratified K-Fold Cross-Validation for Logistic Regression
Classifier ---
--- Cross-validation for Logistic Regression complete. ---

Logistic Regression - Average Metrics across 5 folds:
  Accuracy: 0.9020 ± 0.0023
  Precision (Class -3): 0.8820 ± 0.0028
  Precision (Class 0): 0.9242 ± 0.0034
  Recall (Class -3): 0.9282 ± 0.0036
  Recall (Class 0): 0.8758 ± 0.0034
  F1-Score (Class -3): 0.9045 ± 0.0022
  F1-Score (Class 0): 0.8994 ± 0.0023
  ROC AUC: 0.9603 ± 0.0020
```

4.7 Implement Stratified K-Fold Cross-Validation for Decision Tree

4.7.1 Subtask:

Conduct Stratified K-Fold Cross-Validation (e.g., with 5 folds) for the Decision Tree Classifier on the SMOTE-transformed training data (`X_train_binary`, `y_train_binary`). Train the model within each fold and collect evaluation metrics (accuracy, precision, recall, F1-score for both classes, and ROC AUC) across all folds. Report the average metrics and their standard deviations.

[ ]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
  ↪f1_score, roc_auc_score
import numpy as np

# Initialize StratifiedKFold with 5 splits
skf_dt = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize empty lists to store metrics for Decision Tree across folds
dt_fold_accuracies = []
dt_fold_precisions_neg3 = [] # For class -3
dt_fold_precisions_0 = []    # For class 0
dt_fold_recalls_neg3 = []
```

```
dt_fold_recalls_0 = []
dt_fold_f1s_neg3 = []
dt_fold_f1s_0 = []
dt_fold_roc_aucs = []

print("--- Starting Stratified K-Fold Cross-Validation for Decision Tree␣
 ↪Classifier ---")

# Iterate through the folds
for fold_idx, (train_index, val_index) in enumerate(skf_dt.
 ↪split(x_train_binary_dt, y_train_binary_dt)):
    # Split data for the current fold
    X_train_fold_dt, X_val_fold_dt = x_train_binary_dt.iloc[train_index],␣
 ↪x_train_binary_dt.iloc[val_index]
    y_train_fold_dt, y_val_fold_dt = y_train_binary_dt.iloc[train_index],␣
 ↪y_train_binary_dt.iloc[val_index]

    # Initialize a Decision Tree Classifier
    dt_model_cv = DecisionTreeClassifier(random_state=42)

    # Train the model
    dt_model_cv.fit(X_train_fold_dt, y_train_fold_dt)

    # Make predictions and probability predictions
    y_pred_fold_dt = dt_model_cv.predict(X_val_fold_dt)

    # Get the index for the positive class (0) in predict_proba output
    classes = dt_model_cv.classes_
    positive_class_index = np.where(classes == 0)[0][0] # Assuming 0 is the␣
 ↪'Pass' class
    y_pred_proba_fold_dt = dt_model_cv.predict_proba(X_val_fold_dt)[:,␣
 ↪positive_class_index]

    # Calculate metrics for the current fold
    dt_fold_accuracies.append(accuracy_score(y_val_fold_dt, y_pred_fold_dt))

    precision_per_class_dt = precision_score(y_val_fold_dt, y_pred_fold_dt,␣
 ↪average=None, zero_division=0)
    dt_fold_precisions_neg3.append(precision_per_class_dt[np.where(classes ==␣
 ↪-3)[0][0]])
    dt_fold_precisions_0.append(precision_per_class_dt[positive_class_index])

    recall_per_class_dt = recall_score(y_val_fold_dt, y_pred_fold_dt,␣
 ↪average=None, zero_division=0)
    dt_fold_recalls_neg3.append(recall_per_class_dt[np.where(classes ==␣
 ↪-3)[0][0]])
```

```
    dt_fold_recalls_0.append(recall_per_class_dt[positive_class_index])

    f1_per_class_dt = f1_score(y_val_fold_dt, y_pred_fold_dt, average=None,␣
 ↪zero_division=0)
    dt_fold_f1s_neg3.append(f1_per_class_dt[np.where(classes == -3)[0][0]])
    dt_fold_f1s_0.append(f1_per_class_dt[positive_class_index])

    # ROC AUC requires binary labels (0 and 1). Map -3 to 0 and 0 to 1.
    y_val_fold_binary_dt = y_val_fold_dt.replace({-3: 0, 0: 1})

    dt_fold_roc_aucs.append(roc_auc_score(y_val_fold_binary_dt,␣
 ↪y_pred_proba_fold_dt))

print("--- Cross-validation for Decision Tree complete. ---")

# Calculate average and standard deviation for each metric
print("\nDecision Tree - Average Metrics across 5 folds:")
print(f"  Accuracy: {np.mean(dt_fold_accuracies):.4f} \u00B1 {np.
 ↪std(dt_fold_accuracies):.4f}")
print(f"  Precision (Class -3): {np.mean(dt_fold_precisions_neg3):.4f} \u00B1␣
 ↪{np.std(dt_fold_precisions_neg3):.4f}")
print(f"  Precision (Class 0): {np.mean(dt_fold_precisions_0):.4f} \u00B1 {np.
 ↪std(dt_fold_precisions_0):.4f}")
print(f"  Recall (Class -3): {np.mean(dt_fold_recalls_neg3):.4f} \u00B1 {np.
 ↪std(dt_fold_recalls_neg3):.4f}")
print(f"  Recall (Class 0): {np.mean(dt_fold_recalls_0):.4f} \u00B1 {np.
 ↪std(dt_fold_recalls_0):.4f}")
print(f"  F1-Score (Class -3): {np.mean(dt_fold_f1s_neg3):.4f} \u00B1 {np.
 ↪std(dt_fold_f1s_neg3):.4f}")
print(f"  F1-Score (Class 0): {np.mean(dt_fold_f1s_0):.4f} \u00B1 {np.
 ↪std(dt_fold_f1s_0):.4f}")
print(f"  ROC AUC: {np.mean(dt_fold_roc_aucs):.4f} \u00B1 {np.
 ↪std(dt_fold_roc_aucs):.4f}")
```

```
--- Starting Stratified K-Fold Cross-Validation for Decision Tree Classifier ---
--- Cross-validation for Decision Tree complete. ---

Decision Tree - Average Metrics across 5 folds:
  Accuracy: 0.9237 ± 0.0021
  Precision (Class -3): 0.9109 ± 0.0049
  Precision (Class 0): 0.9375 ± 0.0041
  Recall (Class -3): 0.9394 ± 0.0045
  Recall (Class 0): 0.9081 ± 0.0059
  F1-Score (Class -3): 0.9249 ± 0.0020
  F1-Score (Class 0): 0.9225 ± 0.0023
  ROC AUC: 0.9237 ± 0.0021
```

4.8 Summary:

4.8.1 Data Analysis Key Findings

- **Random Forest Classifier Exhibits Superior Performance**: The Random Forest model significantly outperformed all other models across various metrics. It achieved the highest average ROC AUC of 0.9979 $\pm$ 0.0002, and strong F1-scores for both classes (0.9756 $\pm$ 0.0015 for Class -3 and 0.9753 $\pm$ 0.0014 for Class 0), indicating excellent overall classification capability and balanced performance for both classes.
- **SVM Shows Robust Performance with Minimal C Sensitivity**: The Support Vector Machine (SVM) model demonstrated strong performance, particularly with C=1, achieving an average ROC AUC of 0.9856 $\pm$ 0.0006 and F1-scores of 0.9428 $\pm$ 0.0016 (Class -3) and 0.9424 $\pm$ 0.0018 (Class 0). The performance of SVM remained highly stable across the tested C values (0.1, 1, 10, 100), with only minor variations, suggesting that the model is not overly sensitive to this hyperparameter within the explored range.
- **Naive Bayes and Logistic Regression Provide Moderate Performance**: Gaussian Naive Bayes and Logistic Regression models showed respectable performance but lagged behind Random Forest and SVM. Naive Bayes achieved an average ROC AUC of 0.9705 $\pm$ 0.0014 and F1-scores of 0.9130 $\pm$ 0.0022 (Class -3) and 0.9100 $\pm$ 0.0021 (Class 0). Logistic Regression had an average ROC AUC of 0.9603 $\pm$ 0.0020 and F1-scores of 0.9045 $\pm$ 0.0022 (Class -3) and 0.8994 $\pm$ 0.0023 (Class 0).
- **Decision Tree Offers Competitive Performance but Lower ROC AUC**: The Decision Tree model performed better than Logistic Regression and Naive Bayes in terms of F1-scores (0.9249 $\pm$ 0.0020 for Class -3 and 0.9225 $\pm$ 0.0023 for Class 0) and accuracy (0.9237 $\pm$ 0.0021), but its ROC AUC (0.9237 $\pm$ 0.0021) was the lowest among all models, indicating less effective discrimination ability compared to others.
- **High Performance Stability Across Models**: All models exhibited good performance stability, as evidenced by the consistently low standard deviations across all metrics in the 5-fold cross-validation. This suggests that the models generalize well to unseen data within the given SMOTE-transformed training distribution.

4.8.2 Insights or Next Steps

- **Identify Best Performing Model**: The **Random Forest Classifier** is the best-performing model based on the comprehensive evaluation, showing both high overall accuracy and strong F1-scores for both classes, coupled with the highest ROC AUC and excellent stability.
- **Further Optimization of Top Models**: For Random Forest and SVM, further hyperparameter tuning (e.g., `n_estimators`, `max_depth` for Random Forest; kernel types and `gamma` for SVM) could potentially yield marginal improvements. Additionally, investigating feature importance from the Random Forest model could offer insights into the most influential features for classification.

[ ]:
```
import pandas as pd
import numpy as np

# Helper function to format metrics
def format_metric(mean, std):
    return f"{mean:.4f} ± {std:.4f}"
```

```
# Data for the comparison table
comparison_data = {
    'Model': [],
    'Accuracy': [],
    'Precision (Class -3)': [],
    'Recall (Class -3)': [],
    'F1-Score (Class -3)': [],
    'Precision (Class 0)': [],
    'Recall (Class 0)': [],
    'F1-Score (Class 0)': [],
    'ROC AUC': []
}

# --- SVM (Using C=10 as representative best performance) ---
svm_c = 10
comparison_data['Model'].append(f'SVM (C={svm_c})')
comparison_data['Accuracy'].
 ↪append(format_metric(all_metrics['accuracy'][svm_c]['mean'],␣
 ↪all_metrics['accuracy'][svm_c]['std']))
comparison_data['Precision (Class -3)'].
 ↪append(format_metric(all_metrics['precision_class_0'][svm_c]['mean'],␣
 ↪all_metrics['precision_class_0'][svm_c]['std']))
comparison_data['Recall (Class -3)'].
 ↪append(format_metric(all_metrics['recall_class_0'][svm_c]['mean'],␣
 ↪all_metrics['recall_class_0'][svm_c]['std']))
comparison_data['F1-Score (Class -3)'].
 ↪append(format_metric(all_metrics['f1_class_0'][svm_c]['mean'],␣
 ↪all_metrics['f1_class_0'][svm_c]['std']))
comparison_data['Precision (Class 0)'].
 ↪append(format_metric(all_metrics['precision_class_1'][svm_c]['mean'],␣
 ↪all_metrics['precision_class_1'][svm_c]['std']))
comparison_data['Recall (Class 0)'].
 ↪append(format_metric(all_metrics['recall_class_1'][svm_c]['mean'],␣
 ↪all_metrics['recall_class_1'][svm_c]['std']))
comparison_data['F1-Score (Class 0)'].
 ↪append(format_metric(all_metrics['f1_class_1'][svm_c]['mean'],␣
 ↪all_metrics['f1_class_1'][svm_c]['std']))
comparison_data['ROC AUC'].
 ↪append(format_metric(all_metrics['roc_auc'][svm_c]['mean'],␣
 ↪all_metrics['roc_auc'][svm_c]['std']))

# --- Random Forest ---
comparison_data['Model'].append('Random Forest')
comparison_data['Accuracy'].append(format_metric(np.mean(rf_fold_accuracies),␣
 ↪np.std(rf_fold_accuracies)))
```

```
comparison_data['Precision (Class -3)'].append(format_metric(np.
 ↪mean(rf_fold_precisions_0), np.std(rf_fold_precisions_0)))
comparison_data['Recall (Class -3)'].append(format_metric(np.
 ↪mean(rf_fold_recalls_0), np.std(rf_fold_recalls_0)))
comparison_data['F1-Score (Class -3)'].append(format_metric(np.
 ↪mean(rf_fold_f1s_0), np.std(rf_fold_f1s_0)))
comparison_data['Precision (Class 0)'].append(format_metric(np.
 ↪mean(rf_fold_precisions_1), np.std(rf_fold_precisions_1)))
comparison_data['Recall (Class 0)'].append(format_metric(np.
 ↪mean(rf_fold_recalls_1), np.std(rf_fold_recalls_1)))
comparison_data['F1-Score (Class 0)'].append(format_metric(np.
 ↪mean(rf_fold_f1s_1), np.std(rf_fold_f1s_1)))
comparison_data['ROC AUC'].append(format_metric(np.mean(rf_fold_roc_aucs), np.
 ↪std(rf_fold_roc_aucs)))

# --- Naive Bayes ---
comparison_data['Model'].append('Naive Bayes')
comparison_data['Accuracy'].append(format_metric(np.mean(nb_fold_accuracies),␣
 ↪np.std(nb_fold_accuracies)))
comparison_data['Precision (Class -3)'].append(format_metric(np.
 ↪mean(nb_fold_precisions_0), np.std(nb_fold_precisions_0)))
comparison_data['Recall (Class -3)'].append(format_metric(np.
 ↪mean(nb_fold_recalls_0), np.std(nb_fold_recalls_0)))
comparison_data['F1-Score (Class -3)'].append(format_metric(np.
 ↪mean(nb_fold_f1s_0), np.std(nb_fold_f1s_0)))
comparison_data['Precision (Class 0)'].append(format_metric(np.
 ↪mean(nb_fold_precisions_1), np.std(nb_fold_precisions_1)))
comparison_data['Recall (Class 0)'].append(format_metric(np.
 ↪mean(nb_fold_recalls_1), np.std(nb_fold_recalls_1)))
comparison_data['F1-Score (Class 0)'].append(format_metric(np.
 ↪mean(nb_fold_f1s_1), np.std(nb_fold_f1s_1)))
comparison_data['ROC AUC'].append(format_metric(np.mean(nb_fold_roc_aucs), np.
 ↪std(nb_fold_roc_aucs)))

# --- Logistic Regression ---
comparison_data['Model'].append('Logistic Regression')
comparison_data['Accuracy'].append(format_metric(np.mean(lr_fold_accuracies),␣
 ↪np.std(lr_fold_accuracies)))
comparison_data['Precision (Class -3)'].append(format_metric(np.
 ↪mean(lr_fold_precisions_neg3), np.std(lr_fold_precisions_neg3)))
comparison_data['Recall (Class -3)'].append(format_metric(np.
 ↪mean(lr_fold_recalls_neg3), np.std(lr_fold_recalls_neg3)))
comparison_data['F1-Score (Class -3)'].append(format_metric(np.
 ↪mean(lr_fold_f1s_neg3), np.std(lr_fold_f1s_neg3)))
comparison_data['Precision (Class 0)'].append(format_metric(np.
 ↪mean(lr_fold_precisions_0), np.std(lr_fold_precisions_0)))
```

```
comparison_data['Recall (Class 0)'].append(format_metric(np.
 ↪mean(lr_fold_recalls_0), np.std(lr_fold_recalls_0)))
comparison_data['F1-Score (Class 0)'].append(format_metric(np.
 ↪mean(lr_fold_f1s_0), np.std(lr_fold_f1s_0)))
comparison_data['ROC AUC'].append(format_metric(np.mean(lr_fold_roc_aucs), np.
 ↪std(lr_fold_roc_aucs)))

# --- Decision Tree ---
comparison_data['Model'].append('Decision Tree')
comparison_data['Accuracy'].append(format_metric(np.mean(dt_fold_accuracies),␣
 ↪np.std(dt_fold_accuracies)))
comparison_data['Precision (Class -3)'].append(format_metric(np.
 ↪mean(dt_fold_precisions_neg3), np.std(dt_fold_precisions_neg3)))
comparison_data['Recall (Class -3)'].append(format_metric(np.
 ↪mean(dt_fold_recalls_neg3), np.std(dt_fold_recalls_neg3)))
comparison_data['F1-Score (Class -3)'].append(format_metric(np.
 ↪mean(dt_fold_f1s_neg3), np.std(dt_fold_f1s_neg3)))
comparison_data['Precision (Class 0)'].append(format_metric(np.
 ↪mean(dt_fold_precisions_0), np.std(dt_fold_precisions_0)))
comparison_data['Recall (Class 0)'].append(format_metric(np.
 ↪mean(dt_fold_recalls_0), np.std(dt_fold_recalls_0)))
comparison_data['F1-Score (Class 0)'].append(format_metric(np.
 ↪mean(dt_fold_f1s_0), np.std(dt_fold_f1s_0)))
comparison_data['ROC AUC'].append(format_metric(np.mean(dt_fold_roc_aucs), np.
 ↪std(dt_fold_roc_aucs)))

# Create the DataFrame
comparison_df = pd.DataFrame(comparison_data)

display(comparison_df)
```

```
                Model         Accuracy Precision (Class -3)  \
0           SVM (C=10)  0.9853 ± 0.0013      0.9717 ± 0.0025
1        Random Forest  0.9754 ± 0.0014      0.9703 ± 0.0018
2          Naive Bayes  0.9115 ± 0.0022      0.8980 ± 0.0033
3  Logistic Regression  0.9020 ± 0.0023      0.8820 ± 0.0028
4        Decision Tree  0.9237 ± 0.0021      0.9109 ± 0.0049

  Recall (Class -3) F1-Score (Class -3) Precision (Class 0) Recall (Class 0)  \
0   0.9999 ± 0.0002     0.9856 ± 0.0013     0.9999 ± 0.0002  0.9708 ± 0.0027
1   0.9809 ± 0.0033     0.9756 ± 0.0015     0.9807 ± 0.0033  0.9700 ± 0.0019
2   0.9285 ± 0.0050     0.9130 ± 0.0022     0.9261 ± 0.0046  0.8945 ± 0.0041
3   0.9282 ± 0.0036     0.9045 ± 0.0022     0.9242 ± 0.0034  0.8758 ± 0.0034
4   0.9394 ± 0.0045     0.9249 ± 0.0020     0.9375 ± 0.0041  0.9081 ± 0.0059

  F1-Score (Class 0)          ROC AUC
0    0.9851 ± 0.0013  0.9991 ± 0.0003
```

```
1    0.9753 ± 0.0014  0.9979 ± 0.0002
2    0.9100 ± 0.0021  0.9705 ± 0.0014
3    0.8994 ± 0.0023  0.9603 ± 0.0020
4    0.9225 ± 0.0023  0.9237 ± 0.0021
```

[2]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

[4]:
```
!apt-get update
!apt-get install -y texlive texlive-xetex texlive-fonts-recommended␣
  ↪texlive-latex-extra
!sudo apt-get update
!sudo apt-get install texlive-xetex texlive-fonts-recommended␣
  ↪texlive-plain-generic
!jupyter nbconvert --to pdf "/MyDrive/Colab Notebooks/Copy of␣
  ↪Copy_of_Copy_of_Machine_Learning_Project.ipynb"
```

```
 0% [Working]              Hit:1 http://security.ubuntu.com/ubuntu jammy-
security InRelease
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:3 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:4 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Fetched 3,917 B in 2s (2,478 B/s)
Reading package lists… Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists… Done
Building dependency tree… Done
Reading state information… Done
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-latex-extra set to manually installed.
texlive-xetex is already the newest version (2021.20220204-1).
The following NEW packages will be installed:
  texlive
0 upgraded, 1 newly installed, 0 to remove and 6 not upgraded.
Need to get 14.3 kB of archives.
After this operation, 73.7 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
```

```
Fetched 14.3 kB in 0s (42.4 kB/s)
Selecting previously unselected package texlive.
(Reading database … 154354 files and directories currently installed.)
Preparing to unpack …/texlive_2021.20220204-1_all.deb …
Unpacking texlive (2021.20220204-1) …
Setting up texlive (2021.20220204-1) …
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:2 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:3 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Fetched 3,917 B in 1s (3,478 B/s)
Reading package lists… Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists… Done
Building dependency tree… Done
Reading state information… Done
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-plain-generic is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 6 not upgraded.
[NbConvertApp] WARNING | pattern '/MyDrive/Colab Notebooks/Copy of
Copy_of_Copy_of_Machine_Learning_Project.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
```

```
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each
cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True
```

```
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the
system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful
for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR',
'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook',
'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
```

```
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                    to output to the directory of each notebook.
To recover
                                    previous default behaviour (outputting to the
current
                                    working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a
copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-
html-slideshow)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
```

```
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown',
'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides',
'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX
includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py
```

```
To see all available configurables, use `--help-all`.
```